



PyQGIS 3.40 developer cookbook

QGIS Project

Feb 24, 2026

CONTENTS

1	Introduction	3
1.1	Scripting in the Python Console	3
1.2	Python Plugins	4
1.2.1	Processing Plugins	5
1.3	Running Python code when QGIS starts	5
1.3.1	The <code>startup.py</code> file	5
1.3.2	The <code>PYQGIS_STARTUP</code> environment variable	5
1.3.3	The <code>--code</code> parameter	5
1.3.4	Additional arguments for Python	6
1.4	Python Applications	6
1.4.1	Using PyQGIS in standalone scripts	6
1.4.2	Using PyQGIS in custom applications	7
1.4.3	Running Custom Applications	8
1.5	Technical notes on PyQt and SIP	8
2	Loading Projects	9
2.1	Resolving bad paths	10
2.2	Using flags to speed up things	11
3	Loading Layers	13
3.1	Vector Layers	13
3.2	Raster Layers	16
3.3	QgsProject instance	18
4	Accessing the Table Of Contents (TOC)	21
4.1	The QgsProject class	21
4.2	QgsLayerTreeGroup class	22
5	Using Raster Layers	25
5.1	Layer Details	25
5.2	Renderer	26
5.2.1	Single Band Rasters	27
5.2.2	Multi Band Rasters	27
5.3	Query Values	28
5.4	Editing raster data	28
6	Using Vector Layers	29
6.1	Retrieving information about attributes	30
6.2	Iterating over Vector Layer	30
6.3	Selecting features	31
6.3.1	Accessing attributes	32
6.3.2	Iterating over selected features	32
6.3.3	Iterating over a subset of features	32
6.4	Modifying Vector Layers	33
6.4.1	Add Features	34

6.4.2	Delete Features	35
6.4.3	Modify Features	35
6.4.4	Modifying Vector Layers with an Editing Buffer	35
6.4.5	Adding and Removing Fields	37
6.5	Using Spatial Index	37
6.6	The QgsVectorLayerUtils class	38
6.7	Creating Vector Layers	39
6.7.1	From an instance of QgsVectorFileWriter	39
6.7.2	Directly from features	40
6.7.3	From an instance of QgsVectorLayer	41
6.8	Appearance (Symbology) of Vector Layers	42
6.8.1	Single Symbol Renderer	43
6.8.2	Categorized Symbol Renderer	44
6.8.3	Graduated Symbol Renderer	45
6.8.4	Working with Symbols	46
6.8.5	Creating Custom Renderers	49
6.9	Further Topics	51
7	Geometry Handling	53
7.1	Geometry Construction	53
7.2	Access to Geometry	54
7.3	Geometry Predicates and Operations	56
8	Projections Support	59
8.1	Coordinate reference systems	59
8.2	CRS Transformation	60
9	Using the Map Canvas	63
9.1	Embedding Map Canvas	64
9.2	Rubber Bands and Vertex Markers	65
9.3	Using Map Tools with Canvas	66
9.3.1	Select a feature using QgsMapToolIdentifyFeature	67
9.3.2	Add items to map canvas contextual menu	67
9.4	Writing Custom Map Tools	67
9.5	Writing Custom Map Canvas Items	69
10	Map Rendering and Printing	71
10.1	Simple Rendering	72
10.2	Rendering layers with different CRS	72
10.3	Output using print layout	72
10.3.1	Checking layout validity	74
10.3.2	Exporting the layout	75
10.3.3	Exporting a layout atlas	75
11	Expressions, Filtering and Calculating Values	77
11.1	Parsing Expressions	78
11.2	Evaluating Expressions	78
11.2.1	Basic Expressions	78
11.2.2	Expressions with features	79
11.2.3	Filtering a layer with expressions	80
11.3	Handling expression errors	80
12	Reading And Storing Settings	83
13	Communicating with the user	85
13.1	Showing messages. The QgsMessageBar class	85
13.2	Showing progress	88
13.3	Logging	88
13.3.1	QgsMessageLog	89

13.3.2	The python built in logging module	89
14	Authentication infrastructure	91
14.1	Introduction	91
14.2	Glossary	92
14.3	QgsAuthManager the entry point	92
14.3.1	Init the manager and set the master password	92
14.3.2	Populate authdb with a new Authentication Configuration entry	93
14.3.3	Remove an entry from authdb	94
14.3.4	Leave authcfg expansion to QgsAuthManager	94
14.4	Adapt plugins to use Authentication infrastructure	95
14.5	Authentication GUIs	96
14.5.1	GUI to select credentials	96
14.5.2	Authentication Editor GUI	96
14.5.3	Authorities Editor GUI	97
15	Tasks - doing heavy work in the background	99
15.1	Introduction	99
15.2	Examples	101
15.2.1	Extending QgsTask	101
15.2.2	Task from function	103
15.2.3	Task from a processing algorithm	104
16	Developing Python Plugins	107
16.1	Structuring Python Plugins	107
16.1.1	Getting started	107
16.1.2	Writing plugin code	108
16.1.3	Documenting plugins	113
16.1.4	Translating plugins	113
16.1.5	Sharing your plugin	115
16.1.6	Tips and Tricks	115
16.2	Code Snippets	116
16.2.1	How to call a method by a key shortcut	117
16.2.2	How to reuse QGIS icons	117
16.2.3	Interface for plugin in the options dialog	117
16.2.4	Embed custom widgets for layers in the layer tree	119
16.3	IDE settings for writing and debugging plugins	120
16.3.1	Useful plugins for writing Python plugins	120
16.3.2	A note on configuring your IDE on Linux and Windows	120
16.3.3	Debugging using Pyscripter IDE (Windows)	120
16.3.4	Debugging using Eclipse and PyDev	121
16.3.5	Debugging with PyCharm on Ubuntu with a compiled QGIS	125
16.3.6	Debugging using PDB	126
16.4	Releasing your plugin	127
16.4.1	Metadata and names	127
16.4.2	Code and help	127
16.4.3	Official Python plugin repository	128
17	Writing a Processing plugin	131
17.1	Creating from scratch	131
17.2	Updating a plugin	131
17.3	Implementing custom Processing algorithms	133
17.3.1	Creating a custom algorithm	133
17.3.2	Customizing the algorithm dialog	134
17.3.3	Managing Qt Signals	135
18	Using Plugin Layers	137
18.1	Subclassing QgsPluginLayer	137

19	Network analysis library	139
19.1	General information	139
19.2	Building a graph	139
19.3	Graph analysis	141
19.3.1	Finding shortest paths	143
19.3.2	Areas of availability	145
20	QGIS Server and Python	147
20.1	Introduction	147
20.2	Server API basics	147
20.3	Standalone or embedding	148
20.4	Server plugins	149
20.4.1	Server filter plugins	149
20.4.2	Custom services	157
20.4.3	Custom APIs	158
21	Cheat sheet for PyQGIS	161
21.1	User Interface	161
21.2	Settings	162
21.3	Toolbars	162
21.4	Menus	162
21.5	Canvas	162
21.6	Layers	163
21.7	Table of contents	166
21.8	Advanced TOC	167
21.9	Processing algorithms	170
21.10	Decorators	171
21.11	Composer	172
21.12	Sources	172

INTRODUCTION

This document is intended to be both a tutorial and a reference guide. While it does not list all possible use cases, it should give a good overview of the principal functionality.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.

A copy of the license is included in the section `gnu_fdl`.

This license also applies to all code snippets in this document.

Python support was first introduced in QGIS 0.9. There are several ways to use Python in QGIS Desktop (covered in the following sections):

- Issue commands in the Python console within QGIS
- Create and use plugins
- Automatically run Python code when QGIS starts
- Create processing algorithms
- Create functions for expressions in QGIS
- Create custom applications based on the QGIS API

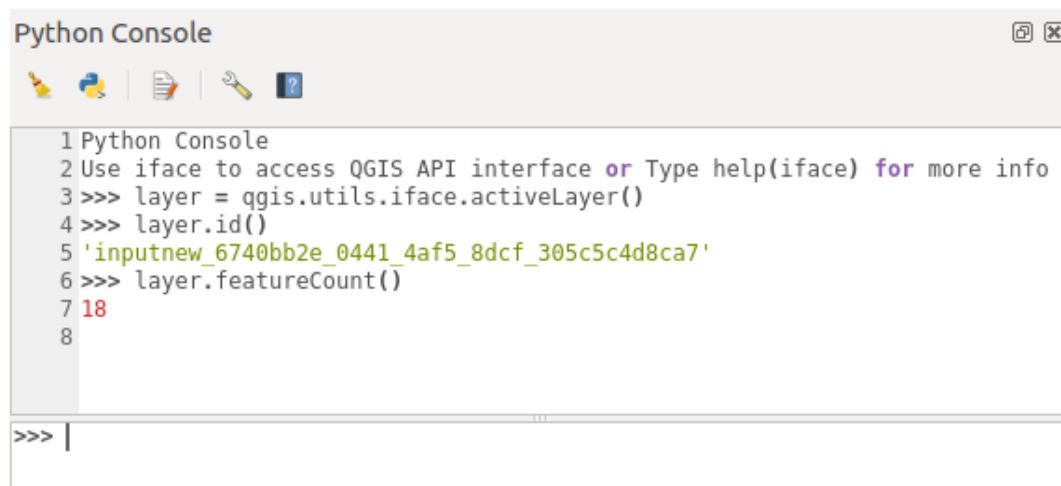
Python bindings are also available for QGIS Server, including Python plugins (see [QGIS Server and Python](#)) and Python bindings that can be used to embed QGIS Server into a Python application.

There is a [complete QGIS C++ API](#) reference that documents the classes from the QGIS libraries. [The Pythonic QGIS API \(pyqgis\)](#) is nearly identical to the C++ API.

Another good resource for learning how to perform common tasks is to download existing plugins from the [plugin repository](#) and examine their code.

1.1 Scripting in the Python Console

QGIS provides an integrated Python console for scripting. It can be opened from the *Plugins ► Python Console* menu:



```
1 Python Console
2 Use iface to access QGIS API interface or Type help(iface) for more info
3 >>> layer = qgis.utils.iface.activeLayer()
4 >>> layer.id()
5 'inputnew_6740bb2e_0441_4af5_8dcf_305c5c4d8ca7'
6 >>> layer.featureCount()
7 18
8
>>> |
```

Fig. 1.1: QGIS Python console

The screenshot above illustrates how to get the layer currently selected in the layer list, show its ID and optionally, if it is a vector layer, show the feature count. For interaction with the QGIS environment, there is an `iface` variable, which is an instance of `QgisInterface`. This interface allows access to the map canvas, menus, toolbars and other parts of the QGIS application.

For user convenience, the following statements are executed when the console is started (in the future it will be possible to set further initial commands)

```
from qgis.core import *
import qgis.utils
```

For those which use the console often, it may be useful to set a shortcut for triggering the console (within *Settings ► Keyboard shortcuts...*)

1.2 Python Plugins

The functionality of QGIS can be extended using plugins. Plugins can be written in Python. The main advantage over C++ plugins is simplicity of distribution (no compiling for each platform) and easier development.

Many plugins covering various functionality have been written since the introduction of Python support. The plugin installer allows users to easily fetch, upgrade and remove Python plugins. See the [Python Plugins](#) page for more information about plugins and plugin development.

Creating plugins in Python is simple, see [Developing Python Plugins](#) for detailed instructions.

Note

Python plugins are also available for QGIS server. See [QGIS Server and Python](#) for further details.

1.2.1 Processing Plugins

Processing Plugins can be used to process data. They are easier to develop, more specific and more lightweight than Python Plugins. *Writing a Processing plugin* explains when the use of Processing algorithms is appropriate and how to develop them.

1.3 Running Python code when QGIS starts

There are different methods to run Python code every time QGIS starts.

1. Creating a startup.py script
2. Setting the PYQGIS_STARTUP environment variable to an existing Python file
3. Specifying a startup script using the `--code init_qgis.py` parameter.

1.3.1 The startup.py file

Every time QGIS starts, the user's Python home directory and a list of system paths are searched for a file named `startup.py`. If that file exists, it is executed by the embedded Python interpreter.

The path in the user's home directory usually is found under:

- Linux: `.local/share/QGIS/QGIS3`
- Windows: `AppData\Roaming\QGIS\QGIS3`
- macOS: `Library/Application Support/QGIS/QGIS3`

The default system paths depend on the operating system. To find the paths that work for you, open the Python Console and run `QStandardPaths.standardLocations(QStandardPaths.AppDataLocation)` to see the list of default directories.

The `startup.py` script is executed immediately upon initializing python in QGIS, early on in the start of the application.

1.3.2 The PYQGIS_STARTUP environment variable

You can run Python code just before QGIS initialization completes by setting the `PYQGIS_STARTUP` environment variable to the path of an existing Python file.

This code will run before QGIS initialization is complete. This method is very useful for cleaning `sys.path`, which may have undesirable paths, or for isolating/loading the initial environment without requiring a virtual environment, e.g. homebrew or MacPorts installs on Mac.

1.3.3 The --code parameter

You can provide custom code to execute as startup parameter to QGIS. To do so, create a python file, for example `qgis_init.py`, to execute and start QGIS from the command line using `qgis --code qgis_init.py`.

Code provided via `--code` is executed late in the QGIS initialization phase, after the application components have been loaded.

1.3.4 Additional arguments for Python

To provide additional arguments for your `--code` script or for other python code that is executed, you can use the `--py-args` argument. Any argument coming after `--py-args` and before a `-- arg` (if present) will be passed to Python but ignored by the QGIS application itself.

In the following example, `myfile.tif` will be available via `sys.argv` in Python but will not be loaded by QGIS. Whereas `otherfile.tif` will be loaded by QGIS but is not present in `sys.argv`.

```
qgis --code qgis_init.py --py-args myfile.tif -- otherfile.tif
```

If you want access to every command line parameter from within Python, you can use `QCoreApplication.arguments()`

```
QgsApplication.instance().arguments()
```

1.4 Python Applications

It is often handy to create scripts for automating processes. With PyQGIS, this is perfectly possible — import the `qgis.core` module, initialize it and you are ready for the processing.

Or you may want to create an interactive application that uses GIS functionality — perform measurements, export a map as PDF, ... The `qgis.gui` module provides various GUI components, most notably the map canvas widget that can be incorporated into the application with support for zooming, panning and/or any further custom map tools.

PyQGIS custom applications or standalone scripts must be configured to locate the QGIS resources, such as projection information and providers for reading vector and raster layers. QGIS Resources are initialized by adding a few lines to the beginning of your application or script. The code to initialize QGIS for custom applications and standalone scripts is similar. Examples of each are provided below.

Note

Do *not* use `qgis.py` as a name for your script. Python will not be able to import the bindings as the script's name will shadow them.

1.4.1 Using PyQGIS in standalone scripts

To start a standalone script, initialize the QGIS resources at the beginning of the script:

```
1 from qgis.core import *
2
3 # Supply path to qgis install location
4 QgsApplication.setPrefixPath("/path/to/qgis/installation", True)
5
6 # Create a reference to the QgsApplication. Setting the
7 # second argument to False disables the GUI.
8 qgs = QgsApplication([], False)
9
10 # Load providers
11 qgs.initQgis()
12
13 # Write your code here to load some layers, use processing
14 # algorithms, etc.
15
16 # Finally, exitQgis() is called to remove the
17 # provider and layer registries from memory
18 qgs.exitQgis()
```

First we import the `qgis.core` module and configure the prefix path. The prefix path is the location where QGIS is installed on your system. It is configured in the script by calling the `setPrefixPath()` method. The second argument of `setPrefixPath()` is set to `True`, specifying that default paths are to be used.

The QGIS install path varies by platform; the easiest way to find it for your system is to use the *Scripting in the Python Console* from within QGIS and look at the output from running:

```
QgsApplication.prefixPath()
```

After the prefix path is configured, we save a reference to `QgsApplication` in the variable `qgs`. The second argument is set to `False`, specifying that we do not plan to use the GUI since we are writing a standalone script. With `QgsApplication` configured, we load the QGIS data providers and layer registry by calling the `initQgis()` method.

```
qgs.initQgis()
```

With QGIS initialized, we are ready to write the rest of the script. Finally, we wrap up by calling `exitQgis()` to remove the data providers and layer registry from memory.

```
qgs.exitQgis()
```

1.4.2 Using PyQGIS in custom applications

The only difference between *Using PyQGIS in standalone scripts* and a custom PyQGIS application is the second argument when instantiating the `QgsApplication`. Pass `True` instead of `False` to indicate that we plan to use a GUI.

```

1 from qgis.core import *
2
3 # Supply the path to the qgis install location
4 QgsApplication.setPrefixPath("/path/to/qgis/installation", True)
5
6 # Create a reference to the QgsApplication.
7 # Setting the second argument to True enables the GUI. We need
8 # this since this is a custom application.
9
10 qgs = QgsApplication([], True)
11
12 # load providers
13 qgs.initQgis()
14
15 # Write your code here to load some layers, use processing
16 # algorithms, etc.
17
18 # Finally, exitQgis() is called to remove the
19 # provider and layer registries from memory
20 qgs.exitQgis()
```

Now you can work with the QGIS API - load layers and do some processing or fire up a GUI with a map canvas. The possibilities are endless :-)

1.4.3 Running Custom Applications

You need to tell your system where to search for QGIS libraries and appropriate Python modules if they are not in a well-known location - otherwise Python will complain:

```
>>> import qgis.core
ImportError: No module named qgis.core
```

This can be fixed by setting the `PYTHONPATH` environment variable. In the following commands, `<qgispath>` should be replaced with your actual QGIS installation path:

- on Linux: **export PYTHONPATH=/<qgispath>/share/qgis/python**
- on Windows: **set PYTHONPATH=c:\<qgispath>\python**
- on macOS: **export PYTHONPATH=/<qgispath>/Contents/Resources/python**

Now, the path to the PyQGIS modules is known, but they depend on the `qgis_core` and `qgis_gui` libraries (the Python modules serve only as wrappers). The path to these libraries may be unknown to the operating system, and then you will get an import error again (the message might vary depending on the system):

```
>>> import qgis.core
ImportError: libqgis_core.so.3.2.0: cannot open shared object file:
  No such file or directory
```

Fix this by adding the directories where the QGIS libraries reside to the search path of the dynamic linker:

- on Linux: **export LD_LIBRARY_PATH=/<qgispath>/lib**
- on Windows: **set PATH=C:\<qgispath>\bin;C:\<qgispath>\apps\<qgisrelease>\bin;%PATH%** where `<qgisrelease>` should be replaced with the type of release you are targeting (eg, `qgis-ltr`, `qgis`, `qgis-dev`)

These commands can be put into a bootstrap script that will take care of the startup. When deploying custom applications using PyQGIS, there are usually two possibilities:

- require the user to install QGIS prior to installing your application. The application installer should look for default locations of QGIS libraries and allow the user to set the path if not found. This approach has the advantage of being simpler, however it requires the user to do more steps.
- package QGIS together with your application. Releasing the application may be more challenging and the package will be larger, but the user will be saved from the burden of downloading and installing additional pieces of software.

The two deployment models can be mixed. You can provide a standalone applications on Windows and macOS, but for Linux leave the installation of GIS up to the user and his package manager.

1.5 Technical notes on PyQt and SIP

We've decided for Python as it's one of the most favoured languages for scripting. PyQGIS bindings in QGIS 3 depend on SIP and PyQt5. The reason for using SIP instead of the more widely used SWIG is that the QGIS code depends on Qt libraries. Python bindings for Qt (PyQt) are done using SIP and this allows seamless integration of PyQGIS with PyQt.

LOADING PROJECTS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     Qgis,  
3     QgsProject,  
4     QgsPathResolver  
5 )  
6  
7 from qgis.gui import (  
8     QgsLayerTreeMapCanvasBridge,  
9 )
```

Sometimes you need to load an existing project from a plugin or (more often) when developing a standalone QGIS Python application (see: [Python Applications](#)).

To load a project into the current QGIS application you need to create an instance of the `QgsProject` class. This is a singleton class, so you must use its `instance()` method to do it. You can call its `read()` method, passing the path of the project to be loaded:

```
1 # If you are not inside a QGIS console you first need to import  
2 # qgis and PyQt classes you will use in this script as shown below:  
3 from qgis.core import QgsProject  
4 # Get the project instance  
5 project = QgsProject.instance()  
6 # Print the current project file name (might be empty in case no projects have  
7 #   ↳ been loaded)  
8 # print(project.fileName())  
9  
10 # Load another project  
11 project.read('testdata/01_project.qgs')  
print(project.fileName())
```

```
testdata/01_project.qgs
```

If you need to make modifications to the project (for example to add or remove some layers) and save your changes, call the `write()` method of your project instance. The `write()` method also accepts an optional path for saving the project to a new location:

```
# Save the project to the same  
project.write()  
# ... or to a new file  
project.write('testdata/my_new_qgis_project.qgs')
```

Both `read()` and `write()` functions return a boolean value that you can use to check if the operation was successful.

Note

If you are writing a QGIS standalone application, in order to synchronise the loaded project with the canvas you need to instantiate a `QgsLayerTreeMapCanvasBridge` as in the example below:

```
bridge = QgsLayerTreeMapCanvasBridge( \
    QgsProject.instance().layerTreeRoot(), canvas)
# Now you can safely load your project and see it in the canvas
project.read('testdata/my_new_qgis_project.qgs')
```

2.1 Resolving bad paths

It can happen that layers loaded in the project are moved to another location. When the project is loaded again all the layer paths are broken. The `QgsPathResolver` class helps you rewrite layers path within the project.

Its `setPathPreprocessor()` method allows setting a custom path pre-processor function to manipulate paths and data sources prior to resolving them to file references or layer sources.

The processor function must accept a single string argument (representing the original file path or data source) and return a processed version of this path. The path pre-processor function is called **before** any bad layer handler. If multiple preprocessors are set, they will be called in sequence based on the order in which they were originally set.

Some use cases:

1. replace an outdated path:

```
def my_processor(path):
    return path.replace('c:/Users/ClintBarton/Documents/Projects', 'x:/
↳Projects/')

QgsPathResolver.setPathPreprocessor(my_processor)
```

2. replace a database host address with a new one:

```
def my_processor(path):
    return path.replace('host=10.1.1.115', 'host=10.1.1.116')

QgsPathResolver.setPathPreprocessor(my_processor)
```

3. replace stored database credentials with new ones:

```
1 def my_processor(path):
2     path= path.replace("user='gis_team'", "user='team_awesome'")
3     path = path.replace("password='cats'", "password='g7as!m*")
4     return path
5
6 QgsPathResolver.setPathPreprocessor(my_processor)
```

Likewise, a `setPathWriter()` method is available for a path writer function.

An example to replace the path with a variable:

```
def my_processor(path):
    return path.replace('c:/Users/ClintBarton/Documents/Projects', '$projectdir$')

QgsPathResolver.setPathWriter(my_processor)
```

Both methods return an id that can be used to remove the pre-processor or writer they added. See `removePathPreprocessor()` and `removePathWriter()`.

2.2 Using flags to speed up things

In some instances where you may not need to use a fully functional project, but only want to access it for a specific reason, flags may be helpful. A full list of flags is available under `ProjectReadFlag`. Multiple flags can be added together.

As an example, if we do not care about actual layers and data and simply want to access a project (e.g. for layout or 3D view settings), we can use `DontResolveLayers` flag to bypass the data validation step and prevent the bad layer dialog from appearing. The following can be done:

```
readflags = Qgs.ProjectReadFlags()
readflags |= Qgs.ProjectReadFlag.DontResolveLayers
project = QgsProject.instance()
project.read('C:/Users/ClintBarton/Documents/Projects/mysweetproject.qgs',
↳readflags)
```

To add more flags the python Bitwise OR operator (`|`) must be used.

LOADING LAYERS

Hint

The code snippets on this page need the following imports:

```
import os # This is is needed in the pyqgis console also
from qgis.core import (
    QgsVectorLayer
)
```

Let's open some layers with data. QGIS recognizes vector and raster layers. Additionally, custom layer types are available, but we are not going to discuss them here.

3.1 Vector Layers

To create and add a vector layer instance to the project, specify the layer's data source identifier. The data source identifier is a string and it is specific to each vector data provider. An optional layer name is used for identifying the layer in the *Layers* panel. It is important to check whether the layer has been loaded successfully. If it was not, an invalid layer instance is returned.

For a geopackage vector layer:

```
1 # get the path to a geopackage
2 path_to_gpkg = "testdata/data/data.gpkg"
3 # append the layername part
4 gpkg_airports_layer = path_to_gpkg + "|layername=airports"
5 vlayer = QgsVectorLayer(gpkg_airports_layer, "Airports layer", "ogr")
6 if not vlayer.isValid():
7     print("Layer failed to load!")
8 else:
9     QgsProject.instance().addMapLayer(vlayer)
```

The quickest way to open and display a vector layer in QGIS is the `addVectorLayer()` method of the `Qgis-Interface` class:

```
vlayer = iface.addVectorLayer(gpkg_airports_layer, "Airports layer", "ogr")
if not vlayer:
    print("Layer failed to load!")
```

This creates a new layer and adds it to the current QGIS project (making it appear in the layer list) in one step. The function returns the layer instance or `None` if the layer couldn't be loaded.

The following list shows how to access various data sources using vector data providers:

- The ogr provider from the GDAL library supports a [wide variety of formats](#), also called drivers in GDAL speak. Examples are ESRI Shapefile, Geopackage, Flatgeobuf, Geojson, ... For single-file formats the filepath usually suffices as uri. For geopackages or dxf, a pipe separated suffix allows to specify the layer to load.

– for ESRI Shapefile:

```
uri = "testdata/airports.shp"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

– for Geopackage (note the internal options in data source uri):

```
uri = "testdata/data/data.gpkg|layername=airports"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

Note that if your GeoPackage contains relationships between layers. Simply using `addMapLayer` does not automatically create these relationships between the layers in your QGIS project. Add the following code to automatically find the relationships in the database and add appropriate ones to the project properties.

```
1 from qgis.core import QgsRelationManager
2
3 project = QgsProject.instance()
4 relation_manager = project.relationManager()
5 existing_relations = list(relation_manager.relations().values())
6
7 layers = [
8     layer for layer in project.mapLayers().values() if layer.type() ==
9     ↳ layer.VectorLayer
10 ]
11
12 discovered_relations = QgsRelationManager.discoverRelations(existing_
13     ↳ relations, layers)
14
15 for relation in discovered_relations:
16     relation_manager.addRelation(relation)
```

– for dxf (note the internal options in data source uri):

```
uri = "testdata/sample.dxf|layername=entities|geometrytype=Polygon"
vlayer = QgsVectorLayer(uri, "layer_name_you_like", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

- PostGIS database - data source is a string with all information needed to create a connection to PostgreSQL database.

`QgsDataSourceUri` class can generate this string for you. Note that QGIS has to be compiled with Postgres support, otherwise this provider isn't available:

```
1 uri = QgsDataSourceUri()
2 # set host name, port, database name, username and password
3 uri.setConnection("localhost", "5432", "dbname", "johnny", "xxx")
4 # set database schema, table name, geometry column and optionally
5 # subset (WHERE clause)
6 uri.setDataSource("public", "roads", "the_geom", "cityid = 2643", "primary_key_
7     ↳ field")
8
9 vlayer = QgsVectorLayer(uri.uri(False), "layer name you like", "postgres")
```

Note

The `False` argument passed to `uri.uri(False)` prevents the expansion of the authentication configuration parameters, if you are not using any authentication configuration this argument does not make any difference.

- CSV or other delimited text files — to open a file with a semicolon as a delimiter, with field “x” for X coordinate and field “y” for Y coordinate you would use something like this:

```
uri = "file://{}/testdata/delimited_xy.csv?delimiter={}&xField={}&yField={}".
    ↪format(os.getcwd(), ";", "x", "y")
vlayer = QgsVectorLayer(uri, "layer name you like", "delimitedtext")
QgsProject.instance().addMapLayer(vlayer)
```

Note

The provider string is structured as a URL, so the path must be prefixed with `file://`. Also it allows WKT (well known text) formatted geometries as an alternative to x and y fields, and allows the coordinate reference system to be specified. For example:

```
uri = "file:///some/path/file.csv?delimiter={}&crs=epsg:4723&wktField={}".
    ↪format(";", "shape")
```

- GPX files — the “gpx” data provider reads tracks, routes and waypoints from gpx files. To open a file, the type (track/route/waypoint) needs to be specified as part of the url:

```
uri = "testdata/layers.gpx?type=track"
vlayer = QgsVectorLayer(uri, "layer name you like", "gpx")
QgsProject.instance().addMapLayer(vlayer)
```

- Spatialite database — Similarly to PostGIS databases, `QgsDataSourceUri` can be used for generation of data source identifier:

```
1 uri = QgsDataSourceUri()
2 uri.setDatabase('/home/martin/test-2.3.sqlite')
3 schema = ''
4 table = 'Towns'
5 geom_column = 'Geometry'
6 uri.setDataSource(schema, table, geom_column)
7
8 display_name = 'Towns'
9 vlayer = QgsVectorLayer(uri.uri(), display_name, 'spatialite')
10 QgsProject.instance().addMapLayer(vlayer)
```

- MySQL WKB-based geometries, through GDAL — data source is the connection string to the table:

```
uri = "MySQL:dbname,host=localhost,port=3306,user=root,
    ↪password=xxx|layername=my_table"
vlayer = QgsVectorLayer(uri, "my table", "ogr")
QgsProject.instance().addMapLayer(vlayer)
```

- WFS connection: the connection is defined with a URI and using the WFS provider:

```
uri = "https://demo.mapserver.org/cgi-bin/wfs?service=WFS&version=2.0.0&
    ↪request=GetFeature&typename=ms:cities"
vlayer = QgsVectorLayer(uri, "my wfs layer", "WFS")
```

The uri can be created using the standard `urllib` library:

```

1 import urllib
2
3 params = {
4     'service': 'WFS',
5     'version': '2.0.0',
6     'request': 'GetFeature',
7     'typename': 'ms:cities',
8     'srsname': "EPSG:4326"
9 }
10 uri2 = 'https://demo.mapserver.org/cgi-bin/wfs?' + urllib.parse.unquote(urllib.
    ↳ parse.urlencode(params))

```

Note

You can change the data source of an existing layer by calling `setDataSource()` on a `QgsVectorLayer` instance, as in the following example:

```

1 uri = "https://demo.mapserver.org/cgi-bin/wfs?service=WFS&version=2.0.0&
    ↳ request=GetFeature&typename=ms:cities"
2 provider_options = QgsDataProvider.ProviderOptions()
3 # Use project's transform context
4 provider_options.transformContext = QgsProject.instance().transformContext()
5 vlayer.setDataSource(uri, "layer name you like", "WFS", provider_options)
6
7 del(vlayer)

```

3.2 Raster Layers

For accessing raster files, GDAL library is used. It supports a wide range of file formats. In case you have troubles with opening some files, check whether your GDAL has support for the particular format (not all formats are available by default). To load a raster from a file, specify its filename and display name:

```

1 # get the path to a tif file e.g. /home/project/data/srtm.tif
2 path_to_tif = "qgis-projects/python_cookbook/data/srtm.tif"
3 rlayer = QgsRasterLayer(path_to_tif, "SRTM layer name")
4 if not rlayer.isValid():
5     print("Layer failed to load!")

```

To load a raster from a geopackage:

```

1 # get the path to a geopackage e.g. /home/project/data/data.gpkg
2 path_to_gpkg = os.path.join(os.getcwd(), "testdata", "sublayers.gpkg")
3 # gpkg_raster_layer = "GPKG:/home/project/data/data.gpkg:srtm"
4 gpkg_raster_layer = "GPKG:" + path_to_gpkg + ":srtm"
5
6 rlayer = QgsRasterLayer(gpkg_raster_layer, "layer name you like", "gdal")
7
8 if not rlayer.isValid():
9     print("Layer failed to load!")

```

Similarly to vector layers, raster layers can be loaded using the `addRasterLayer` function of the `QgisInterface` object:

```
iface.addRasterLayer(path_to_tif, "layer name you like")
```

This creates a new layer and adds it to the current project (making it appear in the layer list) in one step.

To load a PostGIS raster:

PostGIS rasters, similar to PostGIS vectors, can be added to a project using a URI string. It is efficient to keep a reusable dictionary of strings for the database connection parameters. This makes it easy to edit the dictionary for the applicable connection. The dictionary is then encoded into a URI using the 'postgresraster' provider metadata object. After that the raster can be added to the project.

```

1 uri_config = {
2     # database parameters
3     'dbname': 'gis_db',          # The PostgreSQL database to connect to.
4     'host': 'localhost',        # The host IP address or localhost.
5     'port': '5432',             # The port to connect on.
6     'sslmode': QgsDataSourceUri.SslDisable, # SslAllow, SslPrefer, SslRequire,
    ↪ SslVerifyCa, SslVerifyFull
7     # user and password are not needed if stored in the authcfg or service
8     'authcfg': 'QconfigId',     # The QGIS authentication database ID holding
    ↪ connection details.
9     'service': None,            # The PostgreSQL service to be used for connection to
    ↪ the database.
10    'username': None,            # The PostgreSQL user name.
11    'password': None,            # The PostgreSQL password for the user.
12    # table and raster column details
13    'schema': 'public',          # The database schema that the table is located in.
14    'table': 'my_rasters',       # The database table to be loaded.
15    'geometrycolumn': 'rast',    # raster column in PostGIS table
16    'sql': None,                 # An SQL WHERE clause. It should be placed at the end
    ↪ of the string.
17    'key': None,                 # A key column from the table.
18    'srid': None,                # A string designating the SRID of the coordinate
    ↪ reference system.
19    'estimatedmetadata': 'False', # A boolean value telling if the metadata is
    ↪ estimated.
20    'type': None,                # A WKT string designating the WKB Type.
21    'selectatid': None,          # Set to True to disable selection by feature ID.
22    'options': None,             # other PostgreSQL connection options not in this list.
23    'enableTime': None,
24    'temporalDefaultTime': None,
25    'temporalFieldIndex': None,
26    'mode': '2',                 # GDAL 'mode' parameter, 2 unions raster tiles, 1 adds
    ↪ tiles separately (may require user input)
27 }
28 # remove any NULL parameters
29 uri_config = {key:val for key, val in uri_config.items() if val is not None}
30 # get the metadata for the raster provider and configure the URI
31 md = QgsProviderRegistry.instance().providerMetadata('postgresraster')
32 uri = QgsDataSourceUri(md.encodeUri(uri_config))
33
34 # the raster can then be loaded into the project
35 rlayer = iface.addRasterLayer(uri.uri(False), "raster layer name", "postgresraster
    ↪ ")

```

Raster layers can also be created from a WCS service:

```

layer_name = 'modis'
url = "https://demo.mapserver.org/cgi-bin/wcs?identifier={}".format(layer_name)
rlayer = QgsRasterLayer(uri, 'my wcs layer', 'wcs')

```

Here is a description of the parameters that the WCS URI can contain:

WCS URI is composed of **key=value** pairs separated by &. It is the same format like query string in URL, encoded the same way. `QgsDataSourceUri` should be used to construct the URI to ensure that special characters are encoded properly.

- **url** (required) : WCS Server URL. Do not use VERSION in URL, because each version of WCS is using different parameter name for **GetCapabilities** version, see param version.

- **identifier** (required) : Coverage name
- **time** (optional) : time position or time period (beginPosition/endPosition[/timeResolution])
- **format** (optional) : Supported format name. Default is the first supported format with tif in name or the first supported format.
- **crs** (optional) : CRS in form AUTHORITY:ID, e.g. EPSG:4326. Default is EPSG:4326 if supported or the first supported CRS.
- **username** (optional) : Username for basic authentication.
- **password** (optional) : Password for basic authentication.
- **IgnoreGetMapUrl** (optional, hack) : If specified (set to 1), ignore GetCoverage URL advertised by GetCapabilities. May be necessary if a server is not configured properly.
- **InvertAxisOrientation** (optional, hack) : If specified (set to 1), switch axis in GetCoverage request. May be necessary for geographic CRS if a server is using wrong axis order.
- **IgnoreAxisOrientation** (optional, hack) : If specified (set to 1), do not invert axis orientation according to WCS standard for geographic CRS.
- **cache** (optional) : cache load control, as described in QNetworkRequest::CacheLoadControl, but request is re-send as PreferCache if failed with AlwaysCache. Allowed values: AlwaysCache, PreferCache, PreferNetwork, AlwaysNetwork. Default is AlwaysCache.

Alternatively you can load a raster layer from WMS server. However currently it's not possible to access GetCapabilities response from API — you have to know what layers you want:

```
urlWithParams = "crs=EPSG:4326&format=image/png&layers=continents&styles&
url=https://demo.mapserver.org/cgi-bin/wms"
rlayer = QgsRasterLayer(urlWithParams, 'some layer name', 'wms')
if not rlayer.isValid():
    print("Layer failed to load!")
```

3.3 QgsProject instance

If you would like to use the opened layers for rendering, do not forget to add them to the `QgsProject` instance. The `QgsProject` instance takes ownership of layers and they can be later accessed from any part of the application by their unique ID. When the layer is removed from the project, it gets deleted, too. Layers can be removed by the user in the QGIS interface, or via Python using the `removeMapLayer()` method.

Adding a layer to the current project is done using the `addMapLayer()` method:

```
QgsProject.instance().addMapLayer(rlayer)
```

To add a layer at an absolute position:

```
1 # first add the layer without showing it
2 QgsProject.instance().addMapLayer(rlayer, False)
3 # obtain the layer tree of the top-level group in the project
4 layerTree = iface.layerTreeCanvasBridge().rootGroup()
5 # the position is a number starting from 0, with -1 an alias for the end
6 layerTree.insertChildNode(-1, QgsLayerTreeLayer(rlayer))
```

If you want to delete the layer use the `removeMapLayer()` method:

```
# QgsProject.instance().removeMapLayer(layer_id)
QgsProject.instance().removeMapLayer(rlayer.id())
```

In the above code, the layer id is passed (you can get it calling the `id()` method of the layer), but you can also pass the layer object itself.

For a list of loaded layers and layer ids, use the `mapLayers()` method:

```
QgsProject.instance().mapLayers()
```


ACCESSING THE TABLE OF CONTENTS (TOC)

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
from qgis.core import (
    QgsProject,
    QgsVectorLayer,
)
```

You can use different classes to access all the loaded layers in the TOC and use them to retrieve information:

- `QgsProject`
- `QgsLayerTreeGroup`

4.1 The `QgsProject` class

You can use `QgsProject` to retrieve information about the TOC and all the layers loaded.

You have to create an instance of `QgsProject` and use its methods to get the loaded layers.

The main method is `mapLayers()`. It will return a dictionary of the loaded layers:

```
layers = QgsProject.instance().mapLayers()
print(layers)
```

```
{'countries_89ae1b0f_f41b_4f42_bca4_caf55ddbe4b6': <QgsVectorLayer: 'countries' (ogr)>}
```

The dictionary keys are the unique layer ids while the values are the related objects.

It is now straightforward to obtain any other information about the layers:

```
1 # list of layer names using list comprehension
2 l = [layer.name() for layer in QgsProject.instance().mapLayers().values()]
3 # dictionary with key = layer name and value = layer object
4 layers_list = {}
5 for l in QgsProject.instance().mapLayers().values():
6     layers_list[l.name()] = l
7
8 print(layers_list)
```

```
{'countries': <QgsVectorLayer: 'countries' (ogr)>}
```

You can also query the TOC using the name of the layer:

```
country_layer = QgsProject.instance().mapLayersByName("countries")[0]
```

Note

A list with all the matching layers is returned, so we index with `[0]` to get the first layer with this name.

4.2 QgsLayerTreeGroup class

The layer tree is a classical tree structure built of nodes. There are currently two types of nodes: group nodes (`QgsLayerTreeGroup`) and layer nodes (`QgsLayerTreeLayer`).

Note

for more information you can read these blog posts of Martin Dobias: [Part 1](#) [Part 2](#) [Part 3](#)

The project layer tree can be accessed easily with the method `layerTreeRoot()` of the `QgsProject` class:

```
root = QgsProject.instance().layerTreeRoot()
```

`root` is a group node and has *children*:

```
root.children()
```

A list of direct children is returned. Sub group children should be accessed from their own direct parent.

We can retrieve one of the children:

```
child0 = root.children()[0]
print(child0)
```

```
<QgsLayerTreeLayer: countries>
```

Layers can also be retrieved using their (unique) id:

```
ids = root.findLayerIds()
# access the first layer of the ids list
root.findLayer(ids[0])
```

And groups can also be searched using their names:

```
root.findGroup('Group Name')
```

`QgsLayerTreeGroup` has many other useful methods that can be used to obtain more information about the TOC:

```
# list of all the checked layers in the TOC
checked_layers = root.checkedLayers()
print(checked_layers)
```

```
[<QgsVectorLayer: 'countries' (ogr)>]
```

Now let's add some layers to the project's layer tree. There are two ways of doing that:

1. **Explicit addition** using the `addLayer()` or `insertLayer()` functions:

```

1 # create a temporary layer
2 layer1 = QgsVectorLayer("path_to_layer", "Layer 1", "memory")
3 # add the layer to the legend, last position
4 root.addLayer(layer1)
5 # add the layer at given position
6 root.insertLayer(5, layer1)

```

2. **Implicit addition:** since the project's layer tree is connected to the layer registry it is enough to add a layer to the map layer registry:

```
QgsProject.instance().addMapLayer(layer1)
```

You can switch between `QgsVectorLayer` and `QgsLayerTreeLayer` easily:

```

node_layer = root.findLayer(country_layer.id())
print("Layer node:", node_layer)
print("Map layer:", node_layer.layer())

```

```

Layer node: <QgsLayerTreeLayer: countries>
Map layer: <QgsVectorLayer: 'countries' (ogr)>

```

Groups can be added with the `addGroup()` method. In the example below, the former will add a group to the end of the TOC while for the latter you can add another group within an existing one:

```

node_group1 = root.addGroup('Simple Group')
# add a sub-group to Simple Group
node_subgroup1 = node_group1.addGroup("I'm a sub group")

```

To moving nodes and groups there are many useful methods.

Moving an existing node is done in three steps:

1. cloning the existing node
2. moving the cloned node to the desired position
3. deleting the original node

```

1 # clone the group
2 cloned_group1 = node_group1.clone()
3 # move the node (along with sub-groups and layers) to the top
4 root.insertChildNode(0, cloned_group1)
5 # remove the original node
6 root.removeChildNode(node_group1)

```

It is a little bit more *complicated* to move a layer around in the legend:

```

1 # get a QgsVectorLayer
2 vl = QgsProject.instance().mapLayersByName("countries")[0]
3 # create a QgsLayerTreeLayer object from vl by its id
4 myvl = root.findLayer(vl.id())
5 # clone the myvl QgsLayerTreeLayer object
6 myvlclone = myvl.clone()
7 # get the parent. If None (layer is not in group) returns ''
8 parent = myvl.parent()
9 # move the cloned layer to the top (0)
10 parent.insertChildNode(0, myvlclone)
11 # remove the original myvl
12 root.removeChildNode(myvl)

```

or moving it to an existing group:

```
1 # get a QgsVectorLayer
2 vl = QgsProject.instance().mapLayersByName("countries")[0]
3 # create a QgsLayerTreeLayer object from vl by its id
4 myvl = root.findLayer(vl.id())
5 # clone the myvl QgsLayerTreeLayer object
6 myvlclone = myvl.clone()
7 # create a new group
8 group1 = root.addGroup("Group1")
9 # get the parent. If None (layer is not in group) returns ''
10 parent = myvl.parent()
11 # move the cloned layer to the top (0)
12 group1.insertChildNode(0, myvlclone)
13 # remove the QgsLayerTreeLayer from its parent
14 parent.removeChildNode(myvl)
```

Some other methods that can be used to modify the groups and layers:

```
1 node_group1 = root.findGroup("Group1")
2 # change the name of the group
3 node_group1.setName("Group X")
4 node_layer2 = root.findLayer(country_layer.id())
5 # change the name of the layer
6 node_layer2.setName("Layer X")
7 # change the visibility of a layer
8 node_group1.setItemVisibilityChecked(True)
9 node_layer2.setItemVisibilityChecked(False)
10 # expand/collapse the group view
11 node_group1.setExpanded(True)
12 node_group1.setExpanded(False)
```

USING RASTER LAYERS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsRasterLayer,  
3     QgsProject,  
4     QgsPointXY,  
5     QgsRaster,  
6     QgsRasterShader,  
7     QgsColorRampShader,  
8     QgsSingleBandPseudoColorRenderer,  
9     QgsSingleBandColorDataRenderer,  
10    QgsSingleBandGrayRenderer,  
11 )  
12  
13 from qgis.PyQt.QtGui import (  
14     QColor,  
15 )
```

5.1 Layer Details

A raster layer consists of one or more raster bands — referred to as single band and multi band rasters. One band represents a matrix of values. A color image (e.g. aerial photo) is a raster consisting of red, blue and green bands. Single band rasters typically represent either continuous variables (e.g. elevation) or discrete variables (e.g. land use). In some cases, a raster layer comes with a palette and the raster values refer to the colors stored in the palette.

The following code assumes `rlayer` is a `QgsRasterLayer` object.

```
rlayer = QgsProject.instance().mapLayersByName('srtm')[0]  
# get the resolution of the raster in layer unit  
print(rlayer.width(), rlayer.height())
```

```
919 619
```

```
# get the extent of the layer as QgsRectangle  
print(rlayer.extent())
```

```
<QgsRectangle: 20.06856808199999875 -34.27001076999999896, 20.83945284300000012 -  
↪33.75077500700000144>
```

```
# get the extent of the layer as Strings  
print(rlayer.extent().toString())
```

```
20.0685680819999988,-34.2700107699999990 : 20.8394528430000001,-33.7507750070000014
```

```
# get the raster type: 0 = GrayOrUndefined (single band), 1 = Palette (single_
↪band), 2 = Multiband
print(rlayer.rasterType())
```

```
0
```

```
# get the total band count of the raster
print(rlayer.bandCount())
```

```
1
```

```
# get the first band name of the raster
print(rlayer.bandName(1))
```

```
Band 1: Height
```

```
# get all the available metadata as a QgsLayerMetadata object
print(rlayer.metadata())
```

```
<qgis._core.QgsLayerMetadata object at 0x13711d558>
```

5.2 Renderer

When a raster layer is loaded, it gets a default renderer based on its type. It can be altered either in the layer properties or programmatically.

To query the current renderer:

```
print(rlayer.renderer())
```

```
<qgis._core.QgsSingleBandGrayRenderer object at 0x7f471c1da8a0>
```

```
print(rlayer.renderer().type())
```

```
singlebandgray
```

To set a renderer, use the `setRenderer()` method of `QgsRasterLayer`. There are a number of renderer classes (derived from `QgsRasterRenderer`):

- `QgsHillshadeRenderer`
- `QgsMultiBandColorRenderer`
- `QgsPalettedRasterRenderer`
- `QgsRasterContourRenderer`
- `QgsSingleBandColorDataRenderer`
- `QgsSingleBandGrayRenderer`
- `QgsSingleBandPseudoColorRenderer`

Single band raster layers can be drawn either in gray colors (low values = black, high values = white) or with a pseudocolor algorithm that assigns colors to the values. Single band rasters with a palette can also be drawn using the palette. Multiband layers are typically drawn by mapping the bands to RGB colors. Another possibility is to use just one band for drawing.

5.2.1 Single Band Rasters

Let's say we want a render single band raster layer with colors ranging from green to yellow (corresponding to pixel values from 0 to 255). In the first stage we will prepare a `QgsRasterShader` object and configure its shader function:

```
1 fcn = QgsColorRampShader()
2 fcn.setColorRampType(QgsColorRampShader.Interpolated)
3 lst = [ QgsColorRampShader.ColorRampItem(0, QColor(0,255,0)),
4         QgsColorRampShader.ColorRampItem(255, QColor(255,255,0)) ]
5 fcn.setColorRampItemList(lst)
6 shader = QgsRasterShader()
7 shader.setRasterShaderFunction(fcn)
```

The shader maps the colors as specified by its color map. The color map is provided as a list of pixel values with associated colors. There are three modes of interpolation:

- linear (Interpolated): the color is linearly interpolated from the color map entries above and below the pixel value
- discrete (Discrete): the color is taken from the closest color map entry with equal or higher value
- exact (Exact): the color is not interpolated, only pixels with values equal to color map entries will be drawn

In the second step we will associate this shader with the raster layer:

```
renderer = QgsSingleBandPseudoColorRenderer(rlayer.dataProvider(), 1, shader)
rlayer.setRenderer(renderer)
```

The number 1 in the code above is the band number (raster bands are indexed from one).

Finally we have to use the `triggerRepaint()` method to see the results:

```
rlayer.triggerRepaint()
```

5.2.2 Multi Band Rasters

By default, QGIS maps the first three bands to red, green and blue to create a color image (this is the `MultiBand-Color` drawing style). In some cases you might want to override these setting. The following code interchanges red band (1) and green band (2):

```
rlayer_multi = QgsProject.instance().mapLayersByName('multiband')[0]
rlayer_multi.renderer().setGreenBand(1)
rlayer_multi.renderer().setRedBand(2)
```

In case only one band is necessary for visualization of the raster, single band drawing can be chosen, either gray levels or pseudocolor.

We have to use `triggerRepaint()` to update the map and see the result:

```
rlayer_multi.triggerRepaint()
```

5.3 Query Values

Raster values can be queried using the `sample()` method of the `QgsRasterDataProvider` class. You have to specify a `QgsPointXY` and the band number of the raster layer you want to query. The method returns a tuple with the value and `True` or `False` depending on the results:

```
val, res = rlayer.dataProvider().sample(QgsPointXY(20.50, -34), 1)
```

Another method to query raster values is using the `identify()` method that returns a `QgsRasterIdentifyResult` object.

```
ident = rlayer.dataProvider().identify(QgsPointXY(20.5, -34), QgsRaster.  
↳ IdentifyFormatValue)  
  
if ident.isValid():  
    print(ident.results())
```

```
{1: 323.0}
```

In this case, the `results()` method returns a dictionary, with band indices as keys, and band values as values. For instance, something like `{1: 323.0}`

5.4 Editing raster data

You can create a raster layer using the `QgsRasterBlock` class. For example, to create a 2x2 raster block with one byte per pixel:

```
block = QgsRasterBlock(Qgis.Byte, 2, 2)  
block.setData(b'\xaa\xbb\xcc\xdd')
```

Raster pixels can be overwritten thanks to the `writeBlock()` method. To overwrite existing raster data at position 0,0 by the 2x2 block:

```
provider = rlayer.dataProvider()  
provider.setEditable(True)  
provider.writeBlock(block, 1, 0, 0)  
provider.setEditable(False)
```

USING VECTOR LAYERS

 Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsApplication,  
3     QgsDataSourceUri,  
4     QgsCategorizedSymbolRenderer,  
5     QgsClassificationRange,  
6     QgsPointXY,  
7     QgsProject,  
8     QgsExpression,  
9     QgsField,  
10    QgsFields,  
11    QgsFeature,  
12    QgsFeatureRequest,  
13    QgsFeatureRenderer,  
14    QgsGeometry,  
15    QgsGraduatedSymbolRenderer,  
16    QgsMarkerSymbol,  
17    QgsMessageLog,  
18    QgsRectangle,  
19    QgsRendererCategory,  
20    QgsRendererRange,  
21    QgsSymbol,  
22    QgsVectorDataProvider,  
23    QgsVectorLayer,  
24    QgsVectorFileWriter,  
25    QgsWkbTypes,  
26    QgsSpatialIndex,  
27    QgsVectorLayerUtils  
28 )  
29  
30 from qgis.core.additions.edit import edit  
31  
32 from qgis.PyQt.QtGui import (  
33     QColor,  
34 )
```

This section summarizes various actions that can be done with vector layers.

Most work here is based on the methods of the `QgsVectorLayer` class.

6.1 Retrieving information about attributes

You can retrieve information about the fields associated with a vector layer by calling `fields()` on a `QgsVectorLayer` object:

```
vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_
↪layer", "ogr")
for field in vlayer.fields():
    print(field.name(), field.typeName())
```

```
1 fid Integer64
2 id Integer64
3 scalerank Integer64
4 featurecla String
5 type String
6 name String
7 abbrev String
8 location String
9 gps_code String
10 iata_code String
11 wikipedia String
12 natlscale Real
```

The `displayField()` and `mapTipTemplate()` methods provide information on the field and template used in the maptips tab.

When you load a vector layer, a field is always chosen by QGIS as the Display Name, while the HTML Map Tip is empty by default. With these methods you can easily get both:

```
vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_
↪layer", "ogr")
print(vlayer.displayField())
```

```
name
```

Note

If you change the Display Name from a field to an expression, you have to use `displayExpression()` instead of `displayField()`.

6.2 Iterating over Vector Layer

Iterating over the features in a vector layer is one of the most common tasks. Below is an example of the simple basic code to perform this task and showing some information about each feature. The `layer` variable is assumed to have a `QgsVectorLayer` object.

```
1 # "layer" is a QgsVectorLayer instance
2 layer = iface.activeLayer()
3 features = layer.getFeatures()
4
5 for feature in features:
6     # retrieve every feature with its geometry and attributes
7     print("Feature ID: ", feature.id())
8     # fetch geometry
9     # show some information about the feature geometry
10    geom = feature.geometry()
```

(continues on next page)

(continued from previous page)

```

11 geomSingleType = QgsWkbTypes.isSingleType(geom.wkbType())
12 if geom.type() == QgsWkbTypes.PointGeometry:
13     # the geometry type can be of single or multi type
14     if geomSingleType:
15         x = geom.asPoint()
16         print("Point: ", x)
17     else:
18         x = geom.asMultiPoint()
19         print("MultiPoint: ", x)
20 elif geom.type() == QgsWkbTypes.LineGeometry:
21     if geomSingleType:
22         x = geom.asPolyline()
23         print("Line: ", x, "length: ", geom.length())
24     else:
25         x = geom.asMultiPolyline()
26         print("MultiLine: ", x, "length: ", geom.length())
27 elif geom.type() == QgsWkbTypes.PolygonGeometry:
28     if geomSingleType:
29         x = geom.asPolygon()
30         print("Polygon: ", x, "Area: ", geom.area())
31     else:
32         x = geom.asMultiPolygon()
33         print("MultiPolygon: ", x, "Area: ", geom.area())
34 else:
35     print("Unknown or invalid geometry")
36 # fetch attributes
37 attrs = feature.attributes()
38 # attrs is a list. It contains all the attribute values of this feature
39 print(attrs)
40 # for this test only print the first feature
41 break

```

```

Feature ID: 1
Point: <QgsPointXY: POINT(7 45)>
[1, 'First feature']

```

6.3 Selecting features

In QGIS desktop, features can be selected in different ways: the user can click on a feature, draw a rectangle on the map canvas or use an expression filter. Selected features are normally highlighted in a different color (default is yellow) to draw user's attention on the selection.

Sometimes it can be useful to programmatically select features or to change the default color.

To select all the features, the `selectAll()` method can be used:

```

# Get the active layer (must be a vector layer)
layer = iface.activeLayer()
layer.selectAll()

```

To select using an expression, use the `selectByExpression()` method:

```

# Assumes that the active layer is points.shp file from the QGIS test suite
# (Class (string) and Heading (number) are attributes in points.shp)
layer = iface.activeLayer()
layer.selectByExpression('"Class"=\'B52\' and "Heading" > 10 and "Heading" <70',
↪QgsVectorLayer.SetSelection)

```

To change the selection color you can use `setSelectionColor()` method of `QgsMapCanvas` as shown in the following example:

```
iface.mapCanvas().setSelectionColor( QColor("red") )
```

To add features to the selected features list for a given layer, you can call `select()` passing to it the list of features IDs:

```
1 selected_fid = []
2
3 # Get the first feature id from the layer
4 feature = next(layer.getFeatures())
5 if feature:
6     selected_fid.append(feature.id())
7
8 # Add that features to the selected list
9 layer.select(selected_fid)
```

To clear the selection:

```
layer.removeSelection()
```

6.3.1 Accessing attributes

Attributes can be referred to by their name:

```
print(feature['name'])
```

```
First feature
```

Alternatively, attributes can be referred to by index. This is a bit faster than using the name. For example, to get the second attribute:

```
print(feature[1])
```

```
First feature
```

6.3.2 Iterating over selected features

If you only need selected features, you can use the `selectedFeatures()` method from the vector layer:

```
selection = layer.selectedFeatures()
for feature in selection:
    # do whatever you need with the feature
    pass
```

6.3.3 Iterating over a subset of features

If you want to iterate over a given subset of features in a layer, such as those within a given area, you have to add a `QgsFeatureRequest` object to the `getFeatures()` call. Here's an example:

```
1 areaOfInterest = QgsRectangle(450290,400520, 450750,400780)
2
3 request = QgsFeatureRequest().setFilterRect(areaOfInterest)
4
5 for feature in layer.getFeatures(request):
```

(continues on next page)

(continued from previous page)

```

6     # do whatever you need with the feature
7     pass

```

For the sake of speed, the intersection is often done only using feature's bounding box. There is however a flag `ExactIntersect` that makes sure that only intersecting features will be returned:

```

request = QgsFeatureRequest().setFilterRect(areaOfInterest) \
    .setFlags(QgsFeatureRequest.ExactIntersect)

```

With `setLimit()` you can limit the number of requested features. Here's an example:

```

request = QgsFeatureRequest()
request.setLimit(2)
for feature in layer.getFeatures(request):
    print(feature)

```

```

<qgis._core.QgsFeature object at 0x7f9b78590948>
<qgis._core.QgsFeature object at 0x7faef5881670>

```

If you need an attribute-based filter instead (or in addition) of a spatial one like shown in the examples above, you can build a `QgsExpression` object and pass it to the `QgsFeatureRequest` constructor. Here's an example:

```

# The expression will filter the features where the field "location_name"
# contains the word "Lake" (case insensitive)
exp = QgsExpression('location_name ILIKE \'%Lake%\'')
request = QgsFeatureRequest(exp)

```

See [Expressions, Filtering and Calculating Values](#) for the details about the syntax supported by `QgsExpression`.

The request can be used to define the data retrieved for each feature, so the iterator returns all features, but returns partial data for each of them.

```

1  # Only return selected fields to increase the "speed" of the request
2  request.setSubsetOfAttributes([0,2])
3
4  # More user friendly version
5  request.setSubsetOfAttributes(['name','id'],layer.fields())
6
7  # Don't return geometry objects to increase the "speed" of the request
8  request.setFlags(QgsFeatureRequest.NoGeometry)
9
10 # Fetch only the feature with id 45
11 request.setFilterFid(45)
12
13 # The options may be chained
14 request.setFilterRect(areaOfInterest).setFlags(QgsFeatureRequest.NoGeometry) \
    ↪ .setFilterFid(45).setSubsetOfAttributes([0,2])

```

6.4 Modifying Vector Layers

Most vector data providers support editing of layer data. Sometimes they support just a subset of possible editing actions. Use the `capabilities()` function to find out what set of functionality is supported.

```

caps = layer.dataProvider().capabilities()
# Check if a particular capability is supported:
if caps & QgsVectorDataProvider.DeleteFeatures:
    print('The layer supports DeleteFeatures')

```

The layer supports `DeleteFeatures`

For a list of all available capabilities, please refer to the [API Documentation of QgsVectorDataProvider](#).

To print layer's capabilities textual description in a comma separated list you can use `capabilitiesString()` as in the following example:

```
1 caps_string = layer.dataProvider().capabilitiesString()
2 # Print:
3 # 'Add Features, Delete Features, Change Attribute Values, Add Attributes,
4 # Delete Attributes, Rename Attributes, Fast Access to Features at ID,
5 # Presimplify Geometries, Presimplify Geometries with Validity Check,
6 # Transactions, Curved Geometries'
```

By using any of the following methods for vector layer editing, the changes are directly committed to the underlying data store (a file, database etc). In case you would like to do only temporary changes, skip to the next section that explains how to do *modifications with editing buffer*.

Note

If you are working inside QGIS (either from the console or from a plugin), it might be necessary to force a redraw of the map canvas in order to see the changes you've done to the geometry, to the style or to the attributes:

```
1 # If caching is enabled, a simple canvas refresh might not be sufficient
2 # to trigger a redraw and you must clear the cached image for the layer
3 if iface.mapCanvas().isCachingEnabled():
4     layer.triggerRepaint()
5 else:
6     iface.mapCanvas().refresh()
```

6.4.1 Add Features

Create some `QgsFeature` instances and pass a list of them to the provider `QgsVectorDataProvider` `addFeatures()` method. It will return two values: result (True or False) and list of added features (their ID is set by the data store).

To set up the attributes of the feature, you can either initialize the feature passing a `QgsFields` object (you can obtain that from the `fields()` method of the vector layer) or call `initAttributes()` passing the number of fields you want to be added.

```
1 if caps & QgsVectorDataProvider.AddFeatures:
2     feat = QgsFeature(layer.fields())
3     feat.setAttributes([0, 'hello'])
4     # Or set a single attribute by key or by index:
5     feat.setAttribute('name', 'hello')
6     feat.setAttribute(0, 'hello')
7     feat.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(123, 456)))
8     (res, outFeats) = layer.dataProvider().addFeatures([feat])
```

6.4.2 Delete Features

To delete some features, just provide a list of their feature IDs.

```
if caps & QgsVectorDataProvider.DeleteFeatures:
    res = layer.dataProvider().deleteFeatures([5, 10])
```

6.4.3 Modify Features

It is possible to either change feature's geometry or to change some attributes. The following example first changes values of attributes with index 0 and 1, then it changes the feature's geometry.

```
1 fid = 100    # ID of the feature we will modify
2
3 if caps & QgsVectorDataProvider.ChangeAttributeValues:
4     attrs = { 0 : "hello", 1 : 123 }
5     layer.dataProvider().changeAttributeValues({ fid : attrs })
6
7 if caps & QgsVectorDataProvider.ChangeGeometries:
8     geom = QgsGeometry.fromPointXY(QgsPointXY(111,222))
9     layer.dataProvider().changeGeometryValues({ fid : geom })
```

Tip

Favor `QgsVectorLayerEditUtils` class for geometry-only edits

If you only need to change geometries, you might consider using the `QgsVectorLayerEditUtils` which provides some useful methods to edit geometries (translate, insert or move vertex, etc.).

6.4.4 Modifying Vector Layers with an Editing Buffer

When editing vectors within QGIS application, you have to first start editing mode for a particular layer, then do some modifications and finally commit (or rollback) the changes. All the changes you make are not written until you commit them — they stay in layer's in-memory editing buffer. It is possible to use this functionality also programmatically — it is just another method for vector layer editing that complements the direct usage of data providers. Use this option when providing some GUI tools for vector layer editing, since this will allow user to decide whether to commit/rollback and allows the usage of undo/redo. When changes are committed, all changes from the editing buffer are saved to data provider.

The methods are similar to the ones we have seen in the provider, but they are called on the `QgsVectorLayer` object instead.

For these methods to work, the layer must be in editing mode. To start the editing mode, use the `startEditing()` method. To stop editing, use the `commitChanges()` or `rollback()` methods. The first one will commit all your changes to the data source, while the second one will discard them and will not modify the data source at all.

To find out whether a layer is in editing mode, use the `isEditable()` method.

Here you have some examples that demonstrate how to use these editing methods.

```
1 from qgis.PyQt.QtCore import QMetaType
2
3 feat1 = feat2 = QgsFeature(layer.fields())
4 fid = 99
5 feat1.setId(fid)
6
7 # add two features (QgsFeature instances)
8 layer.addFeatures([feat1, feat2])
```

(continues on next page)

(continued from previous page)

```

9  # delete a feature with specified ID
10 layer.deleteFeature(fid)
11
12 # set new geometry (QgsGeometry instance) for a feature
13 geometry = QgsGeometry.fromWkt("POINT(7 45)")
14 layer.changeGeometry(fid, geometry)
15 # update an attribute with given field index (int) to a given value
16 fieldIndex = 1
17 value = 'My new name'
18 layer.changeAttributeValue(fid, fieldIndex, value)
19
20 # add new field
21 layer.addAttribute(QgsField("mytext", QMetaType.Type.QString))
22 # remove a field
23 layer.deleteAttribute(fieldIndex)

```

In order to make undo/redo work properly, the above mentioned calls have to be wrapped into undo commands. (If you do not care about undo/redo and want to have the changes stored immediately, then you will have easier work by *editing with data provider*.)

Here is how you can use the undo functionality:

```

1  layer.beginEditCommand("Feature triangulation")
2
3  # ... call layer's editing methods ...
4
5  if problem_occurred:
6      layer.destroyEditCommand()
7      # ... tell the user that there was a problem
8      # and return
9
10 # ... more editing ...
11
12 layer.endEditCommand()

```

The `beginEditCommand()` method will create an internal “active” command and will record subsequent changes in vector layer. With the call to `endEditCommand()` the command is pushed onto the undo stack and the user will be able to undo/redo it from GUI. In case something went wrong while doing the changes, the `destroyEditCommand()` method will remove the command and rollback all changes done while this command was active.

You can also use the `with edit(layer):`-statement to wrap commit and rollback into a more semantic code block as shown in the example below:

```

with edit(layer):
    feat = next(layer.getFeatures())
    feat[0] = 5
    layer.updateFeature(feat)

```

This will automatically call `commitChanges()` in the end. If any exception occurs, it will `rollback()` all the changes. In case a problem is encountered within `commitChanges()` (when the method returns False) a `QgsEditError` exception will be raised.

6.4.5 Adding and Removing Fields

To add fields (attributes), you need to specify a list of field definitions. For deletion of fields just provide a list of field indexes.

```

1 from qgis.PyQt.QtCore import QMetaType
2
3 if caps & QgsVectorDataProvider.AddAttributes:
4     res = layer.dataProvider().addAttributes(
5         [QgsField("mytext", QMetaType.Type.QString),
6          QgsField("myint", QMetaType.Type.Int)])
7
8 if caps & QgsVectorDataProvider.DeleteAttributes:
9     res = layer.dataProvider().deleteAttributes([0])

```



```

1 # Alternate methods for removing fields
2 # first create temporary fields to be removed (f1-3)
3 layer.dataProvider().addAttributes([QgsField("f1", QMetaType.Type.Int),
4                                     QgsField("f2", QMetaType.Type.Int),
5                                     QgsField("f3", QMetaType.Type.Int)])
6 layer.updateFields()
7 count=layer.fields().count() # count of layer fields
8 ind_list=list((count-3, count-2)) # create list
9
10 # remove a single field with an index
11 layer.dataProvider().deleteAttributes([count-1])
12
13 # remove multiple fields with a list of indices
14 layer.dataProvider().deleteAttributes(ind_list)

```

After adding or removing fields in the data provider the layer's fields need to be updated because the changes are not automatically propagated.

```
layer.updateFields()
```

Tip

Directly save changes using with based command

Using `with edit(layer) :` the changes will be committed automatically calling `commitChanges()` at the end. If any exception occurs, it will `rollback()` all the changes. See [Modifying Vector Layers with an Editing Buffer](#).

6.5 Using Spatial Index

Spatial indexes can dramatically improve the performance of your code if you need to do frequent queries to a vector layer. Imagine, for instance, that you are writing an interpolation algorithm, and that for a given location you need to know the 10 closest points from a points layer, in order to use those point for calculating the interpolated value. Without a spatial index, the only way for QGIS to find those 10 points is to compute the distance from each and every point to the specified location and then compare those distances. This can be a very time consuming task, especially if it needs to be repeated for several locations. If a spatial index exists for the layer, the operation is much more effective.

Think of a layer without a spatial index as a telephone book in which telephone numbers are not ordered or indexed. The only way to find the telephone number of a given person is to read from the beginning until you find it.

Spatial indexes are not created by default for a QGIS vector layer, but you can create them easily. This is what you have to do:

- create spatial index using the `QgsSpatialIndex` class:

```
index = QgsSpatialIndex()
```

- add features to index — index takes `QgsFeature` object and adds it to the internal data structure. You can create the object manually or use one from a previous call to the provider's `getFeatures()` method.

```
index.addFeature(feats)
```

- alternatively, you can load all features of a layer at once using bulk loading

```
index = QgsSpatialIndex(layer.getFeatures())
```

- once spatial index is filled with some values, you can do some queries

```
1 # returns array of feature IDs of five nearest features
2 nearest = index.nearestNeighbor(QgsPointXY(25.4, 12.7), 5)
3
4 # returns array of IDs of features which intersect the rectangle
5 intersect = index.intersects(QgsRectangle(22.5, 15.3, 23.1, 17.2))
```

You can also use the `QgsSpatialIndexKDBush` spatial index. This index is similar to the *standard* `QgsSpatialIndex` but:

- supports **only** single point features
- is **static** (no additional features can be added to the index after the construction)
- is **much faster!**
- allows direct retrieval of the original feature's points, without requiring additional feature requests
- supports true *distance based* searches, i.e. return all points within a radius from a search point

6.6 The `QgsVectorLayerUtils` class

The `QgsVectorLayerUtils` class contains some very useful methods that you can use with vector layers.

For example the `createFeature()` method prepares a `QgsFeature` to be added to a vector layer keeping all the eventual constraints and default values of each field:

```
vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_
↳layer", "ogr")
feat = QgsVectorLayerUtils.createFeature(vlayer)
```

The `getValues()` method allows you to quickly get the values of a field or expression:

```
1 vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_
↳layer", "ogr")
2 # select only the first feature to make the output shorter
3 vlayer.selectByIds([1])
4 val = QgsVectorLayerUtils.getValues(vlayer, "NAME", selectedOnly=True)
5 print(val)
```

```
(['Sahnewal'], True)
```

6.7 Creating Vector Layers

There are several ways to generate a vector layer dataset:

- the `QgsVectorFileWriter` class: A convenient class for writing vector files to disk, using either a static call to `writeAsVectorFormatV3()` which saves the whole vector layer or creating an instance of the class and issue calls to inherited `addFeature()`. This class supports all the vector formats that GDAL supports (GeoPackage, Shapefile, GeoJSON, KML and others).
- the `QgsVectorLayer` class: instantiates a data provider that interprets the supplied path (url) of the data source to connect to and access the data. It can be used to create temporary, memory-based layers (memory) and connect to GDAL vector datasets (ogr), databases (postgres, spatialite, mysql, mssql) and more (wfs, gpx, delimitedtext...).

6.7.1 From an instance of `QgsVectorFileWriter`

```

1  # SaveVectorOptions contains many settings for the writer process
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  transform_context = QgsProject.instance().transformContext()
4  # Write to a GeoPackage (default)
5  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
6                                                    "testdata/my_new_file.gpkg",
7                                                    transform_context,
8                                                    save_options)
9  if error[0] == QgsVectorFileWriter.NoError:
10     print("success!")
11  else:
12     print(error)

```

```

1  # Write to an ESRI Shapefile format dataset using UTF-8 text encoding
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  save_options.driverName = "ESRI Shapefile"
4  save_options.fileEncoding = "UTF-8"
5  transform_context = QgsProject.instance().transformContext()
6  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
7                                                    "testdata/my_new_shapefile",
8                                                    transform_context,
9                                                    save_options)
10  if error[0] == QgsVectorFileWriter.NoError:
11     print("success again!")
12  else:
13     print(error)

```

```

1  # Write to an ESRI GDB file
2  save_options = QgsVectorFileWriter.SaveVectorOptions()
3  save_options.driverName = "FileGDB"
4  # if no geometry
5  save_options.overrideGeometryType = QgsWkbTypes.Unknown
6  save_options.actionOnExistingFile = QgsVectorFileWriter.CreateOrOverwriteLayer
7  save_options.layerName = 'my_new_layer_name'
8  transform_context = QgsProject.instance().transformContext()
9  gdb_path = "testdata/my_example.gdb"
10  error = QgsVectorFileWriter.writeAsVectorFormatV3(layer,
11                                                    gdb_path,
12                                                    transform_context,
13                                                    save_options)
14  if error[0] == QgsVectorFileWriter.NoError:
15     print("success!")
16  else:
17     print(error)

```

You can also convert fields to make them compatible with different formats by using the `FieldValueConverter`. For example, to convert array variable types (e.g. in Postgres) to a text type, you can do the following:

```

1 LIST_FIELD_NAME = 'xxxx'
2
3 class ESRIValueConverter(QgsVectorFileWriter.FieldValueConverter):
4
5     def __init__(self, layer, list_field):
6         QgsVectorFileWriter.FieldValueConverter.__init__(self)
7         self.layer = layer
8         self.list_field_idx = self.layer.fields().indexOfName(list_field)
9
10    def convert(self, fieldIdxInLayer, value):
11        if fieldIdxInLayer == self.list_field_idx:
12            return QgsListFieldFormatter().representValue(layer=vlayer,
13                                                         fieldIndex=self.list_field_idx,
14                                                         config={},
15                                                         cache=None,
16                                                         value=value)
17
18        else:
19            return value
20
21    def fieldDefinition(self, field):
22        idx = self.layer.fields().indexOfName(field.name())
23        if idx == self.list_field_idx:
24            return QgsField(LIST_FIELD_NAME, QMetaType.Type.QString)
25        else:
26            return self.layer.fields()[idx]
27
28 converter = ESRIValueConverter(vlayer, LIST_FIELD_NAME)
29 opts = QgsVectorFileWriter.SaveVectorOptions()
30 opts.fieldValueConverter = converter

```

A destination CRS may also be specified — if a valid instance of `QgsCoordinateReferenceSystem` is passed as the fourth parameter, the layer is transformed to that CRS.

For valid driver names please call the `supportedFiltersAndFormats()` method or consult the [supported formats by OGR](#) — you should pass the value in the “Code” column as the driver name.

Optionally you can set whether to export only selected features, pass further driver-specific options for creation or tell the writer not to create attributes... There are a number of other (optional) parameters; see the `QgsVectorFileWriter` documentation for details.

6.7.2 Directly from features

```

1 from qgis.PyQt.QtCore import QMetaType
2
3 # define fields for feature attributes. A QgsFields object is needed
4 fields = QgsFields()
5 fields.append(QgsField("first", QMetaType.Type.Int))
6 fields.append(QgsField("second", QMetaType.Type.QString))
7
8 """ create an instance of vector file writer, which will create the vector file.
9 Arguments:
10 1. path to new file (will fail if exists already)
11 2. field map
12 3. geometry type - from WKBTYP enum
13 4. layer's spatial reference (instance of
14    QgsCoordinateReferenceSystem)
15 5. coordinate transform context
16 6. save options (driver name for the output file, encoding etc.)

```

(continues on next page)

(continued from previous page)

```

17  """
18
19  crs = QgsProject.instance().crs()
20  transform_context = QgsProject.instance().transformContext()
21  save_options = QgsVectorFileWriter.SaveVectorOptions()
22  save_options.driverName = "ESRI Shapefile"
23  save_options.fileEncoding = "UTF-8"
24
25  writer = QgsVectorFileWriter.create(
26      "testdata/my_new_shapefile.shp",
27      fields,
28      QgsWkbTypes.Point,
29      crs,
30      transform_context,
31      save_options
32  )
33
34  if writer.hasError() != QgsVectorFileWriter.NoError:
35      print("Error when creating shapefile: ", writer.errorMessage())
36
37  # add a feature
38  fet = QgsFeature()
39
40  fet.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
41  fet.setAttributes([1, "text"])
42  writer.addFeature(fet)
43
44  # delete the writer to flush features to disk
45  del writer

```

6.7.3 From an instance of `QgsVectorLayer`

Among all the data providers supported by the `QgsVectorLayer` class, let's focus on the memory-based layers. Memory provider is intended to be used mainly by plugin or 3rd party app developers. It does not store data on disk, allowing developers to use it as a fast backend for some temporary layers.

The provider supports string, int and double fields.

The memory provider also supports spatial indexing, which is enabled by calling the provider's `createSpatialIndex()` function. Once the spatial index is created you will be able to iterate over features within smaller regions faster (since it's not necessary to traverse all the features, only those in specified rectangle).

A memory provider is created by passing "memory" as the provider string to the `QgsVectorLayer` constructor.

The constructor also takes a URI defining the geometry type of the layer, one of: "Point", "LineString", "Polygon", "MultiPoint", "MultiLineString", "MultiPolygon" or "None".

The URI can also specify the coordinate reference system, fields, and indexing of the memory provider in the URI. The syntax is:

crs=definition

Specifies the coordinate reference system, where definition may be any of the forms accepted by `QgsCoordinateReferenceSystem.createFromString()`

index=yes

Specifies that the provider will use a spatial index

field=name:type(length,precision)

Specifies an attribute of the layer. The attribute has a name, and optionally a type (integer, double, or string), length, and precision. There may be multiple field definitions.

The following example of a URI incorporates all these options

```
"Point?crs=epsg:4326&field=id:integer&field=name:string(20)&index=yes"
```

The following example code illustrates creating and populating a memory provider

```
1 from qgis.PyQt.QtCore import QDateTime, QMetaType, Qt
2
3 # create layer
4 vl = QgsVectorLayer("Point", "temporary_points", "memory")
5 pr = vl.dataProvider()
6
7 # add fields
8 pr.addAttributes([QgsField("name", QMetaType.Type.QString),
9                       QgsField("age", QMetaType.Type.Int),
10                      QgsField("size", QMetaType.Type.Double),
11                      QgsField("birthday", QMetaType.Type.QDateTime)])
12 vl.updateFields() # tell the vector layer to fetch changes from the provider
13
14 # add a feature
15 fet = QgsFeature()
16 fet.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
17 fmt = Qt.DateFormat.ISODate
18 t = QDateTime.fromString("2000-01-01T12:00:00", fmt)
19 fet.setAttributes(["Johnny", 2, 0.3, t])
20 pr.addFeatures([fet])
21
22 # update layer's extent when new features have been added
23 # because change of extent in provider is not propagated to the layer
24 vl.updateExtents()
```

Finally, let's check whether everything went well

```
1 # show some stats
2 print("fields:", len(pr.fields()))
3 print("features:", pr.featureCount())
4 e = vl.extent()
5 print("extent:", e.xMinimum(), e.yMinimum(), e.xMaximum(), e.yMaximum())
6
7 # iterate over features
8 features = vl.getFeatures()
9 for fet in features:
10     print("F:", fet.id(), fet.attributes(), fet.geometry().asPoint())
```

```
fields: 4
features: 1
extent: 10.0 10.0 10.0 10.0
F: 1 ['Johnny', 2, 0.3, PyQt5.QtCore.QDateTime(2000, 1, 1, 12, 0)] <QgsPointXY:
↪ POINT(10 10)>
```

6.8 Appearance (Symbology) of Vector Layers

When a vector layer is being rendered, the appearance of the data is given by **renderer** and **symbols** associated with the layer. Symbols are classes which take care of drawing of visual representation of features, while renderers determine what symbol will be used for a particular feature.

The renderer for a given layer can be obtained as shown below:

```
renderer = layer.renderer()
```

And with that reference, let us explore it a bit

```
print("Type:", renderer.type())
```

```
Type: singleSymbol
```

There are several known renderer types available in the QGIS core library:

Type	Class	Description
singleSymbol	<code>QgsSingleSymbolRenderer</code>	Renders all features with the same symbol
categorizedSymbol	<code>QgsCategorizedSymbolRenderer</code>	Renders features using a different symbol for each category
graduatedSymbol	<code>QgsGraduatedSymbolRenderer</code>	Renders features using a different symbol for each range of values

There might be also some custom renderer types, so never make an assumption there are just these types. You can query the application's `QgsRendererRegistry` to find out currently available renderers:

```
print(QgsApplication.rendererRegistry().renderersList())
```

```
['nullSymbol', 'singleSymbol', 'categorizedSymbol', 'graduatedSymbol',  
↪ 'RuleRenderer', 'pointDisplacement', 'pointCluster', 'mergedFeatureRenderer',  
↪ 'invertedPolygonRenderer', 'heatmapRenderer', '25dRenderer', 'embeddedSymbol']
```

It is possible to obtain a dump of a renderer contents in text form — can be useful for debugging

```
renderer.dump()
```

```
SINGLE: MARKER SYMBOL (1 layers) color 190,207,80,255
```

6.8.1 Single Symbol Renderer

You can get the symbol used for rendering by calling `symbol()` method and change it with `setSymbol()` method (note for C++ devs: the renderer takes ownership of the symbol.)

You can change the symbol used by a particular vector layer by calling `setSymbol()` passing an instance of the appropriate symbol instance. Symbols for *point*, *line* and *polygon* layers can be created by calling the `createSimple()` function of the corresponding classes `QgsMarkerSymbol`, `QgsLineSymbol` and `QgsFillSymbol`.

The dictionary passed to `createSimple()` sets the style properties of the symbol.

For example you can replace the symbol used by a particular **point** layer by calling `setSymbol()` passing an instance of a `QgsMarkerSymbol`, as in the following code example:

```
symbol = QgsMarkerSymbol.createSimple({'name': 'square', 'color': 'red'})  
layer.renderer().setSymbol(symbol)  
# show the change  
layer.triggerRepaint()
```

`name` indicates the shape of the marker, and can be any of the following:

- circle
- square
- cross
- rectangle
- diamond
- pentagon

- triangle
- equilateral_triangle
- star
- regular_star
- arrow
- filled_arrowhead
- x

To get the full list of properties for the first symbol layer of a symbol instance you can follow the example code:

```
print(layer.renderer().symbol().symbolLayers()[0].properties())
```

```
{'angle': '0', 'cap_style': 'square', 'color': '255,0,0,255,rgb:1,0,0,1',
↪ 'horizontal_anchor_point': '1', 'joinstyle': 'bevel', 'name': 'square', 'offset
↪ ': '0,0', 'offset_map_unit_scale': '3x:0,0,0,0,0,0', 'offset_unit': 'MM',
↪ 'outline_color': '35,35,35,255,rgb:0.13725490196078433,0.13725490196078433,0.
↪ 13725490196078433,1', 'outline_style': 'solid', 'outline_width': '0', 'outline_
↪ width_map_unit_scale': '3x:0,0,0,0,0,0', 'outline_width_unit': 'MM', 'scale_
↪ method': 'diameter', 'size': '2', 'size_map_unit_scale': '3x:0,0,0,0,0,0', 'size_
↪ unit': 'MM', 'vertical_anchor_point': '1'}
```

This can be useful if you want to alter some properties:

```
1 # You can alter a single property...
2 layer.renderer().symbol().symbolLayer(0).setSize(3)
3 # ... but not all properties are accessible from methods,
4 # you can also replace the symbol completely:
5 props = layer.renderer().symbol().symbolLayer(0).properties()
6 props['color'] = 'yellow'
7 props['name'] = 'square'
8 layer.renderer().setSymbol(QgsMarkerSymbol.createSimple(props))
9 # show the changes
10 layer.triggerRepaint()
```

6.8.2 Categorized Symbol Renderer

When using a categorized renderer, you can query and set the attribute that is used for classification: use the `classAttribute()` and `setClassAttribute()` methods.

To get a list of categories

```
1 categorized_renderer = QgsCategorizedSymbolRenderer()
2 # Add a few categories
3 cat1 = QgsRendererCategory('1', QgsMarkerSymbol(), 'category 1')
4 cat2 = QgsRendererCategory('2', QgsMarkerSymbol(), 'category 2')
5 categorized_renderer.addCategory(cat1)
6 categorized_renderer.addCategory(cat2)
7
8 for cat in categorized_renderer.categories():
9     print("{}: {} :: {}".format(cat.value(), cat.label(), cat.symbol()))
```

```
1: category 1 :: <qgis._core.QgsMarkerSymbol object at 0x7f378ffcd9d8>
2: category 2 :: <qgis._core.QgsMarkerSymbol object at 0x7f378ffcd9d8>
```

Where `value()` is the value used for discrimination between categories, `label()` is a text used for category description and `symbol()` method returns the assigned symbol.

The renderer usually stores also original symbol and color ramp which were used for the classification: `sourceColorRamp()` and `sourceSymbol()` methods.

6.8.3 Graduated Symbol Renderer

This renderer is very similar to the categorized symbol renderer described above, but instead of one attribute value per class it works with ranges of values and thus can be used only with numerical attributes.

To find out more about ranges used in the renderer

```

1 graduated_renderer = QgsGraduatedSymbolRenderer()
2 # Add a few categories
3 graduated_renderer.addClassRange(QgsRendererRange(QgsClassificationRange('class 0-
   ↳100', 0, 100), QgsMarkerSymbol()))
4 graduated_renderer.addClassRange(QgsRendererRange(QgsClassificationRange('class_
   ↳101-200', 101, 200), QgsMarkerSymbol()))
5
6 for ran in graduated_renderer.ranges():
7     print("{} - {}: {} {}".format(
8         ran.lowerValue(),
9         ran.upperValue(),
10        ran.label(),
11        ran.symbol()
12    ))

```

```

0.0 - 100.0: class 0-100 <qgis._core.QgsMarkerSymbol object at 0x7f8bad281b88>
101.0 - 200.0: class 101-200 <qgis._core.QgsMarkerSymbol object at 0x7f8bad281b88>

```

you can again use the `classAttribute()` (to find the classification attribute name), `sourceSymbol()` and `sourceColorRamp()` methods. Additionally there is the `mode()` method which determines how the ranges were created: using equal intervals, quantiles or some other method.

If you wish to create your own graduated symbol renderer you can do so as illustrated in the example snippet below (which creates a simple two class arrangement)

```

1 from qgis.PyQt import QtGui
2
3 myVectorLayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports",
   ↳"Airports layer", "ogr")
4 myTargetField = 'scalerank'
5 myRangeList = []
6 myOpacity = 1
7 # Make our first symbol and range...
8 myMin = 0.0
9 myMax = 50.0
10 myLabel = 'Group 1'
11 myColour = QtGui.QColor('#ffee00')
12 mySymbol1 = QgsSymbol.defaultSymbol(myVectorLayer.geometryType())
13 mySymbol1.setColor(myColour)
14 mySymbol1.setOpacity(myOpacity)
15 myRange1 = QgsRendererRange(myMin, myMax, mySymbol1, myLabel)
16 myRangeList.append(myRange1)
17 #now make another symbol and range...
18 myMin = 50.1
19 myMax = 100
20 myLabel = 'Group 2'
21 myColour = QtGui.QColor('#00eeff')
22 mySymbol2 = QgsSymbol.defaultSymbol(
23     myVectorLayer.geometryType())
24 mySymbol2.setColor(myColour)
25 mySymbol2.setOpacity(myOpacity)

```

(continues on next page)

(continued from previous page)

```

26 myRange2 = QgsRendererRange(myMin, myMax, mySymbol2, myLabel)
27 myRangeList.append(myRange2)
28 myRenderer = QgsGraduatedSymbolRenderer(' ', myRangeList)
29 myClassificationMethod = QgsApplication.classificationMethodRegistry().method(
    ↪ "EqualInterval")
30 myRenderer.setClassificationMethod(myClassificationMethod)
31 myRenderer.setClassAttribute(myTargetField)
32
33 myVectorLayer.setRenderer(myRenderer)

```

6.8.4 Working with Symbols

For representation of symbols, there is `QgsSymbol` base class with three derived classes:

- `QgsMarkerSymbol` — for point features
- `QgsLineSymbol` — for line features
- `QgsFillSymbol` — for polygon features

Every symbol consists of one or more symbol layers (classes derived from `QgsSymbolLayer`). The symbol layers do the actual rendering, the symbol class itself serves only as a container for the symbol layers.

Having an instance of a symbol (e.g. from a renderer), it is possible to explore it: the `type()` method says whether it is a marker, line or fill symbol. There is a `dump()` method which returns a brief description of the symbol. To get a list of symbol layers:

```

marker_symbol = QgsMarkerSymbol()
for i in range(marker_symbol.symbolLayerCount()):
    lyr = marker_symbol.symbolLayer(i)
    print("{}: {}".format(i, lyr.layerType()))

```

```
0: SimpleMarker
```

To find out symbol's color use `color()` method and `setColor()` to change its color. With marker symbols additionally you can query for the symbol size and rotation with the `size()` and `angle()` methods. For line symbols the `width()` method returns the line width.

Size and width are in millimeters by default, angles are in degrees.

Working with Symbol Layers

As said before, symbol layers (subclasses of `QgsSymbolLayer`) determine the appearance of the features. There are several basic symbol layer classes for general use. It is possible to implement new symbol layer types and thus arbitrarily customize how features will be rendered. The `layerType()` method uniquely identifies the symbol layer class — the basic and default ones are `SimpleMarker`, `SimpleLine` and `SimpleFill` symbol layers types.

You can get a complete list of the types of symbol layers you can create for a given symbol layer class with the following code:

```

1 from qgis.core import QgsSymbolLayerRegistry
2 myRegistry = QgsApplication.symbolLayerRegistry()
3 myMetadata = myRegistry.symbolLayerMetadata("SimpleFill")
4 for item in myRegistry.symbolLayersForType(QgsSymbol.Marker):
5     print(item)

```

```

1 AnimatedMarker
2 EllipseMarker
3 FilledMarker
4 FontMarker
5 GeometryGenerator
6 MaskMarker
7 RasterMarker
8 SimpleMarker
9 SvgMarker
10 VectorField

```

The `QgsSymbolLayerRegistry` class manages a database of all available symbol layer types.

To access symbol layer data, use its `properties()` method that returns a key-value dictionary of properties which determine the appearance. Each symbol layer type has a specific set of properties that it uses. Additionally, there are the generic methods `color()`, `size()`, `angle()` and `width()`, with their setter counterparts. Of course size and angle are available only for marker symbol layers and width for line symbol layers.

Creating Custom Symbol Layer Types

Imagine you would like to customize the way how the data gets rendered. You can create your own symbol layer class that will draw the features exactly as you wish. Here is an example of a marker that draws red circles with specified radius

```

1 from qgis.core import QgsMarkerSymbolLayer
2 from qgis.PyQt.QtGui import QColor
3
4 class FooSymbolLayer(QgsMarkerSymbolLayer):
5
6     def __init__(self, radius=4.0):
7         QgsMarkerSymbolLayer.__init__(self)
8         self.radius = radius
9         self.color = QColor(255,0,0)
10
11     def layerType(self):
12         return "FooMarker"
13
14     def properties(self):
15         return { "radius" : str(self.radius) }
16
17     def startRender(self, context):
18         pass
19
20     def stopRender(self, context):
21         pass
22
23     def renderPoint(self, point, context):
24         # Rendering depends on whether the symbol is selected (QGIS >= 1.5)
25         color = context.selectionColor() if context.selected() else self.color
26         p = context.renderContext().painter()
27         p.setPen(color)
28         p.drawEllipse(point, self.radius, self.radius)
29
30     def clone(self):
31         return FooSymbolLayer(self.radius)

```

The `layerType()` method determines the name of the symbol layer; it has to be unique among all symbol layers. The `properties()` method is used for persistence of attributes. The `clone()` method must return a copy of the symbol layer with all attributes being exactly the same. Finally there are rendering methods: `startRender()` is called before rendering the first feature, `stopRender()` when the rendering is done, and `renderPoint()` is called to do the rendering. The coordinates of the point(s) are already transformed to the output coordinates.

For polylines and polygons the only difference would be in the rendering method: you would use `renderPolyline()` which receives a list of lines, while `renderPolygon()` receives a list of points on the outer ring as the first parameter and a list of inner rings (or None) as a second parameter.

Usually it is convenient to add a GUI for setting attributes of the symbol layer type to allow users to customize the appearance: in case of our example above we can let user set circle radius. The following code implements such widget

```

1 from qgis.gui import QgsSymbolLayerWidget
2
3 class FooSymbolLayerWidget(QgsSymbolLayerWidget):
4     def __init__(self, parent=None):
5         QgsSymbolLayerWidget.__init__(self, parent)
6
7         self.layer = None
8
9         # setup a simple UI
10        self.label = QLabel("Radius:")
11        self.spinRadius = QDoubleSpinBox()
12        self.hbox = QHBoxLayout()
13        self.hbox.addWidget(self.label)
14        self.hbox.addWidget(self.spinRadius)
15        self.setLayout(self.hbox)
16        self.connect(self.spinRadius, SIGNAL("valueChanged(double)"), \
17                     self.radiusChanged)
18
19    def setSymbolLayer(self, layer):
20        if layer.layerType() != "FooMarker":
21            return
22        self.layer = layer
23        self.spinRadius.setValue(layer.radius)
24
25    def symbolLayer(self):
26        return self.layer
27
28    def radiusChanged(self, value):
29        self.layer.radius = value
30        self.emit(SIGNAL("changed()"))

```

This widget can be embedded into the symbol properties dialog. When the symbol layer type is selected in symbol properties dialog, it creates an instance of the symbol layer and an instance of the symbol layer widget. Then it calls the `setSymbolLayer()` method to assign the symbol layer to the widget. In that method the widget should update the UI to reflect the attributes of the symbol layer. The `symbolLayer()` method is used to retrieve the symbol layer again by the properties dialog to use it for the symbol.

On every change of attributes, the widget should emit the `changed()` signal to let the properties dialog update the symbol preview.

Now we are missing only the final glue: to make QGIS aware of these new classes. This is done by adding the symbol layer to registry. It is possible to use the symbol layer also without adding it to the registry, but some functionality will not work: e.g. loading of project files with the custom symbol layers or inability to edit the layer's attributes in GUI.

We will have to create metadata for the symbol layer

```

1 from qgis.core import QgsSymbol, QgsSymbolLayerAbstractMetadata, \
2   ↳QgsSymbolLayerRegistry
3
4 class FooSymbolLayerMetadata(QgsSymbolLayerAbstractMetadata):
5
6     def __init__(self):
7         super().__init__("FooMarker", "My new Foo marker", QgsSymbol.Marker)

```

(continues on next page)

(continued from previous page)

```

8  def createSymbolLayer(self, props):
9      radius = float(props["radius"]) if "radius" in props else 4.0
10     return FooSymbolLayer(radius)
11
12 fslmetadata = FooSymbolLayerMetadata()

```

```
QgsApplication.symbolLayerRegistry().addSymbolLayerType(fslmetadata)
```

You should pass layer type (the same as returned by the layer) and symbol type (marker/line/fill) to the constructor of the parent class. The `createSymbolLayer()` method takes care of creating an instance of symbol layer with attributes specified in the `props` dictionary. And there is the `createSymbolLayerWidget()` method which returns the settings widget for this symbol layer type.

The last step is to add this symbol layer to the registry — and we are done.

6.8.5 Creating Custom Renderers

It might be useful to create a new renderer implementation if you would like to customize the rules how to select symbols for rendering of features. Some use cases where you would want to do it: symbol is determined from a combination of fields, size of symbols changes depending on current scale etc.

The following code shows a simple custom renderer that creates two marker symbols and chooses randomly one of them for every feature

```

1  import random
2  from qgis.core import QgsWkbTypes, QgsSymbol, QgsFeatureRenderer
3
4
5  class RandomRenderer(QgsFeatureRenderer):
6      def __init__(self, syms=None):
7          super().__init__("RandomRenderer")
8          self.syms = syms if syms else [
9              QgsSymbol.defaultSymbol(QgsWkbTypes.geometryType(QgsWkbTypes.Point)),
10             QgsSymbol.defaultSymbol(QgsWkbTypes.geometryType(QgsWkbTypes.Point))
11         ]
12
13     def symbolForFeature(self, feature, context):
14         return random.choice(self.syms)
15
16     def startRender(self, context, fields):
17         super().startRender(context, fields)
18         for s in self.syms:
19             s.startRender(context, fields)
20
21     def stopRender(self, context):
22         super().stopRender(context)
23         for s in self.syms:
24             s.stopRender(context)
25
26     def usedAttributes(self, context):
27         return []
28
29     def clone(self):
30         return RandomRenderer(self.syms)

```

The constructor of the parent `QgsFeatureRenderer` class needs a renderer name (which has to be unique among renderers). The `symbolForFeature()` method is the one that decides what symbol will be used for a particular feature. `startRender()` and `stopRender()` take care of initialization/finalization of symbol rendering. The `usedAttributes()` method can return a list of field names that the renderer expects to be present. Finally, the `clone()` method should return a copy of the renderer.

Like with symbol layers, it is possible to attach a GUI for configuration of the renderer. It has to be derived from `QgsRendererWidget`. The following sample code creates a button that allows the user to set the first symbol

```

1 from qgis.gui import QgsRendererWidget, QgsColorButton
2
3
4 class RandomRendererWidget(QgsRendererWidget):
5     def __init__(self, layer, style, renderer):
6         super().__init__(layer, style)
7         if renderer is None or renderer.type() != "RandomRenderer":
8             self.r = RandomRenderer()
9         else:
10            self.r = renderer
11        # setup UI
12        self.btn1 = QgsColorButton()
13        self.btn1.setColor(self.r.syms[0].color())
14        self.vbox = QVBoxLayout()
15        self.vbox.addWidget(self.btn1)
16        self.setLayout(self.vbox)
17        self.btn1.colorChanged.connect(self.setColor1)
18
19    def setColor1(self):
20        color = self.btn1.color()
21        if not color.isValid(): return
22        self.r.syms[0].setColor(color)
23
24    def renderer(self):
25        return self.r

```

The constructor receives instances of the active layer (`QgsVectorLayer`), the global style (`QgsStyle`) and the current renderer. If there is no renderer or the renderer has different type, it will be replaced with our new renderer, otherwise we will use the current renderer (which has already the type we need). The widget contents should be updated to show current state of the renderer. When the renderer dialog is accepted, the widget's `renderer()` method is called to get the current renderer — it will be assigned to the layer.

The last missing bit is the renderer metadata and registration in registry, otherwise loading of layers with the renderer will not work and user will not be able to select it from the list of renderers. Let us finish our `RandomRenderer` example

```

1 from qgis.core import (
2     QgsRendererAbstractMetadata,
3     QgsRendererRegistry,
4     QgsApplication
5 )
6
7 class RandomRendererMetadata(QgsRendererAbstractMetadata):
8
9     def __init__(self):
10         super().__init__("RandomRenderer", "Random renderer")
11
12     def createRenderer(self, element):
13         return RandomRenderer()
14
15     def createRendererWidget(self, layer, style, renderer):
16         return RandomRendererWidget(layer, style, renderer)
17
18 rrmetadata = RandomRendererMetadata()

```

```
QgsApplication.rendererRegistry().addRenderer(rrmetadata)
```

Similarly as with symbol layers, abstract metadata constructor awaits renderer name, name visible for users and optionally name of renderer's icon. The `createRenderer()` method passes a `QDomElement` instance that can

be used to restore the renderer's state from the DOM tree. The `createRendererWidget()` method creates the configuration widget. It does not have to be present or can return `None` if the renderer does not come with GUI.

To associate an icon with the renderer you can assign it in the `QgsRendererAbstractMetadata` constructor as a third (optional) argument — the base class constructor in the `RandomRendererMetadata` `__init__()` function becomes

```
QgsRendererAbstractMetadata.__init__(self,
    "RandomRenderer",
    "Random renderer",
    QIcon(QPixmap("RandomRendererIcon.png", "png")))
```

The icon can also be associated at any later time using the `setIcon()` method of the metadata class. The icon can be loaded from a file (as shown above) or can be loaded from a [Qt resource](#) (PyQt5 includes .qrc compiler for Python).

6.9 Further Topics

TODO:

- creating/modifying symbols
- working with style (`QgsStyle`)
- working with color ramps (`QgsColorRamp`)
- exploring symbol layer and renderer registries

GEOMETRY HANDLING

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsGeometry,  
3     QgsGeometryCollection,  
4     QgsPoint,  
5     QgsPointXY,  
6     QgsWkbTypes,  
7     QgsProject,  
8     QgsFeatureRequest,  
9     QgsVectorLayer,  
10    QgsDistanceArea,  
11    QgsUnitTypes,  
12    QgsCoordinateTransform,  
13    QgsCoordinateReferenceSystem  
14 )
```

Points, linestrings and polygons that represent a spatial feature are commonly referred to as geometries. In QGIS they are represented with the `QgsGeometry` class.

Sometimes one geometry is actually a collection of simple (single-part) geometries. Such a geometry is called a multi-part geometry. If it contains just one type of simple geometry, we call it multi-point, multi-linestring or multi-polygon. For example, a country consisting of multiple islands can be represented as a multi-polygon.

The coordinates of geometries can be in any coordinate reference system (CRS). When fetching features from a layer, associated geometries will have coordinates in CRS of the layer.

Description and specifications of all possible geometries construction and relationships are available in the [OGC Simple Feature Access Standards](#) for advanced details.

7.1 Geometry Construction

PyQGIS provides several options for creating a geometry:

- from coordinates

```
1 gPnt = QgsGeometry.fromPointXY(QgsPointXY(1,1))  
2 print(gPnt)  
3 gLine = QgsGeometry.fromPolyline([QgsPoint(1, 1), QgsPoint(2, 2)])  
4 print(gLine)  
5 gPolygon = QgsGeometry.fromPolygonXY([[QgsPointXY(1, 1),  
6     QgsPointXY(2, 2), QgsPointXY(2, 1)])]  
7 print(gPolygon)
```

Coordinates are given using `QgsPoint` class or `QgsPointXY` class. The difference between these classes is that `QgsPoint` supports M and Z dimensions.

A Polyline (Linestring) is represented by a list of points.

A Polygon is represented by a list of linear rings (i.e. closed linestrings). The first ring is the outer ring (boundary), optional subsequent rings are holes in the polygon. Note that unlike some programs, QGIS will close the ring for you so there is no need to duplicate the first point as the last.

Multi-part geometries go one level further: multi-point is a list of points, multi-linestring is a list of linestrings and multi-polygon is a list of polygons.

- from well-known text (WKT)

```
geom = QgsGeometry.fromWkt("POINT(3 4)")
print(geom)
```

- from well-known binary (WKB)

```
1 g = QgsGeometry()
2 wkb = bytes.fromhex("01010000000000000000000045400000000000001440")
3 g.fromWkb(wkb)
4
5 # print WKT representation of the geometry
6 print(g.asWkt())
```

7.2 Access to Geometry

First, you should find out the geometry type. The `wkbType()` method is the one to use. It returns a value from the `QgsWkbTypes.Type` enumeration.

```
1 print(gPnt.wkbType())
2 # output: 1
3 print(gLine.wkbType())
4 # output: 2
5 print(gPolygon.wkbType())
6 # output: 3
```

As an alternative, one can use the `type()` method which returns a value from the `QgsWkbTypes.GeometryType` enumeration.

```
print(gLine.type())
# output: 1
```

You can use the `displayString()` function to get a human readable geometry type.

```
1 print(QgsWkbTypes.displayString(gPnt.wkbType()))
2 # output: 'Point'
3 print(QgsWkbTypes.displayString(gLine.wkbType()))
4 # output: 'LineString'
5 print(QgsWkbTypes.displayString(gPolygon.wkbType()))
6 # output: 'Polygon'
```

There is also a helper function `isMultipart()` to find out whether a geometry is multipart or not.

To extract information from a geometry there are accessor functions for every vector type. Here's an example on how to use these accessors:

```
1 print(gPnt.asPoint())
2 # output: <QgsPointXY: POINT(1 1)>
3 print(gLine.asPolyline())
```

(continues on next page)

(continued from previous page)

```

4 # output: [<QgsPointXY: POINT(1 1)>, <QgsPointXY: POINT(2 2)>]
5 print(gPolygon.asPolygon())
6 # output: [[<QgsPointXY: POINT(1 1)>, <QgsPointXY: POINT(2 2)>, <QgsPointXY:
  ↳POINT(2 1)>, <QgsPointXY: POINT(1 1)>]]

```

Note

The tuples (x,y) are not real tuples, they are `QgsPoint` objects, the values are accessible with `x()` and `y()` methods.

For multipart geometries there are similar accessor functions: `asMultiPoint()`, `asMultiPolyline()` and `asMultiPolygon()`.

It is possible to iterate over all the parts of a geometry, regardless of the geometry's type. E.g.

```

geom = QgsGeometry.fromWkt( 'MultiPoint( 0 0, 1 1, 2 2)' )
for part in geom.parts():
    print(part.asWkt())

```

```

Point (0 0)
Point (1 1)
Point (2 2)

```

```

geom = QgsGeometry.fromWkt( 'LineString( 0 0, 10 10 )' )
for part in geom.parts():
    print(part.asWkt())

```

```

LineString (0 0, 10 10)

```

```

gc = QgsGeometryCollection()
gc.fromWkt( 'GeometryCollection( Point(1 2), Point(11 12), LineString(33 34, 44 45))
  ↳' )
print(gc[1].asWkt())

```

```

Point (11 12)

```

It's also possible to modify each part of the geometry using `QgsGeometry.parts()` method.

```

1 geom = QgsGeometry.fromWkt( 'MultiPoint( 0 0, 1 1, 2 2)' )
2 for part in geom.parts():
3     part.transform(QgsCoordinateTransform(
4         QgsCoordinateReferenceSystem("EPSG:4326"),
5         QgsCoordinateReferenceSystem("EPSG:3111"),
6         QgsProject.instance())
7     )
8
9 print(geom.asWkt())

```

```

MultiPoint ((-10334726.79314758814871311 -5360105.10101194866001606), (-10462133.
  ↳82917747274041176 -5217484.34365733992308378), (-10589398.51346861757338047 -
  ↳5072020.35880533326417208))

```

7.3 Geometry Predicates and Operations

QGIS uses GEOS library for advanced geometry operations such as geometry predicates (`contains()`, `intersects()`, ...) and set operations (`combine()`, `difference()`, ...). It can also compute geometric properties of geometries, such as area (in the case of polygons) or lengths (for polygons and lines).

Let's see an example that combines iterating over the features in a given layer and performing some geometric computations based on their geometries. The below code will compute and print the area and perimeter of each country in the `countries` layer within our tutorial QGIS project.

The following code assumes `layer` is a `QgsVectorLayer` object that has Polygon feature type.

```
1 # let's access the 'countries' layer
2 layer = QgsProject.instance().mapLayersByName('countries')[0]
3
4 # let's filter for countries that begin with Z, then get their features
5 query = '"name" LIKE \'Z%\''
6 features = layer.getFeatures(QgsFeatureRequest().setFilterExpression(query))
7
8 # now loop through the features, perform geometry computation and print the results
9 for f in features:
10     geom = f.geometry()
11     name = f.attribute('NAME')
12     print(name)
13     print('Area: ', geom.area())
14     print('Perimeter: ', geom.length())
```

```
1 Zambia
2 Area: 62.82279065343119
3 Perimeter: 50.65232014052552
4 Zimbabwe
5 Area: 33.41113559136517
6 Perimeter: 26.608288555013935
```

Now you have calculated and printed the areas and perimeters of the geometries. You may however quickly notice that the values are strange. That is because areas and perimeters don't take CRS into account when computed using the `area()` and `length()` methods from the `QgsGeometry` class. For a more powerful area and distance calculation, the `QgsDistanceArea` class can be used, which can perform ellipsoid based calculations:

The following code assumes `layer` is a `QgsVectorLayer` object that has Polygon feature type.

```
1 d = QgsDistanceArea()
2 d.setEllipsoid('WGS84')
3
4 layer = QgsProject.instance().mapLayersByName('countries')[0]
5
6 # let's filter for countries that begin with Z, then get their features
7 query = '"name" LIKE \'Z%\''
8 features = layer.getFeatures(QgsFeatureRequest().setFilterExpression(query))
9
10 for f in features:
11     geom = f.geometry()
12     name = f.attribute('NAME')
13     print(name)
14     print("Perimeter (m):", d.measurePerimeter(geom))
15     print("Area (m2):", d.measureArea(geom))
16
17 # let's calculate and print the area again, but this time in square kilometers
18 print("Area (km2):", d.convertAreaMeasurement(d.measureArea(geom), QgsUnitTypes.
    ↳ AreaSquareKilometers))
```

```

1 Zambia
2 Perimeter (m): 5539361.250294601
3 Area (m2): 751989035032.9031
4 Area (km2): 751989.0350329031
5 Zimbabwe
6 Perimeter (m): 2865021.3325076113
7 Area (m2): 389267821381.6008
8 Area (km2): 389267.8213816008

```

Alternatively, you may want to know the distance between two points.

```

1 d = QgsDistanceArea()
2 d.setEllipsoid('WGS84')
3
4 # Let's create two points.
5 # Santa claus is a workaholic and needs a summer break,
6 # lets see how far is Tenerife from his home
7 santa = QgsPointXY(25.847899, 66.543456)
8 tenerife = QgsPointXY(-16.5735, 28.0443)
9
10 print("Distance in meters: ", d.measureLine(santa, tenerife))

```

You can find many example of algorithms that are included in QGIS and use these methods to analyze and transform vector data. Here are some links to the code of a few of them.

- Distance and area using the `QgsDistanceArea` class: [Distance matrix algorithm](#)
- [Lines to polygons algorithm](#)

PROJECTIONS SUPPORT

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsCoordinateReferenceSystem,  
3     QgsCoordinateTransform,  
4     QgsProject,  
5     QgsPointXY,  
6 )
```

8.1 Coordinate reference systems

Coordinate reference systems (CRS) are encapsulated by the `QgsCoordinateReferenceSystem` class. Instances of this class can be created in several different ways:

- specify CRS by its ID

```
# EPSG 4326 is allocated for WGS84  
crs = QgsCoordinateReferenceSystem("EPSG:4326")  
print(crs.isValid())
```

```
True
```

QGIS supports different CRS identifiers with the following formats:

- EPSG:<code> — ID assigned by the EPSG organization - handled with `createFromOgcWms()`
- POSTGIS:<srid> — ID used in PostGIS databases - handled with `createFromSrid()`
- INTERNAL:<srsid> — ID used in the internal QGIS database - handled with `createFromSrsId()`
- PROJ:<proj> - handled with `createFromProj()`
- WKT:<wkt> - handled with `createFromWkt()`

If no prefix is specified, WKT definition is assumed.

- specify CRS by its well-known text (WKT)

```
1 wkt = 'GEOGCS["WGS84", DATUM["WGS84", SPHEROID["WGS84", 6378137.0, 298.  
   ↳257223563]], ' \\  
2     'PRIMEM["Greenwich", 0.0], UNIT["degree",0.017453292519943295], ' \\  
3     'AXIS["Longitude",EAST], AXIS["Latitude",NORTH]] '  
4 crs = QgsCoordinateReferenceSystem(wkt)  
5 print(crs.isValid())
```

```
True
```

- create an invalid CRS and then use one of the `create*` functions to initialize it. In the following example we use a Proj string to initialize the projection.

```
crs = QgsCoordinateReferenceSystem()
crs.createFromProj("+proj=longlat +ellps=WGS84 +datum=WGS84 +no_defs")
print(crs.isValid())
```

```
True
```

It's wise to check whether creation (i.e. lookup in the database) of the CRS has been successful: `isValid()` must return `True`.

Note that for initialization of spatial reference systems QGIS needs to look up appropriate values in its internal database `srs.db`. Thus in case you create an independent application you need to set paths correctly with `QgsApplication.setPrefixPath()`, otherwise it will fail to find the database. If you are running the commands from the QGIS Python console or developing a plugin you do not care: everything is already set up for you.

Accessing spatial reference system information:

```
1 crs = QgsCoordinateReferenceSystem("EPSG:4326")
2
3 print("QGIS CRS ID:", crs.srsid())
4 print("PostGIS SRID:", crs.postgisSrid())
5 print("Description:", crs.description())
6 print("Projection Acronym:", crs.projectionAcronym())
7 print("Ellipsoid Acronym:", crs.ellipsoidAcronym())
8 print("Proj String:", crs.toProj())
9 # check whether it's geographic or projected coordinate system
10 print("Is geographic:", crs.isGeographic())
11 # check type of map units in this CRS (values defined in Qgs::units enum)
12 print("Map units:", crs.mapUnits())
```

Output:

```
1 QGIS CRS ID: 3452
2 PostGIS SRID: 4326
3 Description: WGS 84
4 Projection Acronym: longlat
5 Ellipsoid Acronym: EPSG:7030
6 Proj String: +proj=longlat +datum=WGS84 +no_defs
7 Is geographic: True
8 Map units: 6
```

8.2 CRS Transformation

You can do transformation between different spatial reference systems by using the `QgsCoordinateTransform` class. The easiest way to use it is to create a source and destination CRS and construct a `QgsCoordinateTransform` instance with them and the current project. Then just repeatedly call `transform()` function to do the transformation. By default it does forward transformation, but it is capable to do also inverse transformation.

```
1 crsSrc = QgsCoordinateReferenceSystem("EPSG:4326") # WGS 84
2 crsDest = QgsCoordinateReferenceSystem("EPSG:32633") # WGS 84 / UTM zone 33N
3 transformContext = QgsProject.instance().transformContext()
4 xform = QgsCoordinateTransform(crsSrc, crsDest, transformContext)
5
6 # forward transformation: src -> dest
7 pt1 = xform.transform(QgsPointXY(18,5))
```

(continues on next page)

(continued from previous page)

```
8 print("Transformed point:", pt1)
9
10 # inverse transformation: dest -> src
11 pt2 = xform.transform(pt1, QgsCoordinateTransform.ReverseTransform)
12 print("Transformed back:", pt2)
```

Output:

```
Transformed point: <QgsPointXY: POINT(832713.79873844375833869 553423.
↪98688333143945783)>
Transformed back: <QgsPointXY: POINT(18 4.99999999999999911)>
```


USING THE MAP CANVAS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.PyQt.QtGui import (
2     QColor,
3 )
4
5 from qgis.PyQt.QtCore import Qt, QRectF
6
7 from qgis.PyQt.QtWidgets import QMenu
8
9 from qgis.core import (
10     QgsVectorLayer,
11     QgsPoint,
12     QgsPointXY,
13     QgsProject,
14     QgsGeometry,
15     QgsMapRendererJob,
16     QgsWkbTypes,
17 )
18
19 from qgis.gui import (
20     QgsMapCanvas,
21     QgsVertexMarker,
22     QgsMapCanvasItem,
23     QgsMapMouseEvent,
24     QgsRubberBand,
25 )
```

The Map canvas widget is probably the most important widget within QGIS because it shows the map composed from overlaid map layers and allows interaction with the map and layers. The canvas always shows a part of the map defined by the current canvas extent. The interaction is done through the use of **map tools**: there are tools for panning, zooming, identifying layers, measuring, vector editing and others. Similar to other graphics programs, there is always one tool active and the user can switch between the available tools.

The map canvas is implemented with the `QgsMapCanvas` class in the `qgis.gui` module. The implementation is based on the Qt Graphics View framework. This framework generally provides a surface and a view where custom graphics items are placed and user can interact with them. We will assume that you are familiar enough with Qt to understand the concepts of the graphics scene, view and items. If not, please read the [overview of the framework](#).

Whenever the map has been panned, zoomed in/out (or some other action that triggers a refresh), the map is rendered again within the current extent. The layers are rendered to an image (using the `QgsMapRendererJob` class) and that image is displayed on the canvas. The `QgsMapCanvas` class also controls refreshing of the rendered map. Besides this item which acts as a background, there may be more **map canvas items**.

Typical map canvas items are rubber bands (used for measuring, vector editing etc.) or vertex markers. The canvas

items are usually used to give visual feedback for map tools, for example, when creating a new polygon, the map tool creates a rubber band canvas item that shows the current shape of the polygon. All map canvas items are subclasses of `QgsMapCanvasItem` which adds some more functionality to the basic `QGraphicsItem` objects.

To summarize, the map canvas architecture consists of three concepts:

- map canvas — for viewing of the map
- map canvas items — additional items that can be displayed on the map canvas
- map tools — for interaction with the map canvas

9.1 Embedding Map Canvas

Map canvas is a widget like any other Qt widget, so using it is as simple as creating and showing it.

```
canvas = QgsMapCanvas()
canvas.show()
```

This produces a standalone window with map canvas. It can be also embedded into an existing widget or window. When using `.ui` files and Qt Designer, place a `QWidget` on the form and promote it to a new class: set `QgsMapCanvas` as class name and set `qgis.gui` as header file. The `pyuic5` utility will take care of it. This is a very convenient way of embedding the canvas. The other possibility is to manually write the code to construct map canvas and other widgets (as children of a main window or dialog) and create a layout.

By default, map canvas has black background and does not use anti-aliasing. To set white background and enable anti-aliasing for smooth rendering

```
canvas.setCanvasColor(Qt.white)
canvas.enableAntiAliasing(True)
```

(In case you are wondering, `Qt` comes from `PyQt.QtCore` module and `Qt.white` is one of the predefined `QColor` instances.)

Now it is time to add some map layers. We will first open a layer and add it to the current project. Then we will set the canvas extent and set the list of layers for the canvas.

```
1 vlayer = QgsVectorLayer("testdata/data/data.gpkg|layername=airports", "Airports_
   ↳layer", "ogr")
2 if not vlayer.isValid():
3     print("Layer failed to load!")
4
5 # add layer to the registry
6 QgsProject.instance().addMapLayer(vlayer)
7
8 # set extent to the extent of our layer
9 canvas.setExtent(vlayer.extent())
10
11 # set the map canvas layer set
12 canvas.setLayers([vlayer])
```

After executing these commands, the canvas should show the layer you have loaded.

9.2 Rubber Bands and Vertex Markers

To show some additional data on top of the map in canvas, use map canvas items. It is possible to create custom canvas item classes (covered below), however there are two useful canvas item classes for convenience: `QgsRubberBand` for drawing polylines or polygons, and `QgsVertexMarker` for drawing points. They both work with map coordinates, so the shape is moved/scaled automatically when the canvas is being panned or zoomed.

To show a polyline:

```
r = QgsRubberBand(canvas, QgsWkbTypes.LineGeometry) # line
points = [QgsPoint(-100, 45), QgsPoint(10, 60), QgsPoint(120, 45)]
r.setToGeometry(QgsGeometry.fromPolyline(points), None)
```

To show a polygon

```
r = QgsRubberBand(canvas, QgsWkbTypes.PolygonGeometry) # polygon
points = [[QgsPointXY(-100, 35), QgsPointXY(10, 50), QgsPointXY(120, 35)]]
r.setToGeometry(QgsGeometry.fromPolygonXY(points), None)
```

Note that points for polygon is not a plain list: in fact, it is a list of rings containing linear rings of the polygon: first ring is the outer border, further (optional) rings correspond to holes in the polygon.

Rubber bands allow some customization, namely to change their color and line width

```
r.setColor(QColor(0, 0, 255))
r.setWidth(3)
```

The canvas items are bound to the canvas scene. To temporarily hide them (and show them again), use the `hide()` and `show()` combo. To completely remove the item, you have to remove it from the scene of the canvas

```
canvas.scene().removeItem(r)
```

(in C++ it's possible to just delete the item, however in Python `del r` would just delete the reference and the object will still exist as it is owned by the canvas)

Rubber band can be also used for drawing points, but the `QgsVertexMarker` class is better suited for this (`QgsRubberBand` would only draw a rectangle around the desired point).

You can use the vertex marker like this:

```
m = QgsVertexMarker(canvas)
m.setCenter(QgsPointXY(10, 40))
```

This will draw a red cross on position [10,45]. It is possible to customize the icon type, size, color and pen width

```
m.setColor(QColor(0, 255, 0))
m.setIconSize(5)
m.setIconType(QgsVertexMarker.ICON_BOX) # or ICON_CROSS, ICON_X
m.setPenWidth(3)
```

For temporary hiding of vertex markers and removing them from canvas, use the same methods as for rubber bands.

9.3 Using Map Tools with Canvas

The following example constructs a window that contains a map canvas and basic map tools for map panning and zooming. Actions are created for activation of each tool: panning is done with `QgsMapToolPan`, zooming in/out with a pair of `QgsMapToolZoom` instances. The actions are set as checkable and later assigned to the tools to allow automatic handling of checked/unchecked state of the actions – when a map tool gets activated, its action is marked as selected and the action of the previous map tool is deselected. The map tools are activated using `setMapTool()` method.

```

1 from qgis.gui import *
2 from qgis.PyQt.QtWidgets import QAction, QMainWindow
3 from qgis.PyQt.QtCore import Qt
4
5 class MyWnd(QMainWindow):
6     def __init__(self, layer):
7         QMainWindow.__init__(self)
8
9         self.canvas = QgsMapCanvas()
10        self.canvas.setCanvasColor(Qt.white)
11
12        self.canvas.setExtent(layer.extent())
13        self.canvas.setLayers([layer])
14
15        self.setCentralWidget(self.canvas)
16
17        self.actionZoomIn = QAction("Zoom in", self)
18        self.actionZoomOut = QAction("Zoom out", self)
19        self.actionPan = QAction("Pan", self)
20
21        self.actionZoomIn.setCheckable(True)
22        self.actionZoomOut.setCheckable(True)
23        self.actionPan.setCheckable(True)
24
25        self.actionZoomIn.triggered.connect(self.zoomIn)
26        self.actionZoomOut.triggered.connect(self.zoomOut)
27        self.actionPan.triggered.connect(self.pan)
28
29        self.toolbar = self.addToolBar("Canvas actions")
30        self.toolbar.addAction(self.actionZoomIn)
31        self.toolbar.addAction(self.actionZoomOut)
32        self.toolbar.addAction(self.actionPan)
33
34        # create the map tools
35        self.toolPan = QgsMapToolPan(self.canvas)
36        self.toolPan.setAction(self.actionPan)
37        self.toolZoomIn = QgsMapToolZoom(self.canvas, False) # false = in
38        self.toolZoomIn.setAction(self.actionZoomIn)
39        self.toolZoomOut = QgsMapToolZoom(self.canvas, True) # true = out
40        self.toolZoomOut.setAction(self.actionZoomOut)
41
42        self.pan()
43
44        def zoomIn(self):
45            self.canvas.setMapTool(self.toolZoomIn)
46
47        def zoomOut(self):
48            self.canvas.setMapTool(self.toolZoomOut)
49
50        def pan(self):
51            self.canvas.setMapTool(self.toolPan)

```

You can try the above code in the Python console editor. To invoke the canvas window, add the following lines to

instantiate the `MyWnd` class. They will render the currently selected layer on the newly created canvas

```
w = MyWnd(iface.activeLayer())
w.show()
```

9.3.1 Select a feature using `QgsMapToolIdentifyFeature`

You can use the map tool `QgsMapToolIdentifyFeature` for asking to the user to select a feature that will be sent to a callback function.

```
1 def callback(feature):
2     """Code called when the feature is selected by the user"""
3     print("You clicked on feature {}".format(feature.id()))
4
5 canvas = iface.mapCanvas()
6 feature_identifier = QgsMapToolIdentifyFeature(canvas)
7
8 # indicates the layer on which the selection will be done
9 feature_identifier.setLayer(vlayer)
10
11 # use the callback as a slot triggered when the user identifies a feature
12 feature_identifier.featureIdentified.connect(callback)
13
14 # activation of the map tool
15 canvas.setMapTool(feature_identifier)
```

9.3.2 Add items to map canvas contextual menu

Interaction with map canvas can also be done through entries you may add to its contextual menu using the `contextMenuAboutToShow` signal.

The following code adds *My menu ► My Action* action next to default entries when you right-click over the map canvas.

```
1 # a slot to populate the context menu
2 def populateContextMenu(menu: QMenu, event: QgsMapMouseEvent):
3     subMenu = menu.addMenu('My Menu')
4     action = subMenu.addAction('My Action')
5     action.triggered.connect(lambda *args:
6                             print(f'Action triggered at {event.x()}, {event.y()}'))
7
8 canvas.contextMenuAboutToShow.connect(populateContextMenu)
9 canvas.show()
```

9.4 Writing Custom Map Tools

You can write your custom tools, to implement a custom behavior to actions performed by users on the canvas.

Map tools should inherit from the `QgsMapTool` class or any derived class, and selected as active tools in the canvas using the `setMapTool()` method as we have already seen.

Here is an example of a map tool that allows to define a rectangular extent by clicking and dragging on the canvas. When the rectangle is defined, it prints its boundary coordinates in the console. It uses the rubber band elements described before to show the selected rectangle as it is being defined.

```

1 class RectangleMapTool(QgsMapToolEmitPoint):
2     def __init__(self, canvas):
3         self.canvas = canvas
4         QgsMapToolEmitPoint.__init__(self, self.canvas)
5         self.rubberBand = QgsRubberBand(self.canvas, QgsWkbTypes.PolygonGeometry)
6         self.rubberBand.setColor(Qt.red)
7         self.rubberBand.setWidth(1)
8         self.reset()
9
10    def reset(self):
11        self.startPoint = self.endPoint = None
12        self.isEmittingPoint = False
13        self.rubberBand.reset(QgsWkbTypes.PolygonGeometry)
14
15    def canvasPressEvent(self, e):
16        self.startPoint = self.toMapCoordinates(e.pos())
17        self.endPoint = self.startPoint
18        self.isEmittingPoint = True
19        self.showRect(self.startPoint, self.endPoint)
20
21    def canvasReleaseEvent(self, e):
22        self.isEmittingPoint = False
23        r = self.rectangle()
24        if r is not None:
25            print("Rectangle:", r.xMinimum(),
26                  r.yMinimum(), r.xMaximum(), r.yMaximum()
27                  )
28
29    def canvasMoveEvent(self, e):
30        if not self.isEmittingPoint:
31            return
32
33        self.endPoint = self.toMapCoordinates(e.pos())
34        self.showRect(self.startPoint, self.endPoint)
35
36    def showRect(self, startPoint, endPoint):
37        self.rubberBand.reset(QgsWkbTypes.PolygonGeometry)
38        if startPoint.x() == endPoint.x() or startPoint.y() == endPoint.y():
39            return
40
41        point1 = QgsPointXY(startPoint.x(), startPoint.y())
42        point2 = QgsPointXY(startPoint.x(), endPoint.y())
43        point3 = QgsPointXY(endPoint.x(), endPoint.y())
44        point4 = QgsPointXY(endPoint.x(), startPoint.y())
45
46        self.rubberBand.addPoint(point1, False)
47        self.rubberBand.addPoint(point2, False)
48        self.rubberBand.addPoint(point3, False)
49        self.rubberBand.addPoint(point4, True)    # true to update canvas
50        self.rubberBand.show()
51
52    def rectangle(self):
53        if self.startPoint is None or self.endPoint is None:
54            return None
55        elif (self.startPoint.x() == self.endPoint.x() or \
56              self.startPoint.y() == self.endPoint.y()):
57            return None
58
59        return QgsRectangle(self.startPoint, self.endPoint)
60
61    def deactivate(self):
62        QgsMapTool.deactivate(self)

```

(continues on next page)

(continued from previous page)

```
63 self.deactivated.emit()
```

9.5 Writing Custom Map Canvas Items

Here is an example of a custom canvas item that draws a circle:

```

1 class CircleCanvasItem(QgsMapCanvasItem):
2     def __init__(self, canvas):
3         super().__init__(canvas)
4         self.center = QgsPoint(0, 0)
5         self.size = 100
6
7     def setCenter(self, center):
8         self.center = center
9
10    def center(self):
11        return self.center
12
13    def setSize(self, size):
14        self.size = size
15
16    def size(self):
17        return self.size
18
19    def boundingRect(self):
20        return QRectF(self.center.x() - self.size/2,
21                      self.center.y() - self.size/2,
22                      self.center.x() + self.size/2,
23                      self.center.y() + self.size/2)
24
25    def paint(self, painter, option, widget):
26        path = QPainterPath()
27        path.moveTo(self.center.x(), self.center.y());
28        path.arcTo(self.boundingRect(), 0.0, 360.0)
29        painter.fillPath(path, QColor("red"))
30
31
32    # Using the custom item:
33    item = CircleCanvasItem(iface.mapCanvas())
34    item.setCenter(QgsPointXY(200,200))
35    item.setSize(80)

```


MAP RENDERING AND PRINTING

Hint

The code snippets on this page need the following imports:

```
1 import os
2
3 from qgis.core import (
4     QgsGeometry,
5     QgsMapSettings,
6     QgsPrintLayout,
7     QgsMapSettings,
8     QgsMapRendererParallelJob,
9     QgsLayoutItemLabel,
10    QgsLayoutItemLegend,
11    QgsLayoutItemMap,
12    QgsLayoutItemPolygon,
13    QgsLayoutItemScaleBar,
14    QgsLayoutExporter,
15    QgsLayoutItem,
16    QgsLayoutPoint,
17    QgsLayoutSize,
18    QgsUnitTypes,
19    QgsProject,
20    QgsFillSymbol,
21    QgsAbstractValidityCheck,
22    check,
23 )
24
25 from qgis.PyQt.QtGui import (
26     QPolygonF,
27     QColor,
28 )
29
30 from qgis.PyQt.QtCore import (
31     QPointF,
32     QRectF,
33     QSize,
34 )
```

There are generally two approaches when input data should be rendered as a map: either do it quick way using *QgsMapRendererJob* or produce more fine-tuned output by composing the map with the *QgsLayout* class.

10.1 Simple Rendering

The rendering is done creating a `QgsMapSettings` object to define the rendering settings, and then constructing a `QgsMapRendererJob` with those settings. The latter is then used to create the resulting image.

Here's an example:

```

1 image_location = os.path.join(QgsProject.instance().homePath(), "render.png")
2
3 vlayer = iface.activeLayer()
4 settings = QgsMapSettings()
5 settings.setLayers([vlayer])
6 settings.setBackgroundColor(QColor(255, 255, 255))
7 settings.setOutputSize(QSize(800, 600))
8 settings.setExtent(vlayer.extent())
9
10 render = QgsMapRendererParallelJob(settings)
11
12 def finished():
13     img = render.renderedImage()
14     # save the image; e.g. img.save("/Users/myuser/render.png", "png")
15     img.save(image_location, "png")
16
17 render.finished.connect(finished)
18
19 # Start the rendering
20 render.start()
21
22 # The following loop is not normally required, we
23 # are using it here because this is a standalone example.
24 from qgis.PyQt.QtCore import QEventLoop
25 loop = QEventLoop()
26 render.finished.connect(loop.quit)
27 loop.exec_()
```

10.2 Rendering layers with different CRS

If you have more than one layer and they have a different CRS, the simple example above will probably not work: to get the right values from the extent calculations you have to explicitly set the destination CRS

```

layers = [iface.activeLayer()]
settings = QgsMapSettings()
settings.setLayers(layers)
settings.setDestinationCrs(layers[0].crs())
```

10.3 Output using print layout

Print layout is a very handy tool if you would like to do a more sophisticated output than the simple rendering shown above. It is possible to create complex map layouts consisting of map views, labels, legend, tables and other elements that are usually present on paper maps. The layouts can be then exported to PDF, SVG, raster images or directly printed on a printer.

The layout consists of a bunch of classes. They all belong to the core library. QGIS application has a convenient GUI for placement of the elements, though it is not available in the GUI library. If you are not familiar with [Qt Graphics View framework](#), then you are encouraged to check the documentation now, because the layout is based on it.

The central class of the layout is the `QgsLayout` class, which is derived from the Qt `QGraphicsScene` class. Let us create an instance of it:

```
project = QgsProject.instance()
layout = QgsPrintLayout(project)
layout.initializeDefaults()
```

This initializes the layout with some default settings, specifically by adding an empty A4 page to the layout. You can create layouts without calling the `initializeDefaults()` method, but you'll need to take care of adding pages to the layout yourself.

The previous code creates a “temporary” layout that is not visible in the GUI. It can be handy to e.g. quickly add some items and export without modifying the project itself nor expose these changes to the user. If you want the layout to be saved/restored along with the project and available in the layout manager, then add:

```
layout.setName("MyLayout")
project.layoutManager().addLayout(layout)
```

Now we can add various elements (map, label, ...) to the layout. All these objects are represented by classes that inherit from the base `QgsLayoutItem` class.

Here's a description of some of the main layout items that can be added to a layout.

- **map** — Here we create a map of a custom size and render the current map canvas

```
1 map = QgsLayoutItemMap(layout)
2 # Set map item position and size (by default, it is a 0 width/0 height item
  ↳ placed at 0,0)
3 map.attemptMove(QgsLayoutPoint(5,5, QgsUnitTypes.LayoutMillimeters))
4 map.attemptResize(QgsLayoutSize(200,200, QgsUnitTypes.LayoutMillimeters))
5 # Provide an extent to render
6 map.zoomToExtent(iface.mapCanvas().extent())
7 layout.addLayoutItem(map)
```

- **label** — allows displaying labels. It is possible to modify its font, color, alignment and margin

```
label = QgsLayoutItemLabel(layout)
label.setText("Hello world")
label.adjustSizeToText()
layout.addLayoutItem(label)
```

- **legend**

```
legend = QgsLayoutItemLegend(layout)
legend.setLinkedMap(map) # map is an instance of QgsLayoutItemMap
layout.addLayoutItem(legend)
```

- **scale bar**

```
1 item = QgsLayoutItemScaleBar(layout)
2 item.setStyle('Numeric') # optionally modify the style
3 item.setLinkedMap(map) # map is an instance of QgsLayoutItemMap
4 item.applyDefaultSize()
5 layout.addLayoutItem(item)
```

- **nodes based shape**

```
1 polygon = QPolygonF()
2 polygon.append(QPointF(0.0, 0.0))
3 polygon.append(QPointF(100.0, 0.0))
4 polygon.append(QPointF(200.0, 100.0))
5 polygon.append(QPointF(100.0, 200.0))
6
7 polygonItem = QgsLayoutItemPolygon(polygon, layout)
8 layout.addLayoutItem(polygonItem)
```

(continues on next page)

(continued from previous page)

```

9
10 props = {}
11 props["color"] = "green"
12 props["style"] = "solid"
13 props["style_border"] = "solid"
14 props["color_border"] = "black"
15 props["width_border"] = "10.0"
16 props["joinstyle"] = "miter"
17
18 symbol = QgsFillSymbol.createSimple(props)
19 polygonItem.setSymbol(symbol)

```

Once an item is added to the layout, it can be moved and resized:

```

item.attemptMove(QgsLayoutPoint(1.4, 1.8, QgsUnitTypes.LayoutCentimeters))
item.attemptResize(QgsLayoutSize(2.8, 2.2, QgsUnitTypes.LayoutCentimeters))

```

A frame is drawn around each item by default. You can remove it as follows:

```

# for a composer label
label.setFrameEnabled(False)

```

Besides creating the layout items by hand, QGIS has support for layout templates which are essentially compositions with all their items saved to a .qpt file (with XML syntax).

Once the composition is ready (the layout items have been created and added to the composition), we can proceed to produce a raster and/or vector output.

10.3.1 Checking layout validity

A layout is a made of a set of interconnected items and it can happen that these connections are broken during modifications (a legend connected to a removed map, an image item with missing source file,...) or you may want to apply custom constraints to the layout items. The `QgsAbstractValidityCheck` helps you achieve this.

A basic check looks like:

```

@check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
def my_layout_check(context, feedback):
    results = ...
    return results

```

Here's a check which throws a warning whenever a layout map item is set to the web mercator projection:

```

1 @check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
2 def layout_map_crs_choice_check(context, feedback):
3     layout = context.layout
4     results = []
5     for i in layout.items():
6         if isinstance(i, QgsLayoutItemMap) and i.crs().authid() == 'EPSG:3857':
7             res = QgsValidityCheckResult()
8             res.type = QgsValidityCheckResult.Warning
9             res.title = 'Map projection is misleading'
10            res.detailedDescription = 'The projection for the map item {} is set to <i>
↪Web Mercator (EPSG:3857)</i> which misrepresents areas and shapes. Consider
↪using an appropriate local projection instead.'.format(i.displayName())
11            results.append(res)
12
13    return results

```

And here's a more complex example, which throws a warning if any layout map items are set to a CRS which is only valid outside of the extent shown in that map item:

```

1 @check.register(type=QgsAbstractValidityCheck.TypeLayoutCheck)
2 def layout_map_crs_area_check(context, feedback):
3     layout = context.layout
4     results = []
5     for i in layout.items():
6         if isinstance(i, QgsLayoutItemMap):
7             bounds = i.crs().bounds()
8             ct = QgsCoordinateTransform(QgsCoordinateReferenceSystem('EPSG:4326'),
↪ i.crs(), QgsProject.instance())
9             bounds_crs = ct.transformBoundingBox(bounds)
10
11             if not bounds_crs.contains(i.extent()):
12                 res = QgsValidityCheckResult()
13                 res.type = QgsValidityCheckResult.Warning
14                 res.title = 'Map projection is incorrect'
15                 res.detailedDescription = 'The projection for the map item {} is
↪ set to \'{}\ ', which is not valid for the area displayed within the map.'.
↪ format(i.displayName(), i.crs().authid())
16                 results.append(res)
17
18     return results

```

10.3.2 Exporting the layout

To export a layout, the `QgsLayoutExporter` class must be used.

```

1 base_path = os.path.join(QgsProject.instance().homePath())
2 pdf_path = os.path.join(base_path, "output.pdf")
3
4 exporter = QgsLayoutExporter(layout)
5 exporter.exportToPdf(pdf_path, QgsLayoutExporter.PdfExportSettings())

```

Use `exportToSvg()` or `exportToImage()` in case you want to export to respectively an SVG or image file instead of a PDF file.

10.3.3 Exporting a layout atlas

If you want to export all pages from a layout that has the atlas option configured and enabled, you need to use the `atlas()` method in the exporter (`QgsLayoutExporter`) with small adjustments. In the following example, the pages are exported to PNG images:

```

exporter.exportToImage(layout.atlas(), base_path, 'png', QgsLayoutExporter.
↪ ImageExportSettings())

```

Notice that the outputs will be saved in the base path folder, using the output filename expression configured on atlas.

EXPRESSIONS, FILTERING AND CALCULATING VALUES

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     edit,  
3     QgsExpression,  
4     QgsExpressionContext,  
5     QgsFeature,  
6     QgsFeatureRequest,  
7     QgsField,  
8     QgsFields,  
9     QgsVectorLayer,  
10    QgsPointXY,  
11    QgsGeometry,  
12    QgsProject,  
13    QgsExpressionContextUtils  
14 )
```

QGIS has some support for parsing of SQL-like expressions. Only a small subset of SQL syntax is supported. The expressions can be evaluated either as boolean predicates (returning `True` or `False`) or as functions (returning a scalar value). See `vector_expressions` in the User Manual for a complete list of available functions.

Three basic types are supported:

- number — both whole numbers and decimal numbers, e.g. `123`, `3.14`
- string — they have to be enclosed in single quotes: `'hello world'`
- column reference — when evaluating, the reference is substituted with the actual value of the field. The names are not escaped.

The following operations are available:

- arithmetic operators: `+`, `-`, `*`, `/`, `^`
- parentheses: for enforcing the operator precedence: `(1 + 1) * 3`
- unary plus and minus: `-12`, `+5`
- mathematical functions: `sqrt`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`
- conversion functions: `to_int`, `to_real`, `to_string`, `to_date`
- geometry functions: `$area`, `$length`
- geometry handling functions: `$x`, `$y`, `$geometry`, `num_geometries`, `centroid`

And the following predicates are supported:

- comparison: `=`, `!=`, `>`, `>=`, `<`, `<=`

- pattern matching: LIKE (using % and _), ~ (regular expressions)
- logical predicates: AND, OR, NOT
- NULL value checking: IS NULL, IS NOT NULL

Examples of predicates:

- `1 + 2 = 3`
- `sin(angle) > 0`
- `'Hello' LIKE 'He%'`
- `(x > 10 AND y > 10) OR z = 0`

Examples of scalar expressions:

- `2 ^ 10`
- `sqrt(val)`
- `$length + 1`

11.1 Parsing Expressions

The following example shows how to check if a given expression can be parsed correctly:

```
1 exp = QgsExpression('1 + 1 = 2')
2 assert(not exp.hasParserError())
3
4 exp = QgsExpression('1 + 1 = ')
5 assert(exp.hasParserError())
6
7 assert(exp.parserErrorString() == '\nsyntax error, unexpected end of file')
```

11.2 Evaluating Expressions

Expressions can be used in different contexts, for example to filter features or to compute new field values. In any case, the expression has to be evaluated. That means that its value is computed by performing the specified computational steps, which can range from simple arithmetic to aggregate expressions.

11.2.1 Basic Expressions

This basic expression evaluates a simple arithmetic operation:

```
exp = QgsExpression('2 * 3')
print(exp)
print(exp.evaluate())
```

```
<QgsExpression: '2 * 3'>
6
```

Expression can also be used for comparison, evaluating to 1 (True) or 0 (False)

```
exp = QgsExpression('1 + 1 = 2')
exp.evaluate()
# 1
```

11.2.2 Expressions with features

To evaluate an expression against a feature, a `QgsExpressionContext` object has to be created and passed to the evaluate function in order to allow the expression to access the feature's field values.

The following example shows how to create a feature with a field called "Column" and how to add this feature to the expression context.

```

1 fields = QgsFields()
2 field = QgsField('Column')
3 fields.append(field)
4 feature = QgsFeature()
5 feature.setFields(fields)
6 feature.setAttribute(0, 99)
7
8 exp = QgsExpression('"Column"')
9 context = QgsExpressionContext()
10 context.setFeature(feature)
11 exp.evaluate(context)
12 # 99

```

The following is a more complete example of how to use expressions in the context of a vector layer, in order to compute new field values:

```

1 from qgis.PyQt.QtCore import QMetaType
2
3 # create a vector layer
4 vl = QgsVectorLayer("Point", "Companies", "memory")
5 pr = vl.dataProvider()
6 pr.addAttributes([QgsField("Name", QMetaType.Type.QString),
7                    QgsField("Employees", QMetaType.Type.Int),
8                    QgsField("Revenue", QMetaType.Type.Double),
9                    QgsField("Rev. per employee", QMetaType.Type.Double),
10                   QgsField("Sum", QMetaType.Type.Double),
11                   QgsField("Fun", QMetaType.Type.Double)])
12 vl.updateFields()
13
14 # add data to the first three fields
15 my_data = [
16     {'x': 0, 'y': 0, 'name': 'ABC', 'emp': 10, 'rev': 100.1},
17     {'x': 1, 'y': 1, 'name': 'DEF', 'emp': 2, 'rev': 50.5},
18     {'x': 5, 'y': 5, 'name': 'GHI', 'emp': 100, 'rev': 725.9}]
19
20 for rec in my_data:
21     f = QgsFeature()
22     pt = QgsPointXY(rec['x'], rec['y'])
23     f.setGeometry(QgsGeometry.fromPointXY(pt))
24     f.setAttributes([rec['name'], rec['emp'], rec['rev']])
25     pr.addFeature(f)
26
27 vl.updateExtents()
28 QgsProject.instance().addMapLayer(vl)
29
30 # The first expression computes the revenue per employee.
31 # The second one computes the sum of all revenue values in the layer.
32 # The final third expression doesn't really make sense but illustrates
33 # the fact that we can use a wide range of expression functions, such
34 # as area and buffer in our expressions:
35 expression1 = QgsExpression('"Revenue"/"Employees"')
36 expression2 = QgsExpression('sum("Revenue")')
37 expression3 = QgsExpression('area(buffer($geometry, "Employees"))')
38

```

(continues on next page)

(continued from previous page)

```

39 # QgsExpressionContextUtils.globalProjectLayerScopes() is a convenience
40 # function that adds the global, project, and layer scopes all at once.
41 # Alternatively, those scopes can also be added manually. In any case,
42 # it is important to always go from "most generic" to "most specific"
43 # scope, i.e. from global to project to layer
44 context = QgsExpressionContext()
45 context.appendScopes(QgsExpressionContextUtils.globalProjectLayerScopes(vl))
46
47 with edit(vl):
48     for f in vl.getFeatures():
49         context.setFeature(f)
50         f['Rev. per employee'] = expression1.evaluate(context)
51         f['Sum'] = expression2.evaluate(context)
52         f['Fun'] = expression3.evaluate(context)
53         vl.updateFeature(f)
54
55 print(f['Sum'])

```

876.5

11.2.3 Filtering a layer with expressions

The following example can be used to filter a layer and return any feature that matches a predicate.

```

1 layer = QgsVectorLayer("Point?field=Test:integer",
2                        "addfeat", "memory")
3
4 layer.startEditing()
5
6 for i in range(10):
7     feature = QgsFeature()
8     feature.setAttributes([i])
9     assert(layer.addFeature(feature))
10 layer.commitChanges()
11
12 expression = 'Test >= 3'
13 request = QgsFeatureRequest().setFilterExpression(expression)
14
15 matches = 0
16 for f in layer.getFeatures(request):
17     matches += 1
18
19 print(matches)

```

7

11.3 Handling expression errors

Expression-related errors can occur during expression parsing or evaluation:

```

1 exp = QgsExpression("1 + 1 = 2")
2 if exp.hasParserError():
3     raise Exception(exp.parserErrorString())
4
5 value = exp.evaluate()

```

(continues on next page)

(continued from previous page)

```
6 if exp.hasEvalError():  
7     raise ValueError(exp.evalErrorString())
```


READING AND STORING SETTINGS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsProject,  
3     QgsSettings,  
4     QgsVectorLayer  
5 )
```

Many times it is useful for a plugin to save some variables so that the user does not have to enter or select them again next time the plugin is run.

These variables can be saved and retrieved with help of Qt and QGIS API. For each variable, you should pick a key that will be used to access the variable — for user's favourite color you could use key "favourite_color" or any other meaningful string. It is recommended to give some structure to naming of keys.

We can differentiate between several types of settings:

- **global settings** — they are bound to the user at a particular machine. QGIS itself stores a lot of global settings, for example, main window size or default snapping tolerance. Settings are handled using the `QgsSettings` class, through for example the `setValue()` and `value()` methods.

Here you can see an example of how these methods are used.

```
1 def store():  
2     s = QgsSettings()  
3     s.setValue("myplugin/mytext", "hello world")  
4     s.setValue("myplugin/myint", 10)  
5     s.setValue("myplugin/myreal", 3.14)  
6  
7 def read():  
8     s = QgsSettings()  
9     mytext = s.value("myplugin/mytext", "default text")  
10    myint = s.value("myplugin/myint", 123)  
11    myreal = s.value("myplugin/myreal", 2.71)  
12    nonexistent = s.value("myplugin/nonexistent", None)  
13    print(mytext)  
14    print(myint)  
15    print(myreal)  
16    print(nonexistent)
```

The second parameter of the `value()` method is optional and specifies the default value that is returned if there is no previous value set for the passed setting name.

For a method to pre-configure the default values of the global settings through the `qgis_global_settings.ini` file, see `deploying_organization` for further details.

- **project settings** — vary between different projects and therefore they are connected with a project file. Map canvas background color or destination coordinate reference system (CRS) are examples — white background and WGS84 might be suitable for one project, while yellow background and UTM projection are better for another one.

An example of usage follows.

```

1 proj = QgsProject.instance()
2
3 # store values
4 proj.writeEntry("myplugin", "mytext", "hello world")
5 proj.writeEntry("myplugin", "myint", 10)
6 proj.writeEntryDouble("myplugin", "mydouble", 0.01)
7 proj.writeEntryBool("myplugin", "mybool", True)
8
9 # read values (returns a tuple with the value, and a status boolean
10 # which communicates whether the value retrieved could be converted to
11 # its type, in these cases a string, an integer, a double and a boolean
12 # respectively)
13
14 mytext, type_conversion_ok = proj.readEntry("myplugin",
15                                           "mytext",
16                                           "default text")
17 myint, type_conversion_ok = proj.readNumEntry("myplugin",
18                                              "myint",
19                                              123)
20 mydouble, type_conversion_ok = proj.readDoubleEntry("myplugin",
21                                                    "mydouble",
22                                                    123)
23 mybool, type_conversion_ok = proj.readBoolEntry("myplugin",
24                                                "mybool",
25                                                123)

```

As you can see, the `writeEntry()` method is used for many data types (integer, string, list), but several methods exist for reading the setting value back, and the corresponding one has to be selected for each data type.

- **map layer settings** — these settings are related to a particular instance of a map layer with a project. They are *not* connected with underlying data source of a layer, so if you create two map layer instances of one shapefile, they will not share the settings. The settings are stored inside the project file, so if the user opens the project again, the layer-related settings will be there again. The value for a given setting is retrieved using the `customProperty()` method, and can be set using the `setCustomProperty()` one.

```

1 vlayer = QgsVectorLayer()
2 # save a value
3 vlayer.setCustomProperty("mytext", "hello world")
4
5 # read the value again (returning "default text" if not found)
6 mytext = vlayer.customProperty("mytext", "default text")

```

COMMUNICATING WITH THE USER

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsMessageLog,  
3     QgsGeometry,  
4 )  
5  
6 from qgis.gui import (  
7     QgsMessageBar,  
8 )  
9  
10 from qgis.PyQt.QtWidgets import (  
11     QSizePolicy,  
12     QPushButton,  
13     QDialog,  
14     QGridLayout,  
15     QDialogButtonBox,  
16 )
```

This section shows some methods and elements that should be used to communicate with the user, in order to keep consistency in the User Interface.

13.1 Showing messages. The QgsMessageBar class

Using message boxes can be a bad idea from a user experience point of view. For showing a small info line or a warning/error messages, the QGIS message bar is usually a better option.

Using the reference to the QGIS interface object, you can show a message in the message bar with the following code

```
from qgis.core import Qgis  
iface.messageBar().pushMessage("Error", "I'm sorry Dave, I'm afraid I can't do that  
↔", level=Qgis.Critical)
```

```
Messages(2): Error : I'm sorry Dave, I'm afraid I can't do that
```

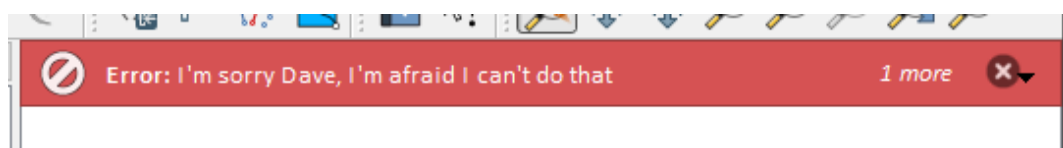


Fig. 13.1: QGIS Message bar

You can set a duration to show it for a limited time

```
iface.messageBar().pushMessage("Oops", "The plugin is not working as it should",
    level=Qgis.Critical, duration=3)
```

```
Messages(2): Oops : The plugin is not working as it should
```

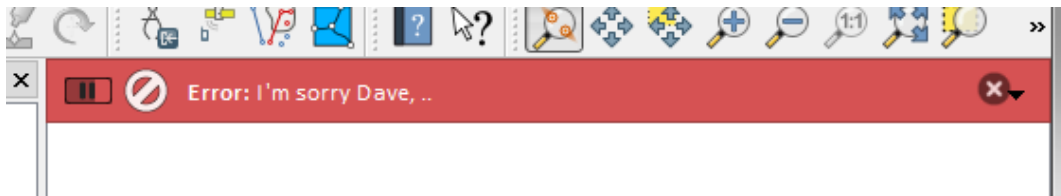


Fig. 13.2: QGIS Message bar with timer

The examples above show an error bar, but the `level` parameter can be used to creating warning messages or info messages, using the `Qgis.MessageLevel` enumeration. You can use up to 4 different levels:

0. Info
1. Warning
2. Critical
3. Success

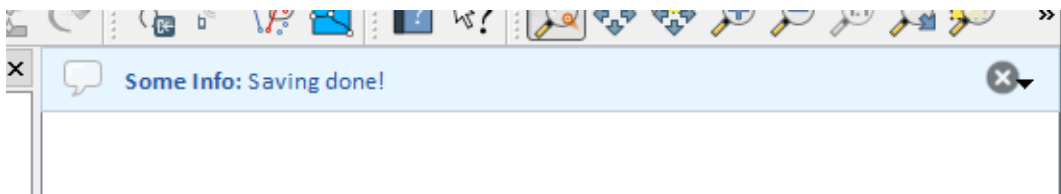


Fig. 13.3: QGIS Message bar (info)

Widgets can be added to the message bar, like for instance a button to show more info

```
1 def showError():
2     pass
3
4 widget = iface.messageBar().createMessage("Missing Layers", "Show Me")
5 button = QPushButton(widget)
6 button.setText("Show Me")
7 button.pressed.connect(showError)
8 widget.layout().addWidget(button)
9 iface.messageBar().pushWidget(widget, Qgis.Warning)
```

```
Messages(1): Missing Layers : Show Me
```

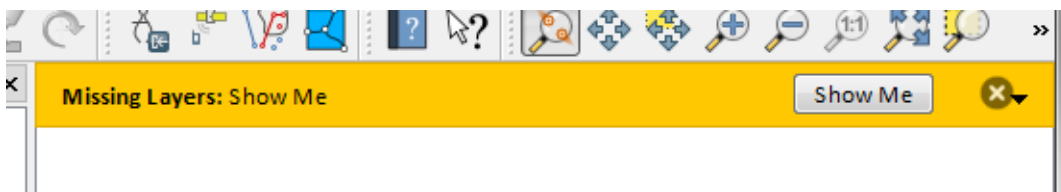


Fig. 13.4: QGIS Message bar with a button

You can even use a message bar in your own dialog so you don't have to show a message box, or if it doesn't make sense to show it in the main QGIS window

```

1 class MyDialog(QDialog):
2     def __init__(self):
3         QDialog.__init__(self)
4         self.bar = QgsMessageBar()
5         self.bar.setSizePolicy( QSizePolicy.Minimum, QSizePolicy.Fixed )
6         self.setLayout(QGridLayout())
7         self.layout().setContentsMargins(0, 0, 0, 0)
8         self.buttonbox = QDialogButtonBox(QDialogButtonBox.Ok)
9         self.buttonbox.accepted.connect(self.run)
10        self.layout().addWidget(self.buttonbox, 0, 0, 2, 1)
11        self.layout().addWidget(self.bar, 0, 0, 1, 1)
12
13    def run(self):
14        self.bar.pushMessage("Hello", "World", level=Qgis.Info)
15
16myDlg = MyDialog()
17myDlg.show()

```

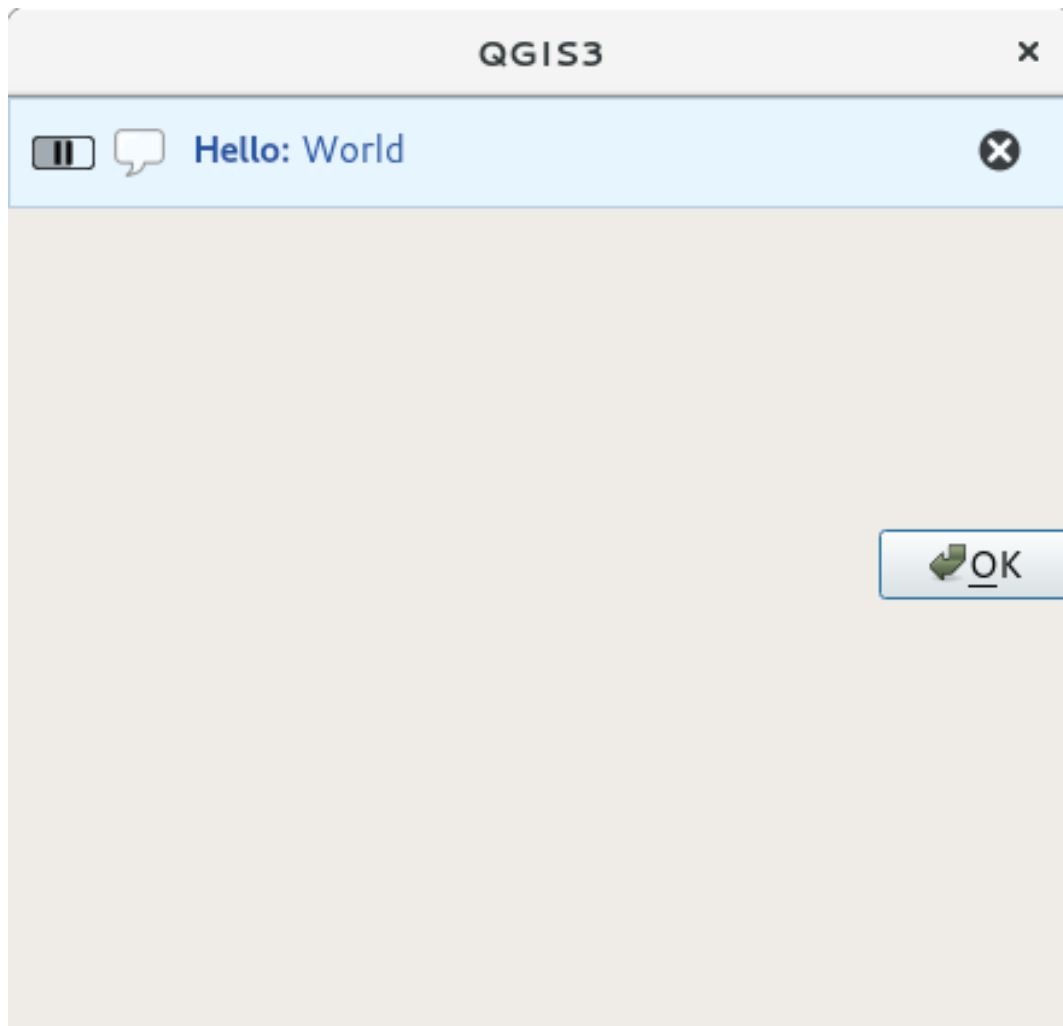


Fig. 13.5: QGIS Message bar in custom dialog

13.2 Showing progress

Progress bars can also be put in the QGIS message bar, since, as we have seen, it accepts widgets. Here is an example that you can try in the console.

```

1 import time
2 from qgis.PyQt.QtWidgets import QProgressBar
3 from qgis.PyQt.QtCore import *
4 progressMessageBar = iface.messageBar().createMessage("Doing something boring...")
5 progress = QProgressBar()
6 progress.setMaximum(10)
7 progress.setAlignment(Qt.AlignLeft|Qt.AlignVCenter)
8 progressMessageBar.layout().addWidget(progress)
9 iface.messageBar().pushWidget(progressMessageBar, Qgis.Info)
10
11 for i in range(10):
12     time.sleep(1)
13     progress.setValue(i + 1)
14
15 iface.messageBar().clearWidgets()
```

Messages(0): Doing something boring...

Also, you can use the built-in status bar to report progress, as in the next example:

```

1 vlayer = iface.activeLayer()
2
3 count = vlayer.featureCount()
4 features = vlayer.getFeatures()
5
6 for i, feature in enumerate(features):
7     # do something time-consuming here
8     print('.') # printing should give enough time to present the progress
9
10    percent = i / float(count) * 100
11    # iface.mainWindow().statusBar().showMessage("Processed {} {}".format(int(percent)))
12    iface.statusBarIface().showMessage("Processed {} {}".format(int(percent)))
13
14 iface.statusBarIface().clearMessage()
```

13.3 Logging

There are three different types of logging available in QGIS to log and save all the information about the execution of your code. Each has its specific output location. Please consider to use the correct way of logging for your purpose:

- `QgsMessageLog` is for messages to communicate issues to the user. The output of the `QgsMessageLog` is shown in the Log Messages Panel.
- The python built in **logging** module is for debugging on the level of the QGIS Python API (PyQGIS). It is recommended for Python script developers that need to debug their python code, e.g. feature ids or geometries
- `QgsLogger` is for messages for *QGIS internal* debugging / developers (i.e. you suspect something is triggered by some broken code). Messages are only visible with developer versions of QGIS.

Examples for the different logging types are shown in the following sections below.

Warning

Use of the Python `print` statement is unsafe to do in any code which may be multithreaded and **extremely slows down the algorithm**. This includes **expression functions, renderers, symbol layers** and **Processing algorithms** (amongst others). In these cases you should always use the python **logging** module or thread safe classes (`QgsLogger` or `QgsMessageLog`) instead.

13.3.1 QgsMessageLog

```
# You can optionally pass a 'tag' and a 'level' parameters
QgsMessageLog.logMessage("Your plugin code has been executed correctly", 'MyPlugin
↪', level=Qgis.Info)
QgsMessageLog.logMessage("Your plugin code might have some problems", level=Qgis.
↪Warning)
QgsMessageLog.logMessage("Your plugin code has crashed!", level=Qgis.Critical)
```

```
MyPlugin(0): Your plugin code has been executed correctly
(1): Your plugin code might have some problems
(2): Your plugin code has crashed!
```

Note

You can see the output of the `QgsMessageLog` in the `log_message_panel`

13.3.2 The python built in logging module

```
1 import logging
2 formatter = '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
3 logfilename=r'c:\temp\example.log'
4 logging.basicConfig(filename=logfilename, level=logging.DEBUG, format=formatter)
5 logging.info("This logging info text goes into the file")
6 logging.debug("This logging debug text goes into the file as well")
```

The `basicConfig` method configures the basic setup of the logging. In the above code the filename, logging level and the format are defined. The filename refers to where to write the logfile to, the logging level defines what levels to output and the format defines the format in which each message is output.

```
2020-10-08 13:14:42,998 - root - INFO - This logging text goes into the file
2020-10-08 13:14:42,998 - root - DEBUG - This logging debug text goes into the
↪file as well
```

If you want to erase the log file every time you execute your script you can do something like:

```
if os.path.isfile(logfilename):
    with open(logfilename, 'w') as file:
        pass
```

Further resources on how to use the python logging facility are available at:

- <https://docs.python.org/3/library/logging.html>
- <https://docs.python.org/3/howto/logging.html>
- <https://docs.python.org/3/howto/logging-cookbook.html>

 Warning

Please note that without logging to a file by setting a filename the logging may be multithreaded which heavily slows down the output.

AUTHENTICATION INFRASTRUCTURE

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (
2     QgsApplication,
3     QgsRasterLayer,
4     QgsAuthMethodConfig,
5     QgsDataSourceUri,
6     QgsPkiBundle,
7     QgsMessageLog,
8 )
9
10 from qgis.gui import (
11     QgsAuthAuthoritiesEditor,
12     QgsAuthConfigEditor,
13     QgsAuthConfigSelect,
14     QgsAuthSettingsWidget,
15 )
16
17 from qgis.PyQt.QtWidgets import (
18     QWidget,
19     QTabWidget,
20 )
21
22 from qgis.PyQt.QtNetwork import QSslCertificate
```

14.1 Introduction

User reference of the Authentication infrastructure can be read in the User Manual in the `authentication_overview` paragraph.

This chapter describes the best practices to use the Authentication system from a developer perspective.

The authentication system is widely used in QGIS Desktop by data providers whenever credentials are required to access a particular resource, for example when a layer establishes a connection to a Postgres database.

There are also a few widgets in the QGIS gui library that plugin developers can use to easily integrate the authentication infrastructure into their code:

- `QgsAuthConfigEditor`
- `QgsAuthConfigSelect`
- `QgsAuthSettingsWidget`

A good code reference can be read from the authentication infrastructure [tests code](#).

Warning

Due to the security constraints that were taken into account during the authentication infrastructure design, only a selected subset of the internal methods are exposed to Python.

14.2 Glossary

Here are some definition of the most common objects treated in this chapter.

Master Password

Password to allow access and decrypt credential stored in the QGIS Authentication DB

Authentication Database

A *Master Password* crypted sqlite db `qgis-auth.db` where *Authentication Configuration* are stored. e.g user/password, personal certificates and keys, Certificate Authorities

Authentication DB

Authentication Database

Authentication Configuration

A set of authentication data depending on *Authentication Method*. e.g Basic authentication method stores the couple of user/password.

Authentication Config

Authentication Configuration

Authentication Method

A specific method used to get authenticated. Each method has its own protocol used to gain the authenticated level. Each method is implemented as shared library loaded dynamically during QGIS authentication infrastructure init.

14.3 QgsAuthManager the entry point

The `QgsAuthManager` singleton is the entry point to use the credentials stored in the QGIS encrypted *Authentication DB*, i.e. the `qgis-auth.db` file under the active user profile folder.

This class takes care of the user interaction: by asking to set a master password or by transparently using it to access encrypted stored information.

14.3.1 Init the manager and set the master password

The following snippet gives an example to set master password to open the access to the authentication settings. Code comments are important to understand the snippet.

```

1 authMgr = QgsApplication.authManager()
2
3 # check if QgsAuthManager has already been initialized... a side effect
4 # of the QgsAuthManager.init() is that AuthDbPath is set.
5 # QgsAuthManager.init() is executed during QGIS application init and hence
6 # you do not normally need to call it directly.
7 if authMgr.authenticationDatabasePath():
8     # already initialized => we are inside a QGIS app.
9     if authMgr.masterPasswordIsSet():
10         msg = 'Authentication master password not recognized'
11         assert authMgr.masterPasswordSame("your master password"), msg
12     else:

```

(continues on next page)

(continued from previous page)

```

13     msg = 'Master password could not be set'
14     # The verify parameter checks if the hash of the password was
15     # already saved in the authentication db
16     assert authMgr.setMasterPassword("your master password",
17                                     verify=True), msg
18 else:
19     # outside qgis, e.g. in a testing environment => setup env var before
20     # db init
21     os.environ['QGIS_AUTH_DB_DIR_PATH'] = "/path/where/located/qgis-auth.db"
22     msg = 'Master password could not be set'
23     assert authMgr.setMasterPassword("your master password", True), msg
24     authMgr.init("/path/where/located/qgis-auth.db")

```

14.3.2 Populate authdb with a new Authentication Configuration entry

Any stored credential is a *Authentication Configuration* instance of the `QgsAuthMethodConfig` class accessed using a unique string like the following one:

```
authcfg = 'fmls770'
```

that string is generated automatically when creating an entry using the QGIS API or GUI, but it might be useful to manually set it to a known value in case the configuration must be shared (with different credentials) between multiple users within an organization.

`QgsAuthMethodConfig` is the base class for any *Authentication Method*. Any Authentication Method sets a configuration hash map where authentication information will be stored. Hereafter a useful snippet to store PKI-path credentials for a hypothetical alice user:

```

1 authMgr = QgsApplication.authManager()
2 # set alice PKI data
3 config = QgsAuthMethodConfig()
4 config.setName("alice")
5 config.setMethod("PKI-Paths")
6 config.setUri("https://example.com")
7 config.setConfig("certpath", "path/to/alice-cert.pem" )
8 config.setConfig("keypath", "path/to/alice-key.pem" )
9 # check if method parameters are correctly set
10 assert config.isValid()
11
12 # register alice data in authdb returning the ``authcfg`` of the stored
13 # configuration
14 authMgr.storeAuthenticationConfig(config)
15 newAuthCfgId = config.id()
16 assert newAuthCfgId

```

Available Authentication methods

Authentication Method libraries are loaded dynamically during authentication manager init. Available authentication methods are:

1. Basic User and password authentication
2. EsriToken ESRI token based authentication
3. Identity-Cert Identity certificate authentication
4. OAuth2 OAuth2 authentication
5. PKI-Paths PKI paths authentication
6. PKI-PKCS#12 PKI PKCS#12 authentication

Populate Authorities

```

1 authMgr = QgsApplication.authManager()
2 # add authorities
3 cacerts = QSslCertificate.fromPath( "/path/to/ca_chains.pem" )
4 assert cacerts is not None
5 # store CA
6 authMgr.storeCertAuthorities(cacerts)
7 # and rebuild CA caches
8 authMgr.rebuildCaCertsCache()
9 authMgr.rebuildTrustedCaCertsCache()

```

Manage PKI bundles with QgsPkiBundle

A convenience class to pack PKI bundles composed on SslCert, SslKey and CA chain is the `QgsPkiBundle` class. Hereafter a snippet to get password protected:

```

1 # add alice cert in case of key with pwd
2 caBundlesList = [] # List of CA bundles
3 bundle = QgsPkiBundle.fromPemPaths( "/path/to/alice-cert.pem",
4                                     "/path/to/alice-key_w-pass.pem",
5                                     "unlock_pwd",
6                                     caBundlesList )
7 assert bundle is not None
8 # You can check bundle validity by calling:
9 # bundle.isValid()

```

Refer to `QgsPkiBundle` class documentation to extract cert/key/CAs from the bundle.

14.3.3 Remove an entry from authdb

We can remove an entry from *Authentication Database* using its `authcfg` identifier with the following snippet:

```

authMgr = QgsApplication.authManager()
authMgr.removeAuthenticationConfig( "authCfg_Id_to_remove" )

```

14.3.4 Leave authcfg expansion to QgsAuthManager

The best way to use an *Authentication Config* stored in the *Authentication DB* is referring it with the unique identifier `authcfg`. Expanding, means convert it from an identifier to a complete set of credentials. The best practice to use stored *Authentication Configs*, is to leave it managed automatically by the Authentication manager. The common use of a stored configuration is to connect to an authentication enabled service like a WMS or WFS or to a DB connection.

Note

Take into account that not all QGIS data providers are integrated with the Authentication infrastructure. Each authentication method, derived from the base class `QgsAuthMethod` and support a different set of Providers. For example the `certIdentity()` method supports the following list of providers:

```

authM = QgsApplication.authManager()
print(authM.authMethod("Identity-Cert").supportedDataProviders())

```

Sample output:

```
['ows', 'wfs', 'wcs', 'wms', 'postgres']
```

For example, to access a WMS service using stored credentials identified with `authcfg = 'fm1s770'`, we just have to use the `authcfg` in the data source URL like in the following snippet:

```
1 authCfg = 'fm1s770'
2 quri = QgsDataSourceUri()
3 quri.setParam("layers", 'usa:states')
4 quri.setParam("styles", '')
5 quri.setParam("format", 'image/png')
6 quri.setParam("crs", 'EPSG:4326')
7 quri.setParam("dpiMode", '7')
8 quri.setParam("featureCount", '10')
9 quri.setParam("authcfg", authCfg) # <---- here my authCfg url parameter
10 quri.setParam("contextualWMSLegend", '0')
11 quri.setParam("url", 'https://my_auth_enabled_server_ip/wms')
12 rlayer = QgsRasterLayer(str(quri.encodedUri(), "utf-8"), 'states', 'wms')
```

In the upper case, the wms provider will take care to expand `authcfg` URI parameter with credential just before setting the HTTP connection.

Warning

The developer would have to leave `authcfg` expansion to the `QgsAuthManager`, in this way he will be sure that expansion is not done too early.

Usually an URI string, built using the `QgsDataSourceURI` class, is used to set a data source in the following way:

```
authCfg = 'fm1s770'
quri = QgsDataSourceUri("my WMS uri here")
quri.setParam("authcfg", authCfg)
rlayer = QgsRasterLayer( quri.uri(False), 'states', 'wms')
```

Note

The `False` parameter is important to avoid URI complete expansion of the `authcfg` id present in the URI.

PKI examples with other data providers

Other example can be read directly in the QGIS tests upstream as in `test_authmanager_pki_ows` or `test_authmanager_pki_postgres`.

14.4 Adapt plugins to use Authentication infrastructure

Many third party plugins are using `httplib2` or other Python networking libraries to manage HTTP connections instead of integrating with `QgsNetworkAccessManager` and its related Authentication Infrastructure integration.

To facilitate this integration a helper Python function has been created called `NetworkAccessManager`. Its code can be found [here](#).

This helper class can be used as in the following snippet:

```
1 http = NetworkAccessManager(authid="my_authCfg", exception_class=My_
2   ↪FailedRequestError)
3 try:
4     response, content = http.request( "my_rest_url" )
5 except My_FailedRequestError, e:
```

(continues on next page)

(continued from previous page)

```

5 # Handle exception
6 pass

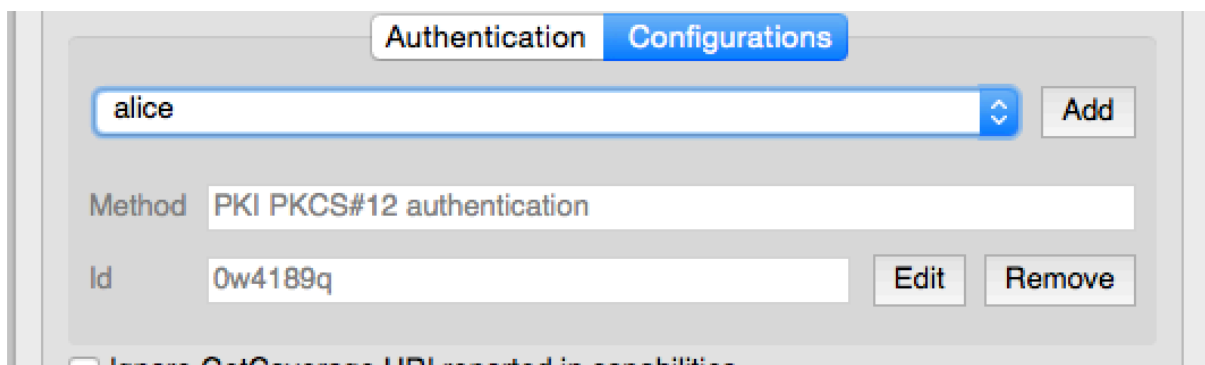
```

14.5 Authentication GUIs

In this paragraph are listed the available GUIs useful to integrate authentication infrastructure in custom interfaces.

14.5.1 GUI to select credentials

If it's necessary to select a *Authentication Configuration* from the set stored in the *Authentication DB* it is available in the GUI class `QgsAuthConfigSelect`.



and can be used as in the following snippet:

```

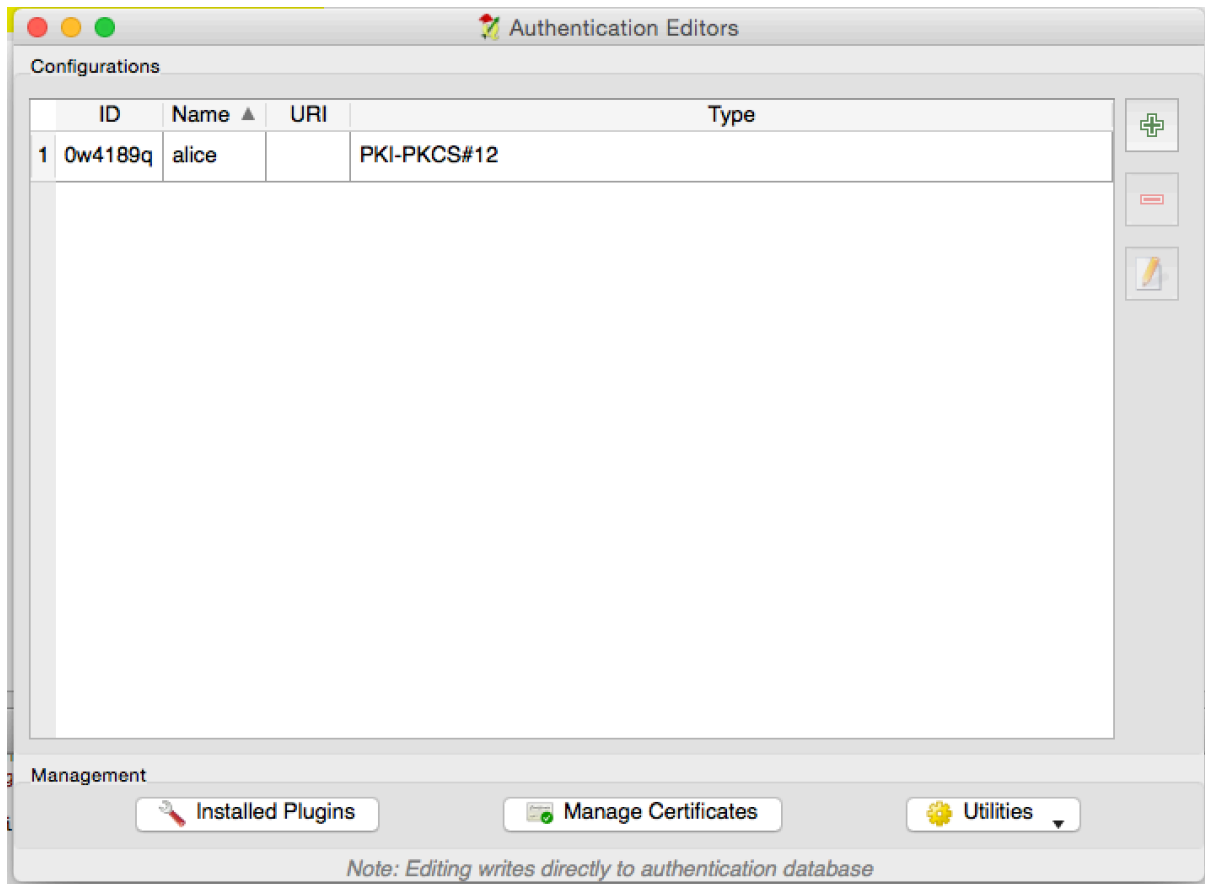
1 # create the instance of the QgsAuthConfigSelect GUI hierarchically linked to
2 # the widget referred with `parent`
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthConfigSelect( parent, "postgres" )
5 # add the above created gui in a new tab of the interface where the
6 # GUI has to be integrated
7 tabGui = QTabWidget()
8 tabGui.insertTab( 1, gui, "Configurations" )

```

The above example is taken from the QGIS source code. The second parameter of the GUI constructor refers to data provider type. The parameter is used to restrict the compatible *Authentication Methods* with the specified provider.

14.5.2 Authentication Editor GUI

The complete GUI used to manage credentials, authorities and to access to Authentication utilities is managed by the `QgsAuthEditorWidgets` class.



and can be used as in the following snippet:

```

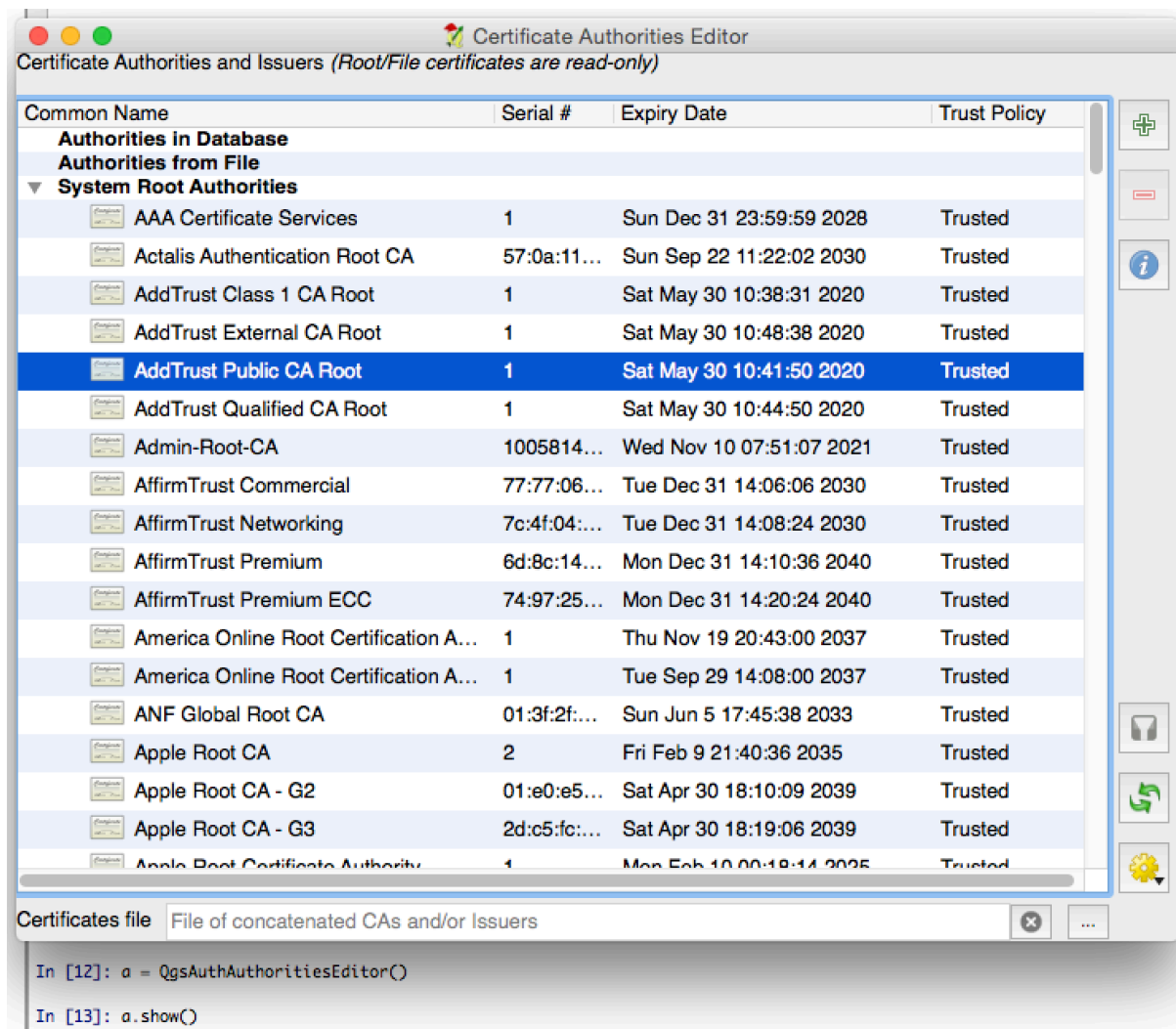
1 # create the instance of the QgsAuthEditorWidgets GUI hierarchically linked to
2 # the widget referred with `parent`
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthConfigSelect( parent )
5 gui.show()

```

An integrated example can be found in the related [test](#).

14.5.3 Authorities Editor GUI

A GUI used to manage only authorities is managed by the `QgsAuthAuthoritiesEditor` class.



and can be used as in the following snippet:

```
1 # create the instance of the QgsAuthAuthoritiesEditor GUI hierarchically
2 # linked to the widget referred with 'parent'
3 parent = QWidget() # Your GUI parent widget
4 gui = QgsAuthAuthoritiesEditor( parent )
5 gui.show()
```

TASKS - DOING HEAVY WORK IN THE BACKGROUND

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     Qgis,  
3     QgsApplication,  
4     QgsMessageLog,  
5     QgsProcessingAlgRunnerTask,  
6     QgsProcessingContext,  
7     QgsProcessingFeedback,  
8     QgsProject,  
9     QgsTask,  
10    QgsTaskManager,  
11 )
```

15.1 Introduction

Background processing using threads is a way to maintain a responsive user interface when heavy processing is going on. Tasks can be used to achieve threading in QGIS.

A task (`QgsTask`) is a container for the code to be performed in the background, and the task manager (`QgsTaskManager`) is used to control the running of the tasks. These classes simplify background processing in QGIS by providing mechanisms for signaling, progress reporting and access to the status for background processes. Tasks can be grouped using subtasks.

The global task manager (found with `QgsApplication.taskManager()`) is normally used. This means that your tasks may not be the only tasks that are controlled by the task manager.

There are several ways to create a QGIS task:

- Create your own task by extending `QgsTask`

```
class SpecialisedTask(QgsTask):  
    pass
```

- Create a task from a function

```
1 def heavyFunction():  
2     # Some CPU intensive processing ...  
3     pass  
4  
5 def workdone():  
6     # ... do something useful with the results  
7     pass
```

(continues on next page)

(continued from previous page)

```

8
9 task = QgsTask.fromFunction('heavy function', heavyFunction,
10                             on_finished=workdone)

```

- Create a task from a processing algorithm

```

1 params = dict()
2 context = QgsProcessingContext()
3 context.setProject(QgsProject.instance())
4 feedback = QgsProcessingFeedback()
5
6 buffer_alg = QgsApplication.instance().processingRegistry().algorithmById(
7     ↳ 'native:buffer')
8 task = QgsProcessingAlgRunnerTask(buffer_alg, params, context,
9                                     feedback)

```

Warning

Any background task (regardless of how it is created) must NEVER use any QObject that lives on the main thread, such as accessing QgsVectorLayer, QgsProject or perform any GUI based operations like creating new widgets or interacting with existing widgets. Qt widgets must only be accessed or modified from the main thread. Data that is used in a task must be copied before the task is started. Attempting to use them from background threads will result in crashes.

Moreover always make sure that `context` and `feedback` live for at least as long as the tasks that use them. QGIS will crash if, upon completion of a task, *QgsTaskManager* fails to access the *context* and *feedback* against which the task was scheduled.

Note

It is a common pattern to call `setProject()` shortly after calling `QgsProcessingContext`. This allows the task as well as its callback function to use most of the project-wide settings. This is especially valuable when working with spatial layers in the callback function.

Dependencies between tasks can be described using the `addSubTask()` function of `QgsTask`. When a dependency is stated, the task manager will automatically determine how these dependencies will be executed. Wherever possible dependencies will be executed in parallel in order to satisfy them as quickly as possible. If a task on which another task depends is canceled, the dependent task will also be canceled. Circular dependencies can make deadlocks possible, so be careful.

If a task depends on a layer being available, this can be stated using the `setDependentLayers()` function of `QgsTask`. If a layer on which a task depends is not available, the task will be canceled.

Once the task has been created it can be scheduled for running using the `addTask()` function of the task manager. Adding a task to the manager automatically transfers ownership of that task to the manager, and the manager will cleanup and delete tasks after they have executed. The scheduling of the tasks is influenced by the task priority, which is set in `addTask()`.

The status of tasks can be monitored using `QgsTask` and `QgsTaskManager` signals and functions.

15.2 Examples

15.2.1 Extending QgsTask

In this example `RandomIntegerSumTask` extends `QgsTask` and will generate 100 random integers between 0 and 500 during a specified period of time. If the random number is 42, the task is aborted and an exception is raised. Several instances of `RandomIntegerSumTask` (with subtasks) are generated and added to the task manager, demonstrating two types of dependencies.

```

1 import random
2 from time import sleep
3
4 from qgis.core import (
5     QgsApplication, QgsTask, QgsMessageLog, Qgis
6 )
7
8 MESSAGE_CATEGORY = 'RandomIntegerSumTask'
9
10 class RandomIntegerSumTask(QgsTask):
11     """This shows how to subclass QgsTask"""
12
13     def __init__(self, description, duration):
14         super().__init__(description, QgsTask.CanCancel)
15         self.duration = duration
16         self.total = 0
17         self.iterations = 0
18         self.exception = None
19
20     def run(self):
21         """Here you implement your heavy lifting.
22         Should periodically test for isCanceled() to gracefully
23         abort.
24         This method MUST return True or False.
25         Raising exceptions will crash QGIS, so we handle them
26         internally and raise them in self.finished
27         """
28         QgsMessageLog.logMessage('Started task "{}'.format(
29             self.description(),
30             MESSAGE_CATEGORY, Qgis.Info)
31
32         wait_time = self.duration / 100
33         for i in range(100):
34             sleep(wait_time)
35             # use setProgress to report progress
36             self.setProgress(i)
37             arandominteger = random.randint(0, 500)
38             self.total += arandominteger
39             self.iterations += 1
40             # check isCanceled() to handle cancellation
41             if self.isCanceled():
42                 return False
43             # simulate exceptions to show how to abort task
44             if arandominteger == 42:
45                 # DO NOT raise Exception('bad value!')
46                 # this would crash QGIS
47                 self.exception = Exception('bad value!')
48                 return False
49             return True
50
51     def finished(self, result):
52         """
53         This function is automatically called when the task has

```

(continues on next page)

```

53     completed (successfully or not).
54     You implement finished() to do whatever follow-up stuff
55     should happen after the task is complete.
56     finished is always called from the main thread, so it's safe
57     to do GUI operations and raise Python exceptions here.
58     result is the return value from self.run.
59     """
60     if result:
61         QgsMessageLog.logMessage(
62             'RandomTask "{name}" completed\n' \
63             'RandomTotal: {total} (with {iterations} '\
64             'iterations)'.format(
65                 name=self.description(),
66                 total=self.total,
67                 iterations=self.iterations),
68             MESSAGE_CATEGORY, Qgs.Success)
69     else:
70         if self.exception is None:
71             QgsMessageLog.logMessage(
72                 'RandomTask "{name}" not successful but without '\
73                 'exception (probably the task was manually '\
74                 'canceled by the user)'.format(
75                     name=self.description(),
76                     MESSAGE_CATEGORY, Qgs.Warning)
77         else:
78             QgsMessageLog.logMessage(
79                 'RandomTask "{name}" Exception: {exception}'.format(
80                     name=self.description(),
81                     exception=self.exception),
82             MESSAGE_CATEGORY, Qgs.Critical)
83         raise self.exception
84
85     def cancel(self):
86         QgsMessageLog.logMessage(
87             'RandomTask "{name}" was canceled'.format(
88                 name=self.description(),
89                 MESSAGE_CATEGORY, Qgs.Info)
90         super().cancel()
91
92
93 longtask = RandomIntegerSumTask('waste cpu long', 20)
94 shorttask = RandomIntegerSumTask('waste cpu short', 10)
95 minitask = RandomIntegerSumTask('waste cpu mini', 5)
96 shortsubtask = RandomIntegerSumTask('waste cpu subtask short', 5)
97 longsubtask = RandomIntegerSumTask('waste cpu subtask long', 10)
98 shortestsubtask = RandomIntegerSumTask('waste cpu subtask shortest', 4)
99
100 # Add a subtask (shortsubtask) to shorttask that must run after
101 # minitask and longtask has finished
102 shorttask.addSubTask(shortsubtask, [minitask, longtask])
103 # Add a subtask (longsubtask) to longtask that must be run
104 # before the parent task
105 longtask.addSubTask(longsubtask, [], QgsTask.ParentDependsOnSubTask)
106 # Add a subtask (shortestsubtask) to longtask
107 longtask.addSubTask(shortestsubtask)
108
109 QgsApplication.taskManager().addTask(longtask)
110 QgsApplication.taskManager().addTask(shorttask)
111 QgsApplication.taskManager().addTask(minitask)

```

```

1 RandomIntegerSumTask(0): Started task "waste cpu subtask shortest"
2 RandomIntegerSumTask(0): Started task "waste cpu short"
3 RandomIntegerSumTask(0): Started task "waste cpu mini"
4 RandomIntegerSumTask(0): Started task "waste cpu subtask long"
5 RandomIntegerSumTask(3): Task "waste cpu subtask shortest" completed
6 RandomTotal: 25452 (with 100 iterations)
7 RandomIntegerSumTask(3): Task "waste cpu mini" completed
8 RandomTotal: 23810 (with 100 iterations)
9 RandomIntegerSumTask(3): Task "waste cpu subtask long" completed
10 RandomTotal: 26308 (with 100 iterations)
11 RandomIntegerSumTask(0): Started task "waste cpu long"
12 RandomIntegerSumTask(3): Task "waste cpu long" completed
13 RandomTotal: 22534 (with 100 iterations)

```

15.2.2 Task from function

Create a task from a function (doSomething in this example). The first parameter of the function will hold the `QgsTask` for the function. An important (named) parameter is `on_finished`, that specifies a function that will be called when the task has completed. The `doSomething` function in this example has an additional named parameter `wait_time`.

```

1 import random
2 from time import sleep
3
4 MESSAGE_CATEGORY = 'TaskFromFunction'
5
6 def doSomething(task, wait_time):
7     """
8     Raises an exception to abort the task.
9     Returns a result if success.
10    The result will be passed, together with the exception (None in
11    the case of success), to the on_finished method.
12    If there is an exception, there will be no result.
13    """
14    QgsMessageLog.logMessage('Started task {}'.format(task.description()),
15                             MESSAGE_CATEGORY, QgsInfo.Info)
16    wait_time = wait_time / 100
17    total = 0
18    iterations = 0
19    for i in range(100):
20        sleep(wait_time)
21        # use task.setProgress to report progress
22        task.setProgress(i)
23        arandominteger = random.randint(0, 500)
24        total += arandominteger
25        iterations += 1
26        # check task.isCanceled() to handle cancellation
27        if task.isCanceled():
28            stopped(task)
29            return None
30        # raise an exception to abort the task
31        if arandominteger == 42:
32            raise Exception('bad value!')
33    return {'total': total, 'iterations': iterations,
34           'task': task.description()}
35
36 def stopped(task):
37    QgsMessageLog.logMessage(
38        'Task "{name}" was canceled'.format(
39            name=task.description()),

```

(continues on next page)

(continued from previous page)

```

40     MESSAGE_CATEGORY, Qgis.Info)
41
42 def completed(exception, result=None):
43     """This is called when doSomething is finished.
44     Exception is not None if doSomething raises an exception.
45     result is the return value of doSomething."""
46     if exception is None:
47         if result is None:
48             QgsMessageLog.logMessage(
49                 'Completed with no exception and no result '\
50                 '(probably manually canceled by the user)',
51                 MESSAGE_CATEGORY, Qgis.Warning)
52         else:
53             QgsMessageLog.logMessage(
54                 'Task {name} completed\n'
55                 'Total: {total} ( with {iterations} '
56                 'iterations)'.format(
57                     name=result['task'],
58                     total=result['total'],
59                     iterations=result['iterations']),
60                 MESSAGE_CATEGORY, Qgis.Info)
61     else:
62         QgsMessageLog.logMessage("Exception: {}".format(exception),
63                                 MESSAGE_CATEGORY, Qgis.Critical)
64     raise exception
65
66 # Create a few tasks
67 task1 = QgsTask.fromFunction('Waste cpu 1', doSomething,
68                             on_finished=completed, wait_time=4)
69 task2 = QgsTask.fromFunction('Waste cpu 2', doSomething,
70                             on_finished=completed, wait_time=3)
71 QgsApplication.taskManager().addTask(task1)
72 QgsApplication.taskManager().addTask(task2)

```

```

1 RandomIntegerSumTask(0): Started task "waste cpu subtask short"
2 RandomTaskFromFunction(0): Started task Waste cpu 1
3 RandomTaskFromFunction(0): Started task Waste cpu 2
4 RandomTaskFromFunction(0): Task Waste cpu 2 completed
5 RandomTotal: 23263 ( with 100 iterations)
6 RandomTaskFromFunction(0): Task Waste cpu 1 completed
7 RandomTotal: 25044 ( with 100 iterations)

```

15.2.3 Task from a processing algorithm

Create a task that uses the algorithm `qgis:randompointsinextent` to generate 50000 random points inside a specified extent. The result is added to the project in a safe way.

```

1 from functools import partial
2 from qgis.core import (QgsTaskManager, QgsMessageLog,
3                       QgsProcessingAlgRunnerTask, QgsApplication,
4                       QgsProcessingContext, QgsProcessingFeedback,
5                       QgsProject)
6
7 MESSAGE_CATEGORY = 'AlgRunnerTask'
8
9 def task_finished(context, successful, results):
10     if not successful:
11         QgsMessageLog.logMessage('Task finished unsuccessfully',
12                                 MESSAGE_CATEGORY, Qgis.Warning)

```

(continues on next page)

(continued from previous page)

```

13     output_layer = context.getMapLayer(results['OUTPUT'])
14     # because getMapLayer doesn't transfer ownership, the layer will
15     # be deleted when context goes out of scope and you'll get a
16     # crash.
17     # takeMapLayer transfers ownership so it's then safe to add it
18     # to the project and give the project ownership.
19     if output_layer and output_layer.isValid():
20         QgsProject.instance().addMapLayer(
21             context.takeResultLayer(output_layer.id()))
22
23 alg = QgsApplication.processingRegistry().algorithmById(
24     'qgis:randompointsinextent')
25
26 # `context` and `feedback` need to
27 # live for as least as long as `task`,
28 # otherwise the program will crash.
29 # Initializing them globally is a sure way
30 # of avoiding this unfortunate situation.
31 context = QgsProcessingContext()
32 feedback = QgsProcessingFeedback()
33 params = {
34     'EXTENT': '0.0,10.0,40,50 [EPSG:4326]',
35     'MIN_DISTANCE': 0.0,
36     'POINTS_NUMBER': 50000,
37     'TARGET_CRS': 'EPSG:4326',
38     'OUTPUT': 'memory:My random points'
39 }
40 task = QgsProcessingAlgRunnerTask(alg, params, context, feedback)
41 task.executed.connect(partial(task_finished, context))
42 QgsApplication.taskManager().addTask(task)

```

See also: <https://www.opengis.ch/2018/06/22/threads-in-pyqgis3/>.

DEVELOPING PYTHON PLUGINS

16.1 Structuring Python Plugins

The main steps for creating a plugin are:

1. *Idea*: Have an idea about what you want to do with your new QGIS plugin.
2. *Setup*: *Create the files for your plugin*. Depending on the plugin type, some are mandatory while others are optional
3. *Develop*: *Write the code* in appropriate files
4. *Document*: *Write the plugin documentation*
5. Optionally: *Translate*: *Translate your plugin* into different languages
6. *Test*: *Reload your plugin* to check if everything is OK
7. *Publish*: Publish your plugin in QGIS repository or make your own repository as an “arsenal” of personal “GIS weapons”.

16.1.1 Getting started

Before starting to write a new plugin, have a look at the [Official Python plugin repository](#). The source code of existing plugins can help you to learn more about programming. You may also find that a similar plugin already exists and you may be able to extend it or at least build on it to develop your own.

Set up plugin file structure

To get started with a new plugin, we need to set up the necessary plugin files.

There are two plugin template resources that can help get you started:

- For educational purposes or whenever a minimalist approach is desired, the [minimal plugin template](#) provides the basic files (skeleton) necessary to create a valid QGIS Python plugin.
- For a more fully feature plugin template, the [Plugin Builder](#) can create templates for multiple different plugin types, including features such as localization (translation) and testing.

A typical plugin directory includes the following files:

- `metadata.txt` - *required* - Contains general info, version, name and some other metadata used by plugins website and plugin infrastructure.
- `__init__.py` - *required* - The starting point of the plugin. It has to have the `classFactory()` method and may have any other initialisation code.

- `mainPlugin.py` - *core code* - The main working code of the plugin. Contains all the information about the actions of the plugin and the main code.
- `form.ui` - *for plugins with custom GUI* - The GUI created by Qt Designer.
- `form.py` - *compiled GUI* - The translation of the `form.ui` described above to Python.
- `resources.qrc` - *optional* - An `.xml` document created by Qt Designer. Contains relative paths to resources used in the GUI forms.
- `resources.py` - *compiled resources, optional* - The translation of the `.qrc` file described above to Python.
- `LICENSE` - *required* if plugin is to be published or updated in the QGIS Plugins Directory, otherwise *optional*. File should be a plain text file with no file extension in the filename.

Warning

If you plan to upload the plugin to the [Official Python plugin repository](#) you must check that your plugin follows some additional rules, required for plugin [Validation](#).

16.1.2 Writing plugin code

The following section shows what content should be added in each of the files introduced above.

metadata.txt

First, the Plugin Manager needs to retrieve some basic information about the plugin such as its name, description etc. This information is stored in `metadata.txt`.

Note

All metadata must be in UTF-8 encoding.

Metadata name	Required	Notes
name	True	a short string containing the name of the plugin
qgisMinimumVersion	True	dotted notation of minimum QGIS version
qgisMaximumVersion	False	dotted notation of maximum QGIS version
description	True	short text which describes the plugin, no HTML allowed
about	True	longer text which describes the plugin in details, no HTML allowed
version	True	short string with the version dotted notation
author	True	author name
email	True	email of the author, only shown on the website to logged in users, but visible in the Plugin Manager after the plugin is installed
changelog	False	string, can be multiline, no HTML allowed
experimental	False	boolean flag, True or False - True if this version is experimental
deprecated	False	boolean flag, True or False, applies to the whole plugin and not just to the uploaded version
tags	False	comma separated list, spaces are allowed inside individual tags
homepage	False	a valid URL pointing to the homepage of your plugin
repository	True	a valid URL for the source code repository
tracker	False	a valid URL for tickets and bug reports
icon	False	a file name or a relative path (relative to the base folder of the plugin's compressed package) of a web friendly image (PNG, JPEG)
category	False	one of Raster, Vector, Database, Mesh and Web
plugin_dependencies	False	PIP-like comma separated list of other plugins to install, use plugin names coming from their metadata's name field
server	False	boolean flag, True or False, determines if the plugin has a server interface
hasProcessingProvider	False	boolean flag, True or False, determines if the plugin provides processing algorithms

By default, plugins are placed in the *Plugins* menu (we will see in the next section how to add a menu entry for your plugin) but they can also be placed into *Raster*, *Vector*, *Database*, *Mesh* and *Web* menus.

A corresponding “category” metadata entry exists to specify that, so the plugin can be classified accordingly. This metadata entry is used as tip for users and tells them where (in which menu) the plugin can be found. Allowed values for “category” are: Vector, Raster, Database or Web. For example, if your plugin will be available from *Raster* menu, add this to `metadata.txt`

```
category=Raster
```

Note

If `qgisMaximumVersion` is empty, it will be automatically set to the major version plus .99 when uploaded to the *Official Python plugin repository*.

An example for this `metadata.txt`

```
; the next section is mandatory

[general]
name=HelloWorld
email=me@example.com
author=Just Me
qgisMinimumVersion=3.0
description=This is an example plugin for greeting the world.
    Multiline is allowed:
```

(continues on next page)

(continued from previous page)

```

lines starting with spaces belong to the same
field, in this case to the "description" field.
HTML formatting is not allowed.
about=This paragraph can contain a detailed description
      of the plugin. Multiline is allowed, HTML is not.
version=version 1.2
tracker=http://bugs.itopen.it
repository=http://www.itopen.it/repo
; end of mandatory metadata

; start of optional metadata
category=Raster
changelog=The changelog lists the plugin versions
          and their changes as in the example below:
          1.0 - First stable release
          0.9 - All features implemented
          0.8 - First testing release

; Tags are in comma separated value format, spaces are allowed within the
; tag name.
; Tags should be in English language. Please also check for existing tags and
; synonyms before creating a new one.
tags=wkt,raster,hello world

; these metadata can be empty, they will eventually become mandatory.
homepage=https://www.itopen.it
icon=icon.png

; experimental flag (applies to the single version)
experimental=True

; deprecated flag (applies to the whole plugin and not only to the uploaded_
↪version)
deprecated=False

; if empty, it will be automatically set to major version + .99
qgisMaximumVersion=3.99

; Since QGIS 3.8, a comma separated list of plugins to be installed
; (or upgraded) can be specified.
; The example below will try to install (or upgrade) "MyOtherPlugin" version 1.12
; and any version of "YetAnotherPlugin".
; Both "MyOtherPlugin" and "YetAnotherPlugin" names come from their own metadata's
; name field
plugin_dependencies=MyOtherPlugin==1.12,YetAnotherPlugin

```

__init__.py

This file is required by Python's import system. Also, QGIS requires that this file contains a `classFactory()` function, which is called when the plugin gets loaded into QGIS. It receives a reference to the instance of `Qgis-Interface` and must return an object of your plugin's class from the `mainplugin.py` — in our case it's called `TestPlugin` (see below). This is how `__init__.py` should look like

```

def classFactory(iface):
    from .mainPlugin import TestPlugin
    return TestPlugin(iface)

# any other initialisation needed

```

mainPlugin.py

This is where the magic happens and this is how magic looks like: (e.g. mainPlugin.py)

```

from qgis.PyQt.QtGui import *
from qgis.PyQt.QtWidgets import *

# initialize Qt resources from file resources.py
from . import resources

class TestPlugin:

    def __init__(self, iface):
        # save reference to the QGIS interface
        self.iface = iface

    def initGui(self):
        # create action that will start plugin configuration
        self.action = QAction(QIcon("testplug:icon.png"),
                               "Test plugin",
                               self.iface.mainWindow())
        self.action.setObjectName("testAction")
        self.action.setWhatsThis("Configuration for test plugin")
        self.action.setStatusTip("This is status tip")
        self.action.triggered.connect(self.run)

        # add toolbar button and menu item
        self.iface.addToolBarIcon(self.action)
        self.iface.addPluginToMenu("&Test plugins", self.action)

        # connect to signal renderComplete which is emitted when canvas
        # rendering is done
        self.iface.mapCanvas().renderComplete.connect(self.renderTest)

    def unload(self):
        # remove the plugin menu item and icon
        self.iface.removePluginMenu("&Test plugins", self.action)
        self.iface.removeToolBarIcon(self.action)

        # disconnect from signal of the canvas
        self.iface.mapCanvas().renderComplete.disconnect(self.renderTest)

    def run(self):
        # create and show a configuration dialog or something similar
        print("TestPlugin: run called!")

    def renderTest(self, painter):
        # use painter for drawing to map canvas
        print("TestPlugin: renderTest called!")

```

The only plugin functions that must exist in the main plugin source file (e.g. mainPlugin.py) are:

- `__init__` which gives access to QGIS interface
- `initGui()` called when the plugin is loaded
- `unload()` called when the plugin is unloaded

In the above example, `addPluginToMenu()` is used. This will add the corresponding menu action to the *Plugins* menu. Alternative methods exist to add the action to a different menu. Here is a list of those methods:

- `addPluginToRasterMenu()`
- `addPluginToVectorMenu()`

- `addPluginToDatabaseMenu()`
- `addPluginToWebMenu()`

All of them have the same syntax as the `addPluginToMenu()` method.

Adding your plugin menu to one of those predefined method is recommended to keep consistency in how plugin entries are organized. However, you can add your custom menu group directly to the menu bar, as the next example demonstrates:

```
def initGui(self):
    self.menu = QMenu(self.iface.mainWindow())
    self.menu.setObjectName("testMenu")
    self.menu.setTitle("MyMenu")

    self.action = QAction(QIcon("testplug:icon.png"),
                           "Test plugin",
                           self.iface.mainWindow())
    self.action.setObjectName("testAction")
    self.action.setWhatsThis("Configuration for test plugin")
    self.action.setStatusTip("This is status tip")
    self.action.triggered.connect(self.run)
    self.menu.addAction(self.action)

    menuBar = self.iface.mainWindow().menuBar()
    menuBar.insertMenu(self.iface.firstRightStandardMenu().menuAction(),
                       self.menu)

def unload(self):
    self.menu.deleteLater()
```

Don't forget to set `QAction` and `QMenu` `objectName` to a name specific to your plugin so that it can be customized.

While help and about actions can also be added to your custom menu, a convenient place to make them available is in the QGIS main *Help ► Plugins* menu. This is done using the `pluginHelpMenu()` method.

```
def initGui(self):

    self.help_action = QAction(
        QIcon("testplug:icon.png"),
        self.tr("Test Plugin..."),
        self.iface.mainWindow()
    )
    # Add the action to the Help menu
    self.iface.pluginHelpMenu().addAction(self.help_action)

    self.help_action.triggered.connect(self.show_help)

    @staticmethod
    def show_help():
        """ Open the online help. """
        QDesktopServices.openUrl(QUrl('https://docs.qgis.org'))

def unload(self):

    self.iface.pluginHelpMenu().removeAction(self.help_action)
    del self.help_action
```

When working on a real plugin it's wise to write the plugin in another (working) directory and create a makefile which will generate UI + resource files and install the plugin into your QGIS installation.

16.1.3 Documenting plugins

The documentation for the plugin can be written as HTML help files. The `qgis.utils` module provides a function, `showPluginHelp()` which will open the help file browser, in the same way as other QGIS help.

The `showPluginHelp()` function looks for help files in the same directory as the calling module. It will look for, in turn, `index-ll_cc.html`, `index-ll.html`, `index-en.html`, `index-en_us.html` and `index.html`, displaying whichever it finds first. Here `ll_cc` is the QGIS locale. This allows multiple translations of the documentation to be included with the plugin.

The `showPluginHelp()` function can also take parameters `packageName`, which identifies a specific plugin for which the help will be displayed, `filename`, which can replace “index” in the names of files being searched, and `section`, which is the name of an html anchor tag in the document on which the browser will be positioned.

16.1.4 Translating plugins

With a few steps you can set up the environment for the plugin localization so that depending on the locale settings of your computer the plugin will be loaded in different languages.

Software requirements

The easiest way to create and manage all the translation files is to install [Qt Linguist](#). In a Debian-based GNU/Linux environment you can install it typing:

```
sudo apt install qttools5-dev-tools
```

Files and directory

When you create the plugin you will find the `i18n` folder within the main plugin directory.

All the translation files have to be within this directory.

.pro file

First you should create a `.pro` file, that is a *project* file that can be managed by **Qt Linguist**.

In this `.pro` file you have to specify all the files and forms you want to translate. This file is used to set up the localization files and variables. A possible project file, matching the structure of our *example plugin*:

```
FORMS = ../form.ui
SOURCES = ../your_plugin.py
TRANSLATIONS = your_plugin_it.ts
```

Your plugin might follow a more complex structure, and it might be distributed across several files. If this is the case, keep in mind that `pylupdate5`, the program we use to read the `.pro` file and update the translatable string, does not expand wild card characters, so you need to place every file explicitly in the `.pro` file. Your project file might then look like something like this:

```
FORMS = ../ui/about.ui ../ui/feedback.ui \
        ../ui/main_dialog.ui
SOURCES = ../your_plugin.py ../computation.py \
        ../utils.py
```

Furthermore, the `your_plugin.py` file is the file that *calls* all the menu and sub-menus of your plugin in the QGIS toolbar and you want to translate them all.

Finally with the `TRANSLATIONS` variable you can specify the translation languages you want.

⚠ Warning

Be sure to name the `ts` file like `your_plugin_ + language + .ts` otherwise the language loading will fail!
Use the 2 letter shortcut for the language (**it** for Italian, **de** for German, etc...)

.ts file

Once you have created the `.pro` you are ready to generate the `.ts` file(s) for the language(s) of your plugin.

Open a terminal, go to `your_plugin/i18n` directory and type:

```
pylupdate5 your_plugin.pro
```

you should see the `your_plugin_language.ts` file(s).

Open the `.ts` file with **Qt Linguist** and start to translate.

.qm file

When you finish to translate your plugin (if some strings are not completed the source language for those strings will be used) you have to create the `.qm` file (the compiled `.ts` file that will be used by QGIS).

Just open a terminal `cd` in `your_plugin/i18n` directory and type:

```
lrelease your_plugin.ts
```

now, in the `i18n` directory you will see the `your_plugin.qm` file(s).

Translate using Makefile

Alternatively you can use the makefile to extract messages from python code and Qt dialogs, if you created your plugin with Plugin Builder. At the beginning of the Makefile there is a `LOCALES` variable:

```
LOCALES = en
```

Add the abbreviation of the language to this variable, for example for Hungarian language:

```
LOCALES = en hu
```

Now you can generate or update the `hu.ts` file (and the `en.ts` too) from the sources by:

```
make transup
```

After this, you have updated `.ts` file for all languages set in the `LOCALES` variable. Use **Qt Linguist** to translate the program messages. Finishing the translation the `.qm` files can be created by the `transcompile`:

```
make transcompile
```

You have to distribute `.ts` files with your plugin.

Load the plugin

In order to see the translation of your plugin, open QGIS, change the language (*Settings ► Options ► General*) and restart QGIS.

You should see your plugin in the correct language.

Warning

If you change something in your plugin (new UIs, new menu, etc..) you have to **generate again** the update version of both `.ts` and `.qm` file, so run again the command of above.

16.1.5 Sharing your plugin

QGIS is hosting hundreds of plugins in the plugin repository. Consider sharing yours! It will extend the possibilities of QGIS and people will be able to learn from your code. All hosted plugins can be found and installed from within QGIS with the Plugin Manager.

Information and requirements are here: plugins.qgis.org.

16.1.6 Tips and Tricks

Plugin Reloader

During development of your plugin you will frequently need to reload it in QGIS for testing. This is very easy using the **Plugin Reloader** plugin. You can find it with the Plugin Manager.

Automate packaging, release and translation with qgis-plugin-ci

`qgis-plugin-ci` provides a command line interface to perform automated packaging and deployment for QGIS plugins on your computer, or using continuous integration like [GitHub workflows](#) or [Gitlab-CI](#) as well as [Transifex](#) for translation.

It allows releasing, translating, publishing or generating an XML plugin repository file via CLI or in CI actions.

Accessing Plugins

You can access all the classes of installed plugins from within QGIS using python, which can be handy for debugging purposes.

```
my_plugin = qgis.utils.plugins['My Plugin']
```

Log Messages

Plugins have their own tab within the `log_message_panel`.

Resource File

Some plugins use resource files, for example `resources.qrc` which define resources for the GUI, such as icons:

```
<RCC>
  <qresource prefix="/plugins/testplug" >
    <file>icon.png</file>
  </qresource>
</RCC>
```

It is good to use a prefix that will not collide with other plugins or any parts of QGIS, otherwise you might get resources you did not want. Now you just need to generate a Python file that will contain the resources. It's done with **pyrcc5** command:

```
pyrcc5 -o resources.py resources.qrc
```

Note

In Windows environments, attempting to run the **pyrcc5** from Command Prompt or Powershell will probably result in the error "Windows cannot access the specified device, path, or file [...]". The easiest solution is probably to use the OSGeo4W Shell but if you are comfortable modifying the PATH environment variable or specifying the path to the executable explicitly you should be able to find it at `<Your QGIS Install Directory>\bin\pyrcc5.exe`.

16.2 Code Snippets

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (
2     QgsProject,
3     QgsApplication,
4     QgsMapLayer,
5 )
6
7 from qgis.gui import (
8     QgsGui,
9     QgsOptionsWidgetFactory,
10    QgsOptionsPageWidget,
11    QgsLayerTreeEmbeddedWidgetProvider,
12    QgsLayerTreeEmbeddedWidgetRegistry,
13 )
14
15 from qgis.PyQt.QtCore import Qt
16 from qgis.PyQt.QtWidgets import (
17     QMessageBox,
18     QAction,
19     QHBoxLayout,
20     QComboBox,
21 )
22 from qgis.PyQt.QtGui import QIcon
```

This section features code snippets to facilitate plugin development.

16.2.1 How to call a method by a key shortcut

In the plug-in add to the `initGui()`

```
self.key_action = QAction("Test Plugin", self.iface.mainWindow())
self.iface.registerMainWindowAction(self.key_action, "Ctrl+I") # action triggered
↳ by Ctrl+I
self.iface.addPluginToMenu("&Test plugins", self.key_action)
self.key_action.triggered.connect(self.key_action_triggered)
```

To `unload()` add

```
self.iface.unregisterMainWindowAction(self.key_action)
```

The method that is called when CTRL+I is pressed

```
def key_action_triggered(self):
    QMessageBox.information(self.iface.mainWindow(), "Ok", "You pressed Ctrl+I")
```

It is also possible to allow users to customize key shortcuts for the provided actions. This is done by adding:

```
1 # in the initGui() function
2 QgsGui.shortcutsManager().registerAction(self.key_action)
3
4 # and in the unload() function
5 QgsGui.shortcutsManager().unregisterAction(self.key_action)
```

16.2.2 How to reuse QGIS icons

Because they are well-known and convey a clear message to the users, you may want sometimes to reuse QGIS icons in your plugin instead of drawing and setting a new one. Use the `getThemeIcon()` method.

For example, to reuse the  `mActionFileOpen.svg` icon available in the QGIS code repository:

```
1 # e.g. somewhere in the initGui
2 self.file_open_action = QAction(
3     QgsApplication.getThemeIcon("/mActionFileOpen.svg"),
4     self.tr("Select a File..."),
5     self.iface.mainWindow()
6 )
7 self.iface.addPluginToMenu("MyPlugin", self.file_open_action)
```

`iconPath()` is another method to call QGIS icons. Find examples of calls to theme icons at [QGIS embedded images - Cheatsheet](#).

16.2.3 Interface for plugin in the options dialog

You can add a custom plugin options tab to *Settings ► Options*. This is preferable over adding a specific main menu entry for your plugin's options, as it keeps all of the QGIS application settings and plugin settings in a single place which is easy for users to discover and navigate.

The following snippet will just add a new blank tab for the plugin's settings, ready for you to populate with all the options and settings specific to your plugin. You can split the following classes into different files. In this example, we are adding two classes into the main `mainPlugin.py` file.

```
1 class MyPluginOptionsFactory(QgsOptionsWidgetFactory):
2
3     def __init__(self):
4         super().__init__()
```

(continues on next page)

(continued from previous page)

```

5
6     def icon(self):
7         return QIcon('icons/my_plugin_icon.svg')
8
9     def createWidget(self, parent):
10        return ConfigOptionsPage(parent)
11
12
13 class ConfigOptionsPage(QgsOptionsPageWidget):
14
15     def __init__(self, parent):
16         super().__init__(parent)
17         layout = QHBoxLayout()
18         layout.setContentsMargins(0, 0, 0, 0)
19         self.setLayout(layout)

```

Finally we are adding the imports and modifying the `__init__` function:

```

1 from qgis.PyQt.QtWidgets import QHBoxLayout
2 from qgis.gui import QgsOptionsWidgetFactory, QgsOptionsPageWidget
3
4
5 class MyPlugin:
6     """QGIS Plugin Implementation."""
7
8     def __init__(self, iface):
9         """Constructor.
10
11         :param iface: An interface instance that will be passed to this class
12                       which provides the hook by which you can manipulate the QGIS
13                       application at run time.
14         :type iface: QgsInterface
15         """
16         # Save reference to the QGIS interface
17         self.iface = iface
18
19
20     def initGui(self):
21         self.options_factory = MyPluginOptionsFactory()
22         self.options_factory.setTitle(self.tr('My Plugin'))
23         iface.registerOptionsWidgetFactory(self.options_factory)
24
25     def unload(self):
26         iface.unregisterOptionsWidgetFactory(self.options_factory)

```

Tip

Add custom tabs to layer properties dialog

You can apply a similar logic to add the plugin custom option to the layer properties dialog using the classes `QgsMapLayerConfigWidgetFactory` and `QgsMapLayerConfigWidget`.

16.2.4 Embed custom widgets for layers in the layer tree

Beside usual layer symbology elements displayed next or below the layer entry in the *Layers* panel, you can add your own widgets, allowing for quick access to some actions that are often used with a layer (setup filtering, selection, style, refreshing a layer with a button widget, create a layer based time slider or just show extra layer information in a Label there, or ...). These so-called **Layer tree embedded widgets** are made available through the layer's properties *Legend* tab for individual layers.

The following code snippet creates a drop-down in the legend which shows you the layer-styles available for the layer, allowing to quickly switch between the different layer styles.

```

1 class LayerStyleComboBox(QComboBox):
2     def __init__(self, layer):
3         QComboBox.__init__(self)
4         self.layer = layer
5         for style_name in layer.styleManager().styles():
6             self.addItem(style_name)
7
8         idx = self.findText(layer.styleManager().currentStyle())
9         if idx != -1:
10            self.setCurrentIndex(idx)
11
12        self.currentIndexChanged.connect(self.on_current_changed)
13
14    def on_current_changed(self, index):
15        self.layer.styleManager().setCurrentStyle(self.itemText(index))
16
17 class LayerStyleWidgetProvider(QgsLayerTreeEmbeddedWidgetProvider):
18     def __init__(self):
19         QgsLayerTreeEmbeddedWidgetProvider.__init__(self)
20
21     def id(self):
22         return "style"
23
24     def name(self):
25         return "Layer style chooser"
26
27     def createWidget(self, layer, widgetIndex):
28         return LayerStyleComboBox(layer)
29
30     def supportsLayer(self, layer):
31         return True # any layer is fine
32
33 provider = LayerStyleWidgetProvider()
34 QgsGui.layerTreeEmbeddedWidgetRegistry().addProvider(provider)

```

Then from a given layer's *Legend* properties tab, drag the Layer style chooser from the *Available widgets* to *Used widgets* to enable the widget in the layer tree. Embedded widgets are ALWAYS displayed at the top of their associated layer node subitems.

If you want to use the widgets from within e.g. a plugin, you can add them like this:

```

1 layer = iface.activeLayer()
2 counter = int(layer.customProperty("embeddedWidgets/count", 0))
3 layer.setCustomProperty("embeddedWidgets/count", counter+1)
4 layer.setCustomProperty("embeddedWidgets/{}/id".format(counter), "style")
5 view = self.iface.layerTreeView()
6 view.layerTreeModel().refreshLayerLegend(view.currentLegendNode())
7 view.currentNode().setExpanded(True)

```

16.3 IDE settings for writing and debugging plugins

Although each programmer has his preferred IDE/Text editor, here are some recommendations for setting up popular IDE's for writing and debugging QGIS Python plugins.

16.3.1 Useful plugins for writing Python plugins

Some plugins are convenient when writing Python plugins. From *Plugins ► Manage and Install plugins...*, install:

- *Plugin reloader*: This will let you reload a plugin and pull new changes without restarting QGIS.
- *First Aid*: This will add a Python console and local debugger to inspect variables when an exception is raised from a plugin.

Warning

Despite our constant efforts, information beyond this line may not be updated for QGIS 3.

16.3.2 A note on configuring your IDE on Linux and Windows

On Linux, all that usually needs to be done is to add the QGIS library locations to the user's `PYTHONPATH` environment variable. Under most distributions, this can be done by editing `~/.bashrc` or `~/.bash-profile` with the following line (tested on OpenSUSE Tumbleweed):

```
export PYTHONPATH="$PYTHONPATH:/usr/share/qgis/python/plugins:/usr/share/qgis/
↳python"
```

Save the file and implement the environment settings by using the following shell command:

```
source ~/.bashrc
```

On Windows, you need to make sure that you have the same environment settings and use the same libraries and interpreter as QGIS. The fastest way to do this is to modify the startup batch file of QGIS.

If you used the OSGeo4W Installer, you can find this under the `bin` folder of your OSGeo4W install. Look for something like `C:\OSGeo4W\bin\qgis-unstable.bat`.

16.3.3 Debugging using Pyscripter IDE (Windows)

For using [Pyscripter IDE](#), here's what you have to do:

1. Make a copy of `qgis-unstable.bat` and rename it `pyscripter.bat`.
2. Open it in an editor. And remove the last line, the one that starts QGIS.
3. Add a line that points to your Pyscripter executable and add the command line argument that sets the version of Python to be used
4. Also add the argument that points to the folder where Pyscripter can find the Python dll used by QGIS, you can find this under the `bin` folder of your OSGeoW install

```
@echo off
SET OSGEO4W_ROOT=C:\OSGeo4W
call "%OSGEO4W_ROOT%\bin\o4w_env.bat"
call "%OSGEO4W_ROOT%\bin\gdal16.bat"
@echo off
path %PATH%;%GISBASE%\bin
Start C:\pyscripter\pyscripter.exe --python25 --pythondllpath=C:\OSGeo4W\bin
```

5. Now when you double click this batch file it will start Pyscripter, with the correct path.

More popular than Pyscripter, Eclipse is a common choice among developers. In the following section, we will be explaining how to configure it for developing and testing plugins.

16.3.4 Debugging using Eclipse and PyDev

Installation

To use Eclipse, make sure you have installed the following

- Eclipse
- Aptana Studio 3 Plugin or PyDev
- QGIS 3.x
- You may also want to install **Remote Debug**, a QGIS plugin. At the moment it's still experimental so enable  *Experimental plugins* under *Plugins ► Manage and Install plugins... ► Options* beforehand.

To prepare your environment for using Eclipse in Windows, you should also create a batch file and use it to start Eclipse:

1. Locate the folder where `qgis_core.dll` resides in. Normally this is `C:\OSGeo4W\apps\qgis\bin`, but if you compiled your own QGIS application this is in your build folder in `output/bin/RelWithDebInfo`
2. Locate your `eclipse.exe` executable.
3. Create the following script and use this to start eclipse when developing QGIS plugins.

```
call "C:\OSGeo4W\bin\o4w_env.bat"
set PATH=%PATH%;C:\path\to\your\qgis_core.dll\parent\folder
start /B C:\path\to\your\eclipse.exe
```

Setting up Eclipse

1. In Eclipse, create a new project. You can select *General Project* and link your real sources later on, so it does not really matter where you place this project.

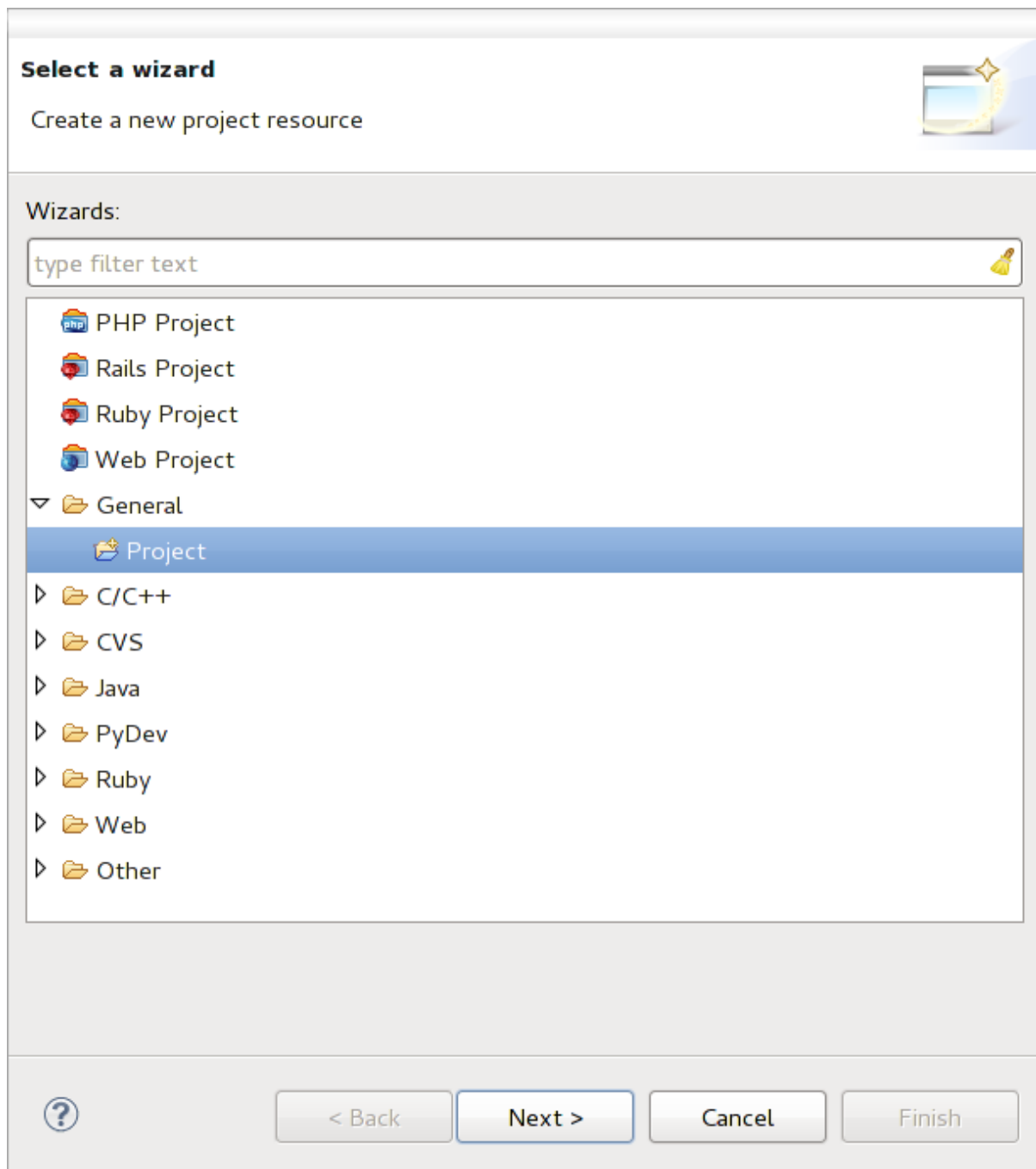


Fig. 16.1: Eclipse project

2. Right-click your new project and choose *New ► Folder*.
3. Click *Advanced* and choose *Link to alternate location (Linked Folder)*. In case you already have sources you want to debug, choose these. In case you don't, create a folder as it was already explained.

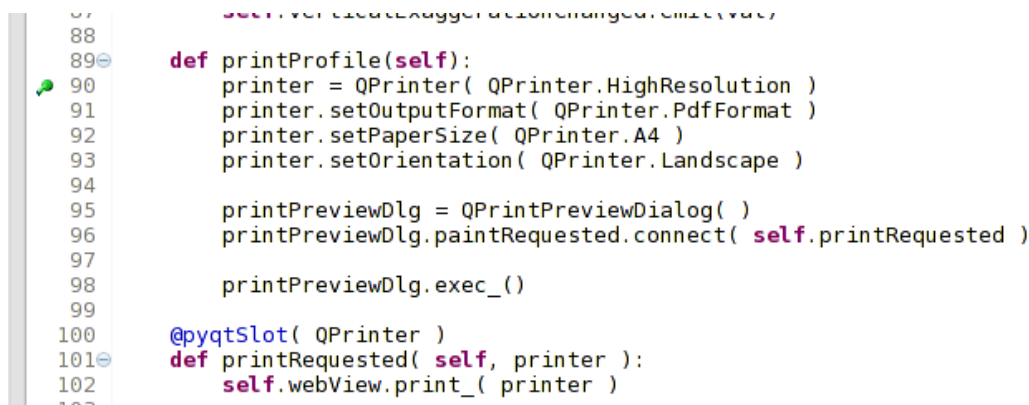
Now in the view *Project Explorer*, your source tree pops up and you can start working with the code. You already have syntax highlighting and all the other powerful IDE tools available.

Configuring the debugger

To get the debugger working:

1. Switch to the Debug perspective in Eclipse (*Window ► Open Perspective ► Other ► Debug*).
2. start the PyDev debug server by choosing *PyDev ► Start Debug Server*.
3. Eclipse is now waiting for a connection from QGIS to its debug server and when QGIS connects to the debug server it will allow it to control the python scripts. That's exactly what we installed the *Remote Debug* plugin for. So start QGIS in case you did not already and click the bug symbol.

Now you can set a breakpoint and as soon as the code hits it, execution will stop and you can inspect the current state of your plugin. (The breakpoint is the green dot in the image below, set one by double clicking in the white space left to the line you want the breakpoint to be set).



```

87         self.verticalExaggerationChanged.emit(val)
88
89     def printProfile(self):
90         printer = QPrinter( QPrinter.HighResolution )
91         printer.setOutputFormat( QPrinter.PdfFormat )
92         printer.setPaperSize( QPrinter.A4 )
93         printer.setOrientation( QPrinter.Landscape )
94
95         printPreviewDlg = QPrintPreviewDialog( )
96         printPreviewDlg.paintRequested.connect( self.printRequested )
97
98         printPreviewDlg.exec_()
99
100    @pyqtSlot( QPrinter )
101    def printRequested( self, printer ):
102        self.webView.print_( printer )
103

```

Fig. 16.2: Breakpoint

A very interesting thing you can make use of now is the debug console. Make sure that the execution is currently stopped at a breakpoint, before you proceed.

1. Open the Console view (*Window ► Show view*). It will show the *Debug Server* console which is not very interesting. But there is a button *Open Console* which lets you change to a more interesting PyDev Debug Console.
2. Click the arrow next to the *Open Console* button and choose *PyDev Console*. A window opens up to ask you which console you want to start.
3. Choose *PyDev Debug Console*. In case its greyed out and tells you to Start the debugger and select the valid frame, make sure that you've got the remote debugger attached and are currently on a breakpoint.

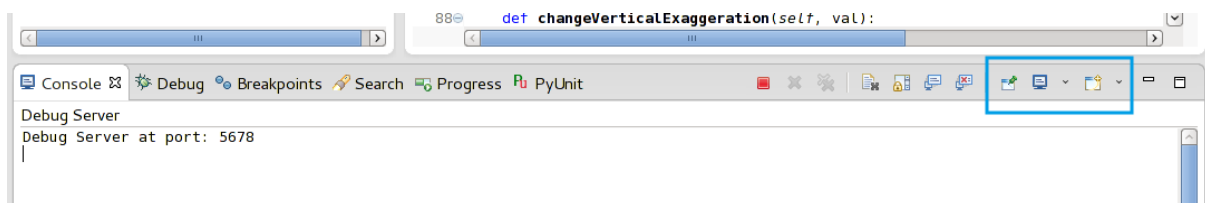


Fig. 16.3: PyDev Debug Console

You have now an interactive console which lets you test any commands from within the current context. You can manipulate variables or make API calls or whatever you like.



Tip

A little bit annoying is, that every time you enter a command, the console switches back to the Debug Server. To stop this behavior, you can click the *Pin Console* button when on the Debug Server page and it should remember this decision at least for the current debug session.

Making eclipse understand the API

A very handy feature is to have Eclipse actually know about the QGIS API. This enables it to check your code for typos. But not only this, it also enables Eclipse to help you with autocompletion from the imports to API calls.

To do this, Eclipse parses the QGIS library files and gets all the information out there. The only thing you have to do is to tell Eclipse where to find the libraries.

1. Click *Window ► Preferences ► PyDev ► Interpreter ► Python*.

You will see your configured python interpreter in the upper part of the window (at the moment python2.7 for QGIS) and some tabs in the lower part. The interesting tabs for us are *Libraries* and *Forced Builtins*.

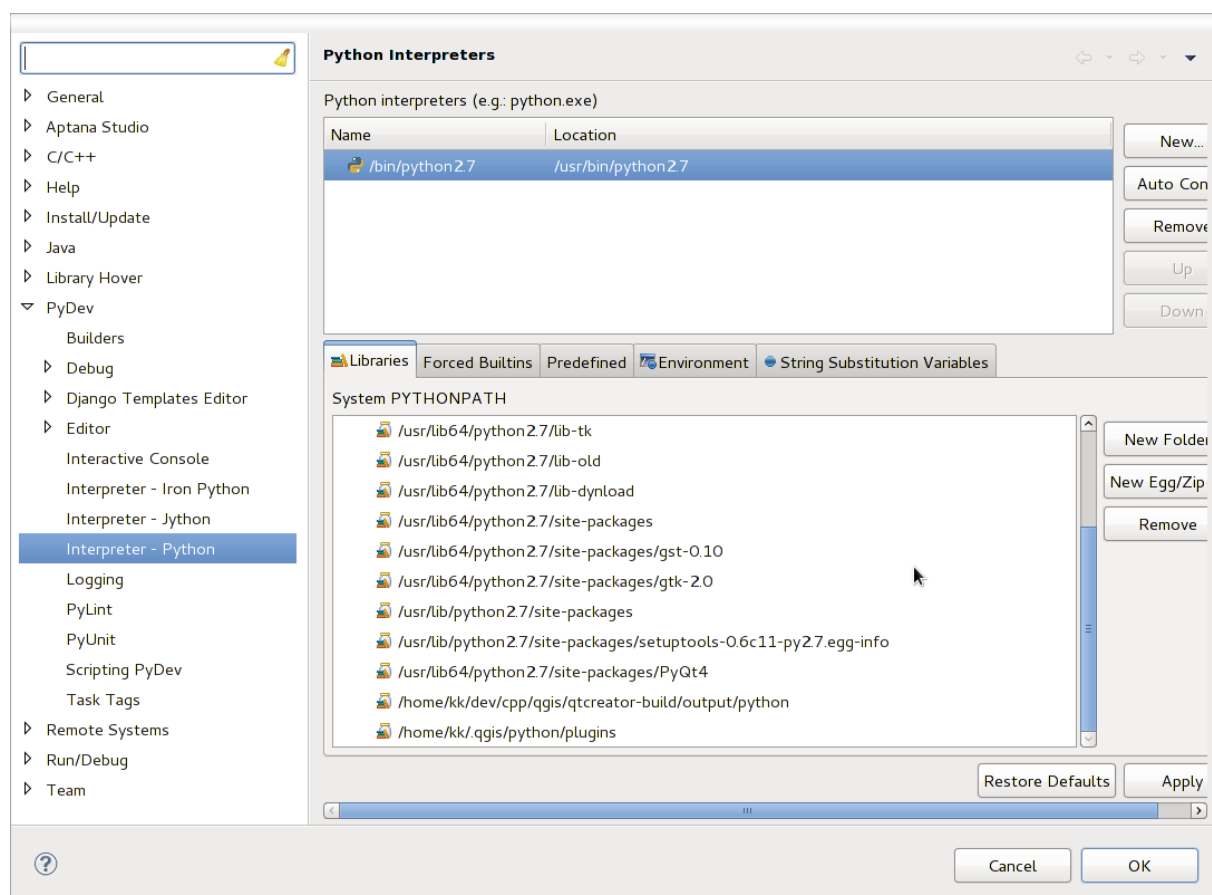


Fig. 16.4: PyDev Debug Console

2. First open the Libraries tab.
3. Add a New Folder and choose the python folder of your QGIS installation. If you do not know where this folder is (it's not the plugins folder):
 1. Open QGIS
 2. Start a python console
 3. Enter `qgis`

4. and press Enter. It will show you which QGIS module it uses and its path.
5. Strip the trailing `/qgis/___init___.pyc` from this path and you've got the path you are looking for.
4. You should also add your plugins folder here (it is in `python/plugins` under the user profile folder).
5. Next jump to the *Forced Builtins* tab, click on *New...* and enter `qgis`. This will make Eclipse parse the QGIS API. You probably also want Eclipse to know about the PyQt API. Therefore also add PyQt as forced builtin. That should probably already be present in your libraries tab.
6. Click *OK* and you're done.

Note

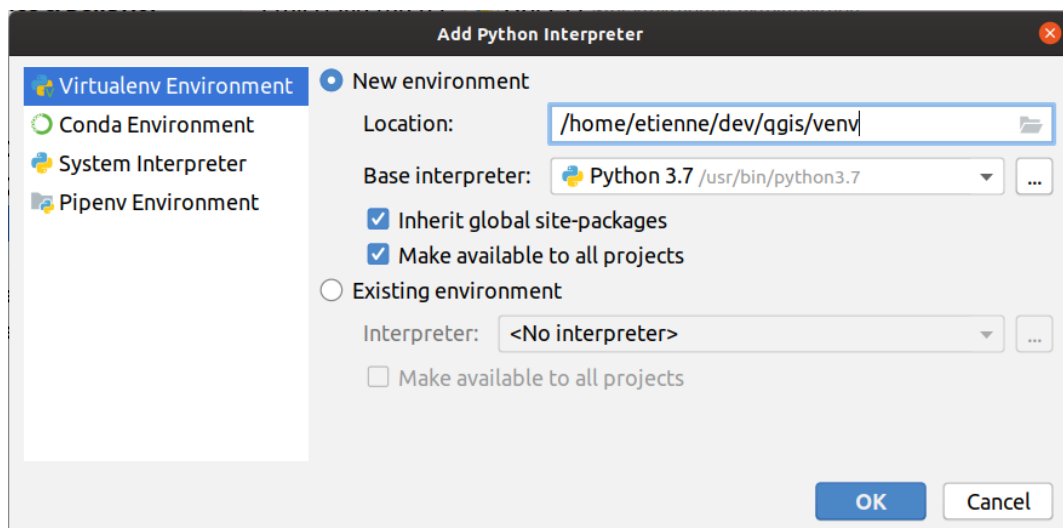
Every time the QGIS API changes (e.g. if you're compiling QGIS master and the SIP file changed), you should go back to this page and simply click *Apply*. This will let Eclipse parse all the libraries again.

16.3.5 Debugging with PyCharm on Ubuntu with a compiled QGIS

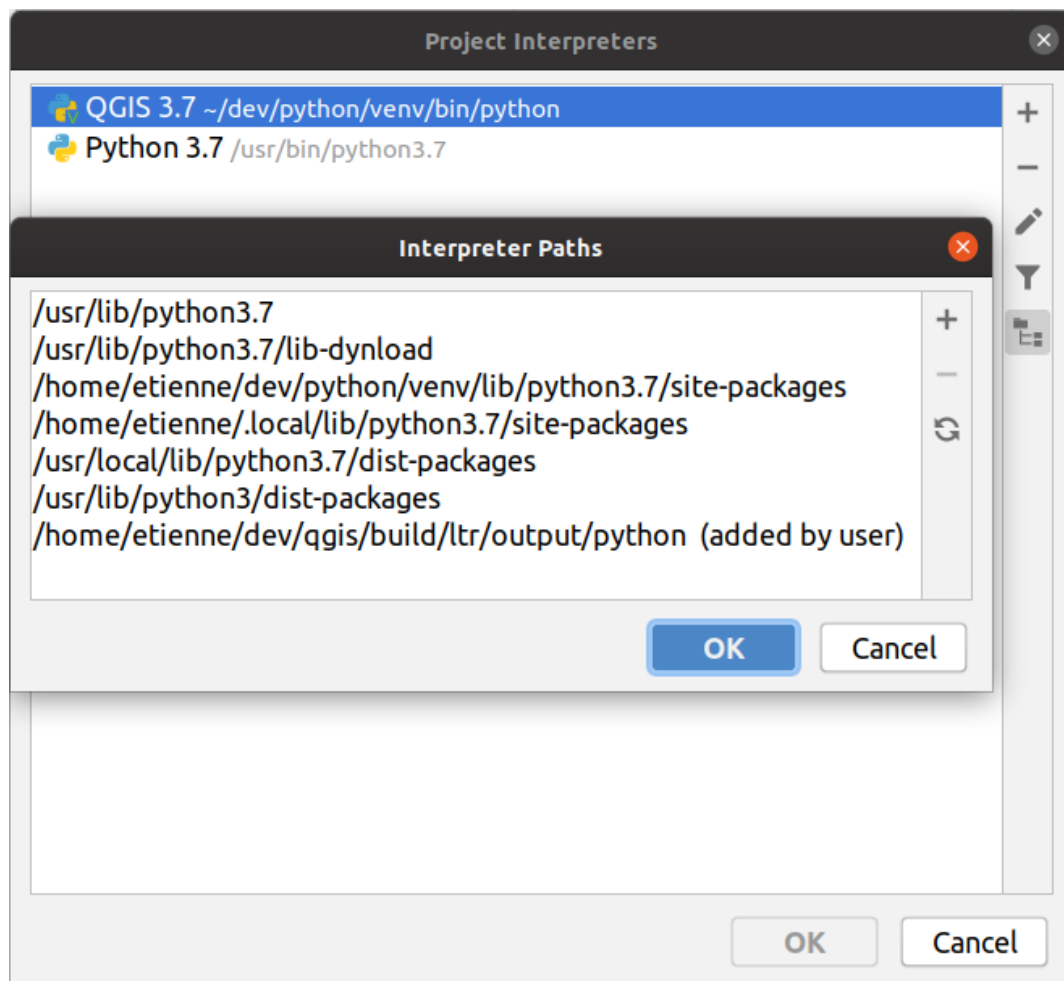
PyCharm is an IDE for Python developed by JetBrains. There is a free version called Community Edition and a paid one called Professional. You can download PyCharm on the website: <https://www.jetbrains.com/pycharm/download>

We are assuming that you have compiled QGIS on Ubuntu with the given build directory `~/dev/qgis/build/master`. It's not compulsory to have a self compiled QGIS, but only this has been tested. Paths must be adapted.

1. In PyCharm, in your *Project Properties*, *Project Interpreter*, we are going to create a Python Virtual environment called QGIS.
2. Click the small gear and then *Add*.
3. Select *Virtualenv environment*.
4. Select a generic location for all your Python projects such as `~/dev/qgis/venv` because we will use this Python interpreter for all our plugins.
5. Choose a Python 3 base interpreter available on your system and check the next two options *Inherit global site-packages* and *Make available to all projects*.



1. Click *OK*, come back on the small gear and click *Show all*.
2. In the new window, select your new interpreter QGIS and click the last icon in the vertical menu *Show paths for the selected interpreter*.
3. Finally, add the following absolute path to the list `~/dev/qgis/build/master/output/python`.



1. Restart PyCharm and you can start using this new Python virtual environment for all your plugins.

PyCharm will be aware of the QGIS API and also of the PyQt API if you use Qt provided by QGIS like `from qgis.PyQt.QtCore import QDir`. The autocompletion should work and PyCharm can inspect your code.

In the professional version of PyCharm, remote debugging is working well. For the Community edition, remote debugging is not available. You can only have access to a local debugger, meaning that the code must run *inside* PyCharm (as script or unittest), not in QGIS itself. For Python code running *in* QGIS, you might use the *First Aid* plugin mentioned above.

16.3.6 Debugging using PDB

If you do not use an IDE such as Eclipse or PyCharm, you can debug using PDB, following these steps.

1. First add this code in the spot where you would like to debug

```
# Use pdb for debugging
import pdb
# also import pyqtRemoveInputHook
from qgis.PyQt.QtCore import pyqtRemoveInputHook
# These lines allow you to set a breakpoint in the app
pyqtRemoveInputHook()
pdb.set_trace()
```

2. Then run QGIS from the command line.

On Linux do:

```
$ ./Qgis
```

On macOS do:

```
$ /Applications/Qgis.app/Contents/MacOS/Qgis
```

3. And when the application hits your breakpoint you can type in the console!

TODO:

Add testing information

16.4 Releasing your plugin

Once your plugin is ready and you think the plugin could be helpful for some people, do not hesitate to upload it to [Official Python plugin repository](#). On that page you can also find packaging guidelines about how to prepare the plugin to work well with the plugin installer. Or in case you would like to set up your own plugin repository, create a simple XML file that will list the plugins and their metadata.

Please take special care to the following suggestions:

16.4.1 Metadata and names

- avoid using a name too similar to existing plugins
- if your plugin has a similar functionality to an existing plugin, please explain the differences in the About field, so the user will know which one to use without the need to install and test it
- avoid repeating “plugin” in the name of the plugin itself
- use the description field in metadata for a 1 line description, the About field for more detailed instructions
- include a code repository, a bug tracker, and a home page; this will greatly enhance the possibility of collaboration, and can be done very easily with one of the available web infrastructures (GitHub, GitLab, Bitbucket, etc.)
- choose tags with care: avoid the uninformative ones (e.g. vector) and prefer the ones already used by others (see the plugin website)
- add a proper icon, do not leave the default one; see QGIS interface for a suggestion of the style to be used

16.4.2 Code and help

- do not include generated file (ui_*.py, resources_rc.py, generated help files...) and useless stuff (e.g. .gitignore) in repository
- add the plugin to the appropriate menu (Vector, Raster, Web, Database)
- when appropriate (plugins performing analyses), consider adding the plugin as a subplugin of Processing framework: this will allow users to run it in batch, to integrate it in more complex workflows, and will free you from the burden of designing an interface
- include at least minimal documentation and, if useful for testing and understanding, sample data.

16.4.3 Official Python plugin repository

You can find the *official* Python plugin repository at <https://plugins.qgis.org/>.

In order to use the official repository you must obtain an OSGEO ID from the [OSGEO web portal](#).

Once you have uploaded your plugin it will be approved by a staff member and you will be notified.

TODO:

Insert a link to the governance document

Permissions

These rules have been implemented in the official plugin repository:

- every registered user can add a new plugin
- *staff* users can approve or disapprove all plugin versions
- users which have the special permission *plugins.can_approve* get the versions they upload automatically approved
- users which have the special permission *plugins.can_approve* can approve versions uploaded by others as long as they are in the list of the plugin *owners*
- a particular plugin can be deleted and edited only by *staff* users and plugin *owners*
- if a user without *plugins.can_approve* permission uploads a new version, the plugin version is automatically unapproved.

Trust management

Staff members can grant *trust* to selected plugin creators setting *plugins.can_approve* permission through the front-end application.

The plugin details view offers direct links to grant trust to the plugin creator or the plugin *owners*.

Validation

Plugin's metadata are automatically imported and validated from the compressed package when the plugin is uploaded.

Here are some validation rules that you should aware of when you want to upload a plugin on the official repository:

1. the name of the main folder containing your plugin must contain only ASCII characters (A-Z and a-z), digits and the characters underscore (`_`) and minus (`-`), also it cannot start with a digit
2. `metadata.txt` is required
3. all required metadata listed in [metadata table](#) must be present
4. the *version* metadata field must be unique
5. a license file must be included, saved as `LICENSE` with no extension (i.e. not `LICENSE.txt` for example)

Plugin structure

Following the validation rules the compressed (.zip) package of your plugin must have a specific structure to validate as a functional plugin. As the plugin will be unzipped inside the users plugins folder it must have it's own directory inside the .zip file to not interfere with other plugins. Mandatory files are: `metadata.txt`, `__init__.py` and `LICENSE`. But it would be nice to have a `README` and of course an icon to represent the plugin. Following is an example of how a `plugin.zip` could look like.

```
plugin.zip
pluginfolder/
|-- i18n
|   |-- translation_file_de.ts
|-- img
|   |-- icon.png
|   |-- iconsource.svg
|-- __init__.py
|-- LICENSE
|-- Makefile
|-- metadata.txt
|-- more_code.py
|-- main_code.py
|-- README
|-- ui_Qt_user_interface_file.ui
```

It is possible to create plugins in the Python programming language. In comparison with classical plugins written in C++ these should be easier to write, understand, maintain and distribute due to the dynamic nature of the Python language.

Python plugins are listed together with C++ plugins in QGIS plugin manager. They are searched for in `~/ (UserProfile)/python/plugins` and these paths:

- UNIX/Mac: `(qgis_prefix)/share/qgis/python/plugins`
- Windows: `(qgis_prefix)/python/plugins`

For definitions of `~` and `(UserProfile)` see `core_and_external_plugins`.

Note

By setting `QGIS_PLUGINPATH` to an existing directory path, you can add this path to the list of paths that are searched for plugins.

WRITING A PROCESSING PLUGIN

Depending on the kind of plugin that you are going to develop, it might be a better option to add its functionality as a Processing algorithm (or a set of them). That would provide a better integration within QGIS, additional functionality (since it can be run in the components of Processing, such as the modeler or the batch processing interface), and a quicker development time (since Processing will take of a large part of the work).

To distribute those algorithms, you should create a new plugin that adds them to the Processing Toolbox. The plugin should contain an algorithm provider, which has to be registered when the plugin is instantiated.

17.1 Creating from scratch

To create a plugin from scratch which contains an algorithm provider, you can follow these steps using the Plugin Builder:

1. Install the **Plugin Builder** plugin
2. Create a new plugin using the Plugin Builder. When the Plugin Builder asks you for the template to use, select “Processing provider”.
3. The created plugin contains a provider with a single algorithm. Both the provider file and the algorithm file are fully commented and contain information about how to modify the provider and add additional algorithms. Refer to them for more information.

17.2 Updating a plugin

If you want to add your existing plugin to Processing, you need to add some code.

1. In your `metadata.txt` file, you need to add a variable:

```
hasProcessingProvider=yes
```

2. In the Python file where your plugin is setup with the `initGui` method, you need to adapt some lines like this:

```
1 from qgis.core import QgsApplication
2 from .processing_provider.provider import Provider
3
4 class YourPluginName:
5
6     def __init__(self):
7         self.provider = None
8
9     def initProcessing(self):
10         self.provider = Provider()
11         QgsApplication.processingRegistry().addProvider(self.provider)
12
```

(continues on next page)

(continued from previous page)

```

13 def initGui(self):
14     self.initProcessing()
15
16 def unload(self):
17     QgsApplication.processingRegistry().removeProvider(self.provider)

```

3. You can create a folder `processing_provider` with three files in it:

- `__init__.py` with nothing in it. This is necessary to make a valid Python package.
- `provider.py` which will create the Processing provider and expose your algorithms.

```

1 from qgis.core import QgsProcessingProvider
2 from qgis.PyQt.QtGui import QIcon
3
4 from .example_processing_algorithm import ExampleProcessingAlgorithm
5
6 class Provider(QgsProcessingProvider):
7
8     """ The provider of our plugin. """
9
10    def loadAlgorithms(self):
11        """ Load each algorithm into the current provider. """
12        self.addAlgorithm(ExampleProcessingAlgorithm())
13        # add additional algorithms here
14        # self.addAlgorithm(MyOtherAlgorithm())
15
16    def id(self) -> str:
17        """The ID of your plugin, used for identifying the provider.
18
19        This string should be a unique, short, character only string,
20        eg "qgis" or "gdal". This string should not be localised.
21        """
22        return 'yourplugin'
23
24    def name(self) -> str:
25        """The human friendly name of your plugin in Processing.
26
27        This string should be as short as possible (e.g. "Lastools", not
28        "Lastools version 1.0.1 64-bit") and localised.
29        """
30        return self.tr('Your plugin')
31
32    def icon(self) -> QIcon:
33        """Should return a QIcon which is used for your provider inside
34        the Processing toolbox.
35        """
36        return QgsProcessingProvider.icon(self)

```

- `example_processing_algorithm.py` which contains the example algorithm file. Copy/paste the content of the [script template file](#) and update it according to your needs.

You should have a tree similar to this:

```

1 └─ your_plugin_root_folder
2   └─ __init__.py
3   └─ LICENSE
4   └─ metadata.txt
5   └─ processing_provider
6       └─ example_processing_algorithm.py
7       └─ __init__.py
8       └─ provider.py

```

4. Now you can reload your plugin in QGIS and you should see your example script in the Processing toolbox and modeler.

17.3 Implementing custom Processing algorithms

17.3.1 Creating a custom algorithm

Here's a simple example of a custom buffer algorithm:

```

1  from qgis.core import (
2      QgsProcessingAlgorithm,
3      QgsProcessingParameterFeatureSource,
4      QgsProcessingParameterNumber,
5      QgsProcessingParameterFeatureSink,
6      QgsFeatureSink,
7  )
8
9  class BufferAlgorithm(QgsProcessingAlgorithm):
10
11      INPUT = 'INPUT'
12      DISTANCE = 'DISTANCE'
13      OUTPUT = 'OUTPUT'
14
15      def initAlgorithm(self, config=None):
16          self.addParameter(QgsProcessingParameterFeatureSource(self.INPUT, 'Input_
↳ layer'))
17          self.addParameter(QgsProcessingParameterNumber(self.DISTANCE, 'Buffer_
↳ distance', defaultValue=100.0))
18          self.addParameter(QgsProcessingParameterFeatureSink(self.OUTPUT, 'Output_
↳ layer'))
19
20      def processAlgorithm(self, parameters, context, feedback):
21          source = self.parameterAsSource(parameters, self.INPUT, context)
22          distance = self.parameterAsDouble(parameters, self.DISTANCE, context)
23          (sink, dest_id) = self.parameterAsSink(parameters, self.OUTPUT, context,
24                                                    source.fields(), source.wkbType(),
↳ source.sourceCrs())
25
26          for f in source.getFeatures():
27              f.setGeometry(f.geometry().buffer(distance, 5))
28              sink.addFeature(f, QgsFeatureSink.FastInsert)
29
30          return {self.OUTPUT: dest_id}
31
32      def name(self):
33          return 'buffer'
34
35      def displayName(self):
36          return 'Buffer Features'
37
38      def group(self):
39          return 'Examples'
40
41      def groupId(self):
42          return 'examples'
43
44      def createInstance(self):
45          return BufferAlgorithm()

```

17.3.2 Customizing the algorithm dialog

Custom dialogs are especially useful when working with nested or dynamic inputs, when parameters depend on external data sources such as APIs (e.g. dynamically populated dropdowns), or when you need advanced validation and custom layout behavior that isn't supported by the default Processing dialog. To override the default UI (e.g. for complex parameter types or dynamic logic), subclass `QgsProcessingAlgorithmDialogBase`. To render your custom UI in the standard Processing dialog window, you must call `self.setMainWidget(panel)`, where `panel` is a `QgsPanelWidget` containing your custom layout. This ensures your interface is correctly displayed and interacts properly with the Processing framework.

Here is an example that integrates signal management using `QTimer` for debounced input:

```

1  from qgis.PyQt.QtCore import Qt, QT_VERSION_STR, QTimer
2  from qgis.core import (
3      QgsProcessingAlgorithm,
4      QgsProcessingContext,
5      QgsProcessingFeedback,
6      Qgs,
7  )
8  from qgis.PyQt.QtWidgets import QWidget, QVBoxLayout, QLineEdit
9  from qgis import gui, processing
10 from datetime import datetime
11 from typing import Dict, Optional
12 from osgeo import gdal
13
14 class CustomAlgorithmDialog(gui.QgsProcessingAlgorithmDialogBase):
15     def __init__(
16         self,
17         algorithm: QgsProcessingAlgorithm,
18         parent: Optional[QWidget] = None,
19         title: Optional[str] = None,
20     ):
21         super().__init__(
22             parent,
23             flags=Qt.WindowFlags(),
24             mode=gui.QgsProcessingAlgorithmDialogBase.DialogMode.Single,
25         )
26         self.context = QgsProcessingContext()
27         self.setAlgorithm(algorithm)
28         self.setModal(True)
29         self.setWindowTitle(title or algorithm.displayName())
30
31         self.panel = gui.QgsPanelWidget()
32         layout = self.buildDialog()
33         self.panel.setLayout(layout)
34         self.setMainWidget(self.panel)
35
36         self.cancelButton().clicked.connect(self.reject)
37
38     def buildDialog(self) -> QVBoxLayout:
39         layout = QVBoxLayout()
40
41         self.input = QLineEdit()
42
43         # Set up a debounced signal using QTimer
44         self._update_timer = QTimer(self, singleShot=True)
45         self._update_timer.timeout.connect(self._on_collection_id_ready)
46         self.input.textChanged.connect(self._on_collection_id_changed)
47
48         layout.addWidget(self.input)
49
50     return layout

```

(continues on next page)

(continued from previous page)

```

51
52     def _on_collection_id_changed(self):
53         self._update_timer.start(500) # Debounce input
54
55     def _on_collection_id_ready(self):
56         self.pushInfo("Fetching metadata for collection ID...")
57
58     def getParameters(self) -> Dict:
59         try:
60             return {'DISTANCE': float(self.input.text())}
61         except ValueError:
62             raise ValueError("Invalid buffer distance")
63
64     def processingContext(self):
65         return self.context
66
67     def createFeedback(self):
68         return QgsProcessingFeedback()
69
70     def runAlgorithm(self):
71         context = self.processingContext()
72         feedback = self.createFeedback()
73         params = self.getParameters()
74
75         self.pushDebugInfo(f"QGIS version: {Qgis.QGIS_VERSION}")
76         self.pushDebugInfo(f"QGIS code revision: {Qgis.QGIS_DEV_VERSION}")
77         self.pushDebugInfo(f"Qt version: {QT_VERSION_STR}")
78         self.pushDebugInfo(f"GDAL version: {gdal.VersionInfo('--version')}")
79         self.pushCommandInfo(f"Algorithm started at: {datetime.now().
80         ↪isoformat(timespec='seconds')}")
81         self.pushCommandInfo(f"Algorithm '{self.algorithm().displayName()}'
82         ↪starting...")
83         self.pushCommandInfo("Input parameters:")
84         for k, v in params.items():
85             self.pushCommandInfo(f"    {k}: {v}")
86
87         results = processing.run(self.algorithm(), params, context=context,
88         ↪feedback=feedback)
89         self.setResults(results)
90         self.showLog()

```

To launch the custom dialog for a given algorithm, simply instantiate `CustomAlgorithmDialog` with your algorithm instance and call `exec()`:

```

dlg = CustomAlgorithmDialog(BufferAlgorithm())
dlg.exec()

```

17.3.3 Managing Qt Signals

When building reactive dialogs, manage signal connections carefully. The above pattern uses a `QTimer` to debounce input from the text field, preventing rapid repeated calls. This is especially useful when fetching metadata or updating UI elements based on user input. Always connect signals once (typically in `__init__`) and use `singleShot=True` to ensure the slot is triggered only once after a delay.

USING PLUGIN LAYERS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.core import (  
2     QgsPluginLayer,  
3     QgsPluginLayerType,  
4     QgsMapLayerRenderer,  
5     QgsApplication,  
6     QgsProject,  
7 )  
8  
9 from qgis.PyQt.QtGui import QImage
```

If your plugin uses its own methods to render a map layer, writing your own layer type based on `QgsPluginLayer` might be the best way to implement that.

18.1 Subclassing `QgsPluginLayer`

Below is an example of a minimal `QgsPluginLayer` implementation. It is based on the original code of the [Watermark example plugin](#).

The custom renderer is the part of the implement that defines the actual drawing on the canvas.

```
1 class WatermarkLayerRenderer(QgsMapLayerRenderer):  
2  
3     def __init__(self, layerId, rendererContext):  
4         super().__init__(layerId, rendererContext)  
5  
6     def render(self):  
7         image = QImage("/usr/share/icons/hicolor/128x128/apps/qgis.png")  
8         painter = self.rendererContext().painter()  
9         painter.save()  
10        painter.drawImage(10, 10, image)  
11        painter.restore()  
12        return True  
13  
14 class WatermarkPluginLayer(QgsPluginLayer):  
15  
16     LAYER_TYPE="watermark"  
17  
18     def __init__(self):  
19         super().__init__(WatermarkPluginLayer.LAYER_TYPE, "Watermark plugin layer")  
20         self.setValid(True)
```

(continues on next page)

(continued from previous page)

```

21
22     def createMapRenderer(self, rendererContext):
23         return WatermarkLayerRenderer(self.id(), rendererContext)
24
25     def setTransformContext(self, ct):
26         pass
27
28     # Methods for reading and writing specific information to the project file can
29     # also be added:
30
31     def readXml(self, node, context):
32         pass
33
34     def writeXml(self, node, doc, context):
35         pass

```

The plugin layer can be added to the project and to the canvas as any other map layer:

```

plugin_layer = WatermarkPluginLayer()
QgsProject.instance().addMapLayer(plugin_layer)

```

When loading a project containing such a layer, a factory class is needed:

```

1  class WatermarkPluginLayerType(QgsPluginLayerType):
2
3      def __init__(self):
4          super().__init__(WatermarkPluginLayer.LAYER_TYPE)
5
6      def createLayer(self):
7          return WatermarkPluginLayer()
8
9      # You can also add GUI code for displaying custom information
10     # in the layer properties
11     def showLayerProperties(self, layer):
12         pass
13
14
15     # Keep a reference to the instance in Python so it won't
16     # be garbage collected
17     plt = WatermarkPluginLayerType()
18
19     assert QgsApplication.pluginLayerRegistry().addPluginLayerType(plt)

```

NETWORK ANALYSIS LIBRARY

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
from qgis.core import (
    QgsVectorLayer,
    QgsPointXY,
)
```

The network analysis library can be used to:

- create mathematical graph from geographical data (polyline vector layers)
- implement basic methods from graph theory (currently only Dijkstra's algorithm)

The network analysis library was created by exporting basic functions from the RoadGraph core plugin and now you can use its methods in plugins or directly from the Python console.

19.1 General information

Briefly, a typical use case can be described as:

1. create graph from geodata (usually polyline vector layer)
2. run graph analysis
3. use analysis results (for example, visualize them)

19.2 Building a graph

The first thing you need to do — is to prepare input data, that is to convert a vector layer into a graph. All further actions will use this graph, not the layer.

As a source we can use any polyline vector layer. Nodes of the polylines become graph vertexes, and segments of the polylines are graph edges. If several nodes have the same coordinates then they are the same graph vertex. So two lines that have a common node become connected to each other.

Additionally, during graph creation it is possible to “fix” (“tie”) to the input vector layer any number of additional points. For each additional point a match will be found — the closest graph vertex or closest graph edge. In the latter case the edge will be split and a new vertex added.

Vector layer attributes and length of an edge can be used as the properties of an edge.

Converting from a vector layer to the graph is done using the [Builder](#) programming pattern. A graph is constructed using a so-called Director. There is only one Director for now: `QgsVectorLayerDirector`. The director sets the basic settings that will be used to construct a graph from a line vector layer, used by the builder to create the graph.

Currently, as in the case with the director, only one builder exists: `QgsGraphBuilder`, that creates `QgsGraph` objects. You may want to implement your own builders that will build a graph compatible with such libraries as `BGL` or `NetworkX`.

To calculate edge properties the programming pattern `strategy` is used. For now only `QgsNetworkDistanceStrategy` (that takes into account the length of the route) and `QgsNetworkSpeedStrategy` (that also considers the speed) are available. You can implement your own strategy that will use all necessary parameters. For example, `RoadGraph` plugin uses a strategy that computes travel time using edge length and speed value from attributes.

It's time to dive into the process.

First of all, to use this library we should import the analysis module

```
from qgis.analysis import *
```

Then some examples for creating a director

```
1 # don't use information about road direction from layer attributes,
2 # all roads are treated as two-way
3 director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '',
4     ↳QgsVectorLayerDirector.DirectionBoth)
5
6 # use field with index 5 as source of information about road direction.
7 # one-way roads with direct direction have attribute value "yes",
8 # one-way roads with reverse direction have the value "1", and accordingly
9 # bidirectional roads have "no". By default roads are treated as two-way.
10 director = QgsVectorLayerDirector(vectorLayer, 5, 'yes', '1', 'no',
    ↳QgsVectorLayerDirector.DirectionBoth)
```

To construct a director, we should pass a vector layer that will be used as the source for the graph structure and information about allowed movement on each road segment (one-way or bidirectional movement, direct or reverse direction). The call looks like this

```
1 director = QgsVectorLayerDirector(vectorLayer,
2     directionFieldId,
3     directDirectionValue,
4     reverseDirectionValue,
5     bothDirectionValue,
6     defaultDirection)
```

And here is full list of what these parameters mean:

- `vectorLayer` — vector layer used to build the graph
- `directionFieldId` — index of the attribute table field, where information about roads direction is stored. If `-1`, then don't use this info at all. An integer.
- `directDirectionValue` — field value for roads with direct direction (moving from first line point to last one). A string.
- `reverseDirectionValue` — field value for roads with reverse direction (moving from last line point to first one). A string.
- `bothDirectionValue` — field value for bidirectional roads (for such roads we can move from first point to last and from last to first). A string.
- `defaultDirection` — default road direction. This value will be used for those roads where field `directionFieldId` is not set or has some value different from any of the three values specified above. Possible values are:
 - `QgsVectorLayerDirector.DirectionForward` — One-way direct
 - `QgsVectorLayerDirector.DirectionBackward` — One-way reverse

– `QgsVectorLayerDirector.DirectionBoth` — Two-way

It is necessary then to create a strategy for calculating edge properties

```

1  # The index of the field that contains information about the edge speed
2  attributeId = 1
3  # Default speed value
4  defaultValue = 50
5  # Conversion from speed to metric units ('1' means no conversion)
6  toMetricFactor = 1
7  strategy = QgsNetworkSpeedStrategy(attributeId, defaultValue, toMetricFactor)

```

And tell the director about this strategy

```

director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '', 3)
director.addStrategy(strategy)

```

Now we can use the builder, which will create the graph. The `QgsGraphBuilder` class constructor takes several arguments:

- `crs` — coordinate reference system to use. Mandatory argument.
- `otfEnabled` — use “on the fly” reprojection or no. By default `True` (use OTF).
- `topologyTolerance` — topological tolerance. Default value is 0.
- `ellipsoidID` — ellipsoid to use. By default “WGS84”.

```

# only CRS is set, all other values are defaults
builder = QgsGraphBuilder(vectorLayer.crs())

```

Also we can define several points, which will be used in the analysis. For example

```

startPoint = QgsPointXY(1179720.1871, 5419067.3507)
endPoint = QgsPointXY(1180616.0205, 5419745.7839)

```

Now all is in place so we can build the graph and “tie” these points to it

```

tiedPoints = director.makeGraph(builder, [startPoint, endPoint])

```

Building the graph can take some time (which depends on the number of features in a layer and layer size). `tiedPoints` is a list with coordinates of “tied” points. When the build operation is finished we can get the graph and use it for the analysis

```

graph = builder.graph()

```

With the next code we can get the vertex indexes of our points

```

startId = graph.findVertex(tiedPoints[0])
endId = graph.findVertex(tiedPoints[1])

```

19.3 Graph analysis

Networks analysis is used to find answers to two questions: which vertexes are connected and how to find a shortest path. To solve these problems the network analysis library provides Dijkstra’s algorithm.

Dijkstra’s algorithm finds the shortest route from one of the vertexes of the graph to all the others and the values of the optimization parameters. The results can be represented as a shortest path tree.

The shortest path tree is a directed weighted graph (or more precisely a tree) with the following properties:

- only one vertex has no incoming edges — the root of the tree

- all other vertexes have only one incoming edge
- if vertex B is reachable from vertex A, then the path from A to B is the single available path and it is optimal (shortest) on this graph

To get the shortest path tree use the methods `shortestTree()` and `dijkstra()` of the `QgsGraphAnalyzer` class. It is recommended to use the `dijkstra()` method because it works faster and uses memory more efficiently.

The `shortestTree()` method is useful when you want to walk around the shortest path tree. It always creates a new graph object (`QgsGraph`) and accepts three variables:

- `source` — input graph
- `startVertexIdx` — index of the point on the tree (the root of the tree)
- `criterionNum` — number of edge property to use (started from 0).

```
tree = QgsGraphAnalyzer.shortestTree(graph, startId, 0)
```

The `dijkstra()` method has the same arguments, but returns a tuple of arrays:

- In the first array, element n contains index of the incoming edge or -1 if there are no incoming edges.
- In the second array, element n contains the distance from the root of the tree to vertex n or `DOUBLE_MAX` if vertex n is unreachable from the root.

```
(tree, cost) = QgsGraphAnalyzer.dijkstra(graph, startId, 0)
```

Here is some very simple code to display the shortest path tree using the graph created with the `shortestTree()` method (select linestring layer in *Layers* panel and replace coordinates with your own).

Warning

Use this code only as an example, it creates a lot of `QgsRubberBand` objects and may be slow on large datasets.

```
1 from qgis.core import *
2 from qgis.gui import *
3 from qgis.analysis import *
4 from qgis.PyQt.QtCore import *
5 from qgis.PyQt.QtGui import *
6
7 vectorLayer = QgsVectorLayer('testdata/network.gpkg|layername=network_lines',
8                               ↳'lines')
9 director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '',
10                                  ↳QgsVectorLayerDirector.DirectionBoth)
11 strategy = QgsNetworkDistanceStrategy()
12 director.addStrategy(strategy)
13 builder = QgsGraphBuilder(vectorLayer.crs())
14
15 pStart = QgsPointXY(1179661.925139, 5419188.074362)
16 tiedPoint = director.makeGraph(builder, [pStart])
17 pStart = tiedPoint[0]
18
19 graph = builder.graph()
20
21 idStart = graph.findVertex(pStart)
22
23 tree = QgsGraphAnalyzer.shortestTree(graph, idStart, 0)
24
25 i = 0
26 while (i < tree.edgeCount()):
27     rb = QgsRubberBand(iface.mapCanvas())
```

(continues on next page)

(continued from previous page)

```

26     rb.setColor (Qt.red)
27     rb.addPoint (tree.vertex(tree.edge(i).fromVertex()).point())
28     rb.addPoint (tree.vertex(tree.edge(i).toVertex()).point())
29     i = i + 1

```

Same thing but using the `dijkstra()` method

```

1  from qgis.core import *
2  from qgis.gui import *
3  from qgis.analysis import *
4  from qgis.PyQt.QtCore import *
5  from qgis.PyQt.QtGui import *
6
7  vectorLayer = QgsVectorLayer('testdata/network.gpkg|layername=network_lines',
8                               ↪ 'lines')
9
10 director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '',
11                                   ↪ QgsVectorLayerDirector.DirectionBoth)
12 strategy = QgsNetworkDistanceStrategy()
13 director.addStrategy(strategy)
14 builder = QgsGraphBuilder(vectorLayer.crs())
15
16 pStart = QgsPointXY(1179661.925139, 5419188.074362)
17 tiedPoint = director.makeGraph(builder, [pStart])
18 pStart = tiedPoint[0]
19
20 graph = builder.graph()
21
22 idStart = graph.findVertex(pStart)
23
24 (tree, costs) = QgsGraphAnalyzer.dijkstra(graph, idStart, 0)
25
26 for edgeId in tree:
27     if edgeId == -1:
28         continue
29     rb = QgsRubberBand(iface.mapCanvas())
30     rb.setColor (Qt.red)
31     rb.addPoint (graph.vertex(graph.edge(edgeId).fromVertex()).point())
32     rb.addPoint (graph.vertex(graph.edge(edgeId).toVertex()).point())

```

19.3.1 Finding shortest paths

To find the optimal path between two points the following approach is used. Both points (start A and end B) are “tied” to the graph when it is built. Then using the `shortestTree()` or `dijkstra()` method we build the shortest path with root in the start point A. In the same tree we also find the end point B and start to walk through the tree from point B to point A. The whole algorithm can be written as:

```

1  assign T = B
2  while T != A
3      add point T to path
4      get incoming edge for point T
5      look for point TT, that is start point of this edge
6      assign T = TT
7  add point A to path

```

At this point we have the path, in the form of the inverted list of vertexes (vertexes are listed in reversed order from end point to start point) that will be visited during traveling by this path.

Here is the sample code for QGIS Python Console (you may need to load and select a linestring layer in TOC and replace coordinates in the code with yours) that uses the `shortestTree()` method

```

1 from qgis.core import *
2 from qgis.gui import *
3 from qgis.analysis import *
4
5 from qgis.PyQt.QtCore import *
6 from qgis.PyQt.QtGui import *
7
8 vectorLayer = QgsVectorLayer('testdata/network.gpkg|layername=network_lines',
9                               ↪ 'lines')
10 builder = QgsGraphBuilder(vectorLayer.sourceCrs())
11 director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '',
12                                   ↪ QgsVectorLayerDirector.DirectionBoth)
13 strategy = QgsNetworkDistanceStrategy()
14 director.addStrategy(strategy)
15
16 startPoint = QgsPointXY(1179661.925139, 5419188.074362)
17 endPoint = QgsPointXY(1180942.970617, 5420040.097560)
18
19 tiedPoints = director.makeGraph(builder, [startPoint, endPoint])
20 tStart, tStop = tiedPoints
21
22 graph = builder.graph()
23 idxStart = graph.findVertex(tStart)
24
25 tree = QgsGraphAnalyzer.shortestTree(graph, idxStart, 0)
26
27 idxStart = tree.findVertex(tStart)
28 idxEnd = tree.findVertex(tStop)
29
30 if idxEnd == -1:
31     raise Exception('No route!')
32
33 # Add last point
34 route = [tree.vertex(idxEnd).point()]
35
36 # Iterate the graph
37 while idxEnd != idxStart:
38     edgeIds = tree.vertex(idxEnd).incomingEdges()
39     if len(edgeIds) == 0:
40         break
41     edge = tree.edge(edgeIds[0])
42     route.insert(0, tree.vertex(edge.fromVertex()).point())
43     idxEnd = edge.fromVertex()
44
45 # Display
46 rb = QgsRubberBand(iface.mapCanvas())
47 rb.setColor(Qt.green)
48
49 # This may require coordinate transformation if project's CRS
50 # is different than layer's CRS
51 for p in route:
52     rb.addPoint(p)

```

And here is the same sample but using the `dijkstra()` method

```

1 from qgis.core import *
2 from qgis.gui import *
3 from qgis.analysis import *
4
5 from qgis.PyQt.QtCore import *
6 from qgis.PyQt.QtGui import *
7

```

(continues on next page)

(continued from previous page)

```

8 vectorLayer = QgsVectorLayer('testdata/network.gpkg|layername=network_lines',
  ↳ 'lines')
9 director = QgsVectorLayerDirector(vectorLayer, -1, '', '', '',
  ↳ QgsVectorLayerDirector.DirectionBoth)
10 strategy = QgsNetworkDistanceStrategy()
11 director.addStrategy(strategy)
12
13 builder = QgsGraphBuilder(vectorLayer.sourceCrs())
14
15 startPoint = QgsPointXY(1179661.925139, 5419188.074362)
16 endPoint = QgsPointXY(1180942.970617, 5420040.097560)
17
18 tiedPoints = director.makeGraph(builder, [startPoint, endPoint])
19 tStart, tStop = tiedPoints
20
21 graph = builder.graph()
22 idxStart = graph.findVertex(tStart)
23 idxEnd = graph.findVertex(tStop)
24
25 (tree, costs) = QgsGraphAnalyzer.dijkstra(graph, idxStart, 0)
26
27 if tree[idxEnd] == -1:
28     raise Exception('No route!')
29
30 # Total cost
31 cost = costs[idxEnd]
32
33 # Add last point
34 route = [graph.vertex(idxEnd).point()]
35
36 # Iterate the graph
37 while idxEnd != idxStart:
38     idxEnd = graph.edge(tree[idxEnd]).fromVertex()
39     route.insert(0, graph.vertex(idxEnd).point())
40
41 # Display
42 rb = QgsRubberBand(iface.mapCanvas())
43 rb.setColor(Qt.red)
44
45 # This may require coordinate transformation if project's CRS
46 # is different than layer's CRS
47 for p in route:
48     rb.addPoint(p)

```

19.3.2 Areas of availability

The area of availability for vertex A is the subset of graph vertexes that are accessible from vertex A and the cost of the paths from A to these vertexes are not greater than some value.

More clearly this can be shown with the following example: “There is a fire station. Which parts of city can a fire truck reach in 5 minutes? 10 minutes? 15 minutes?”. Answers to these questions are fire station’s areas of availability.

To find the areas of availability we can use the `dijkstra()` method of the `QgsGraphAnalyzer` class. It is enough to compare the elements of the cost array with a predefined value. If `cost[i]` is less than or equal to a predefined value, then vertex `i` is inside the area of availability, otherwise it is outside.

A more difficult problem is to get the borders of the area of availability. The bottom border is the set of vertexes that are still accessible, and the top border is the set of vertexes that are not accessible. In fact this is simple: it is the availability border based on the edges of the shortest path tree for which the source vertex of the edge is accessible and the target vertex of the edge is not.

Here is an example

```

1 director = QgsVectorLayerDirector(vectorLayer, -1, ' ', ' ', ' ',
  ↳QgsVectorLayerDirector.DirectionBoth)
2 strategy = QgsNetworkDistanceStrategy()
3 director.addStrategy(strategy)
4 builder = QgsGraphBuilder(vectorLayer.crs())
5
6
7 pStart = QgsPointXY(1179661.925139, 5419188.074362)
8 delta = iface.mapCanvas().getCoordinateTransform().mapUnitsPerPixel() * 1
9
10 rb = QgsRubberBand(iface.mapCanvas())
11 rb.setColor(Qt.green)
12 rb.addPoint(QgsPointXY(pStart.x() - delta, pStart.y() - delta))
13 rb.addPoint(QgsPointXY(pStart.x() + delta, pStart.y() - delta))
14 rb.addPoint(QgsPointXY(pStart.x() + delta, pStart.y() + delta))
15 rb.addPoint(QgsPointXY(pStart.x() - delta, pStart.y() + delta))
16
17 tiedPoints = director.makeGraph(builder, [pStart])
18 graph = builder.graph()
19 tStart = tiedPoints[0]
20
21 idStart = graph.findVertex(tStart)
22
23 (tree, cost) = QgsGraphAnalyzer.dijkstra(graph, idStart, 0)
24
25 upperBound = []
26 r = 1500.0
27 i = 0
28 tree.reverse()
29
30 while i < len(cost):
31     if cost[i] > r and tree[i] != -1:
32         outVertexId = graph.edge(tree[i]).toVertex()
33         if cost[outVertexId] < r:
34             upperBound.append(i)
35     i = i + 1
36
37 for i in upperBound:
38     centerPoint = graph.vertex(i).point()
39     rb = QgsRubberBand(iface.mapCanvas())
40     rb.setColor(Qt.red)
41     rb.addPoint(QgsPointXY(centerPoint.x() - delta, centerPoint.y() - delta))
42     rb.addPoint(QgsPointXY(centerPoint.x() + delta, centerPoint.y() - delta))
43     rb.addPoint(QgsPointXY(centerPoint.x() + delta, centerPoint.y() + delta))
44     rb.addPoint(QgsPointXY(centerPoint.x() - delta, centerPoint.y() + delta))

```

QGIS SERVER AND PYTHON

20.1 Introduction

To learn more about QGIS Server, read the QGIS-Server-manual.

QGIS Server is three different things:

1. QGIS Server library: a library that provides an API for creating OGC web services
2. QGIS Server FCGI: a FCGI binary application `qgis_mapserv.fcgi` that together with a web server implements a set of OGC services (WMS, WFS, WCS etc.) and OGC APIs (WFS3/OAPIF)
3. QGIS Development Server: a development server binary application `qgis_mapserver` that implements a set of OGC services (WMS, WFS, WCS etc.) and OGC APIs (WFS3/OAPIF)

This chapter of the cookbook focuses on the first topic and by explaining the usage of QGIS Server API it shows how it is possible to use Python to extend, enhance or customize the server behavior or how to use the QGIS Server API to embed QGIS server into another application.

There are a few different ways you can alter the behavior of QGIS Server or extend its capabilities to offer new custom services or APIs, these are the main scenarios you may face:

- EMBEDDING → Use QGIS Server API from another Python application
- STANDALONE → Run QGIS Server as a standalone WSGI/HTTP service
- FILTERS → Enhance/Customize QGIS Server with filter plugins
- SERVICES → Add a new *SERVICE*
- OGC APIs → Add a new *OGC API*

Embedding and standalone applications require using the QGIS Server Python API directly from another Python script or application. The remaining options are better suited for when you want to add custom features to a standard QGIS Server binary application (FCGI or development server): in this case you'll need to write a Python plugin for the server application and register your custom filters, services or APIs.

20.2 Server API basics

The fundamental classes involved in a typical QGIS Server application are:

- `QgsServer` the server instance (typically a single instance for the whole application life)
- `QgsServerRequest` the request object (typically recreated on each request)
- `QgsServer.handleRequest(request, response)` processes the request and populates the response

The QGIS Server FCGI or development server workflow can be summarized as follows:

```

1 initialize the QgsApplication
2 create the QgsServer
3 the main server loop waits forever for client requests:
4     for each incoming request:
5         create a QgsServerRequest request
6         create a QgsServerResponse response
7         call QgsServer.handleRequest(request, response)
8             filter plugins may be executed
9         send the output to the client

```

Inside the `QgsServer.handleRequest(request, response)` method the filter plugins callbacks are called and `QgsServerRequest` and `QgsServerResponse` are made available to the plugins through the `QgsServerInterface` class.

Warning

QGIS server classes are not thread safe, you should always use a multiprocessing model or containers when building scalable applications based on QGIS Server API.

20.3 Standalone or embedding

For standalone server applications or embedding, you will need to use the above mentioned server classes directly, wrapping them up into a web server implementation that manages all the HTTP protocol interactions with the client.

A minimal example of the QGIS Server API usage (without the HTTP part) follows:

```

1 from qgis.core import QgsApplication
2 from qgis.server import *
3 app = QgsApplication([], False)
4
5 # Create the server instance, it may be a single one that
6 # is reused on multiple requests
7 server = QgsServer()
8
9 # Create the request by specifying the full URL and an optional body
10 # (for example for POST requests)
11 request = QgsBufferServerRequest(
12     'http://localhost:8081/?MAP=/qgis-server/projects/helloworld.qgs' +
13     '&SERVICE=WMS&REQUEST=GetCapabilities')
14
15 # Create a response objects
16 response = QgsBufferServerResponse()
17
18 # Handle the request
19 server.handleRequest(request, response)
20
21 print(response.headers())
22 print(response.body().data().decode('utf8'))
23
24 app.exitQgis()

```

Here is a complete standalone application example developed for the continuous integrations testing on QGIS source code repository, it showcases a wide set of different plugin filters and authentication schemes (not mean for production because they were developed for testing purposes only but still interesting for learning): [qgis_wrapped_server.py](#)

20.4 Server plugins

Server python plugins are loaded once when the QGIS Server application starts and can be used to register filters, services or APIs.

The structure of a server plugin is very similar to their desktop counterpart, a `QgsServerInterface` object is made available to the plugins and the plugins can register one or more custom filters, services or APIs to the corresponding registry by using one of the methods exposed by the server interface.

20.4.1 Server filter plugins

Filters come in three different flavors and they can be instantiated by subclassing one of the classes below and by calling the corresponding method of `QgsServerInterface`:

Filter Type	Base Class	QgsServerInterface registration
I/O	<code>QgsServerFilter</code>	<code>registerFilter()</code>
Access Control	<code>QgsAccessControlFilter</code>	<code>registerAccessControl()</code>
Cache	<code>QgsServerCacheFilter</code>	<code>registerServerCache()</code>

I/O filters

I/O filters can modify the server input and output (the request and the response) of the core services (WMS, WFS etc.) allowing to do any kind of manipulation of the services workflow. It is possible for example to restrict the access to selected layers, to inject an XSL stylesheet to the XML response, to add a watermark to a generated WMS image and so on.

From this point, you might find useful a quick look to the [server plugins API docs](#).

Each filter should implement at least one of three callbacks:

- `onRequestReady()`
- `onResponseComplete()`
- `onSendResponse()`

All filters have access to the request/response object (`QgsRequestHandler`) and can manipulate all its properties (input/output) and raise exceptions (while in a quite particular way as we'll see below).

All these methods return a boolean value indicating if the call should be propagated to the subsequent filters. If one of these method returns `False` then the chain stop, otherwise the call will propagate to the next filter.

Here is the pseudo code showing how the server handles a typical request and when the filter's callbacks are called:

```

1 for each incoming request:
2     create GET/POST request handler
3     pass request to an instance of QgsServerInterface
4     call onRequestReady filters
5
6     if there is not a response:
7         if SERVICE is WMS/WFS/WCS:
8             create WMS/WFS/WCS service
9             call service's executeRequest
10            possibly call onSendResponse for each chunk of bytes
11            sent to the client by a streaming services (WFS)
12            call onResponseComplete
13            request handler sends the response to the client

```

The following paragraphs describe the available callbacks in details.

onRequestReady

This is called when the request is ready: incoming URL and data have been parsed and before entering the core services (WMS, WFS etc.) switch, this is the point where you can manipulate the input and perform actions like:

- authentication/authorization
- redirects
- add/remove certain parameters (typenames for example)
- raise exceptions

You could even substitute a core service completely by changing **SERVICE** parameter and hence bypassing the core service completely (not that this make much sense though).

onSendResponse

This is called whenever any partial output is flushed from response buffer (i.e to **FCGI** `stdout` if the fcgi server is used) and from there, to the client. This occurs when huge content is streamed (like WFS GetFeature). In this case `onSendResponse()` may be called multiple times.

Note that if the response is not streamed, then `onSendResponse()` will not be called at all.

In all case, the last (or unique) chunk will be sent to client after a call to `onResponseComplete()`.

Returning `False` will prevent flushing of data to the client. This is desirable when a plugin wants to collect all chunks from a response and examine or change the response in `onResponseComplete()`.

onResponseComplete

This is called once when core services (if hit) finish their process and the request is ready to be sent to the client. As discussed above, this method will be called before the last (or unique) chunk of data is sent to the client. For streaming services, multiple calls to `onSendResponse()` might have been called.

`onResponseComplete()` is the ideal place to provide new services implementation (WPS or custom services) and to perform direct manipulation of the output coming from core services (for example to add a watermark upon a WMS image).

Note that returning `False` will prevent the next plugins to execute `onResponseComplete()` but, in any case, prevent response to be sent to the client.

Raising exceptions from a plugin

Some work has still to be done on this topic: the current implementation can distinguish between handled and unhandled exceptions by setting a `QgsRequestHandler` property to an instance of `QgsMapServiceException`, this way the main C++ code can catch handled python exceptions and ignore unhandled exceptions (or better: log them).

This approach basically works but it is not very “pythonic”: a better approach would be to raise exceptions from python code and see them bubbling up into C++ loop for being handled there.

Writing a server plugin

A server plugin is a standard QGIS Python plugin as described in *Developing Python Plugins*, that just provides an additional (or alternative) interface: a typical QGIS desktop plugin has access to QGIS application through the `QgisInterface` instance, a server plugin has only access to a `QgsServerInterface` when it is executed within the QGIS Server application context.

To make QGIS Server aware that a plugin has a server interface, a special metadata entry is needed (in `metadata.txt`):

```
server=True
```

Important

Only plugins that have the `server=True` metadata set will be loaded and executed by QGIS Server.

The `qgis3-server-vagrant` example plugin discussed here (with many more) is available on github, a few server plugins are also published in the official [QGIS plugins repository](#).

Plugin files

Here's the directory structure of our example server plugin.

```
1 PYTHON_PLUGINS_PATH/
2   HelloServer/
3     __init__.py    --> *required*
4     HelloServer.py --> *required*
5     metadata.txt   --> *required*
```

`__init__.py`

This file is required by Python's import system. Also, QGIS Server requires that this file contains a `serverClassFactory()` function, which is called when the plugin gets loaded into QGIS Server when the server starts. It receives reference to instance of `QgsServerInterface` and must return instance of your plugin's class. This is how the example plugin `__init__.py` looks like:

```
def serverClassFactory(serverIface):
    from .HelloServer import HelloServerServer
    return HelloServerServer(serverIface)
```

`HelloServer.py`

This is where the magic happens and this is how magic looks like: (e.g. `HelloServer.py`)

A server plugin typically consists in one or more callbacks packed into instances of a `QgsServerFilter`.

Each `QgsServerFilter` implements one or more of the following callbacks:

- `onRequestReady()`
- `onResponseComplete()`
- `onSendResponse()`

The following example implements a minimal filter which prints *HelloServer!* in case the **SERVICE** parameter equals to "HELLO":

```

1 class HelloFilter(QgsServerFilter):
2
3     def __init__(self, serverIface):
4         super().__init__(serverIface)
5
6     def onRequestReady(self) -> bool:
7         QgsMessageLog.logMessage("HelloFilter.onRequestReady")
8         return True
9
10    def onSendResponse(self) -> bool:
11        QgsMessageLog.logMessage("HelloFilter.onSendResponse")
12        return True
13
14    def onResponseComplete(self) -> bool:
15        QgsMessageLog.logMessage("HelloFilter.onResponseComplete")
16        request = self.serverInterface().requestHandler()
17        params = request.parameterMap()
18        if params.get('SERVICE', '').upper() == 'HELLO':
19            request.clear()
20            request.setResponseHeader('Content-type', 'text/plain')
21            # Note that the content is of type "bytes"
22            request.appendBody(b'HelloServer!')
23        return True

```

The filters must be registered into the **serverIface** as in the following example:

```

class HelloServerServer:
    def __init__(self, serverIface):
        serverIface.registerFilter(HelloFilter(serverIface), 100)

```

The second parameter of `registerFilter()` sets a priority which defines the order for the callbacks with the same name (the lower priority is invoked first).

By using the three callbacks, plugins can manipulate the input and/or the output of the server in many different ways. In every moment, the plugin instance has access to the `QgsRequestHandler` through the `QgsServerInterface`. The `QgsRequestHandler` class has plenty of methods that can be used to alter the input parameters before entering the core processing of the server (by using `requestReady()`) or after the request has been processed by the core services (by using `sendResponse()`).

The following examples cover some common use cases:

Modifying the input

The example plugin contains a test example that changes input parameters coming from the query string, in this example a new parameter is injected into the (already parsed) `parameterMap`, this parameter is then visible by core services (WMS etc.), at the end of core services processing we check that the parameter is still there:

```

1 class ParamsFilter(QgsServerFilter):
2
3     def __init__(self, serverIface):
4         super(ParamsFilter, self).__init__(serverIface)
5
6     def onRequestReady(self) -> bool:
7         request = self.serverInterface().requestHandler()
8         params = request.parameterMap()
9         request.setParameter('TEST_NEW_PARAM', 'ParamsFilter')
10        return True
11
12    def onResponseComplete(self) -> bool:
13        request = self.serverInterface().requestHandler()

```

(continues on next page)

(continued from previous page)

```

14     params = request.parameterMap( )
15     if params.get('TEST_NEW_PARAM') == 'ParamsFilter':
16         QgsMessageLog.logMessage("SUCCESS - ParamsFilter.onResponseComplete")
17     else:
18         QgsMessageLog.logMessage("FAIL - ParamsFilter.onResponseComplete")
19     return True

```

This is an extract of what you see in the log file:

```

1  src/core/qgsmessageolog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 plugin[0]␣
   ↳HelloServerServer - loading filter ParamsFilter
2  src/core/qgsmessageolog.cpp: 45: (logMessage) [1ms] 2014-12-12T12:39:29 Server[0]␣
   ↳Server plugin HelloServer loaded!
3  src/core/qgsmessageolog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 Server[0]␣
   ↳Server python plugins loaded
4  src/mapserver/qgshttprequesthandler.cpp: 547: (requestStringToParameterMap) [1ms]␣
   ↳inserting pair SERVICE // HELLO into the parameter map
5  src/mapserver/qgsserverfilter.cpp: 42: (onRequestReady) [0ms] QgsServerFilter␣
   ↳plugin default onRequestReady called
6  src/core/qgsmessageolog.cpp: 45: (logMessage) [0ms] 2014-12-12T12:39:29 plugin[0]␣
   ↳SUCCESS - ParamsFilter.onResponseComplete

```

On the highlighted line the “SUCCESS” string indicates that the plugin passed the test.

The same technique can be exploited to use a custom service instead of a core one: you could for example skip a **WFS SERVICE** request or any other core request just by changing the **SERVICE** parameter to something different and the core service will be skipped. Then you can inject your custom results into the output and send them to the client (this is explained below).

💡 Tip

If you really want to implement a custom service it is recommended to subclass `QgsService` and register your service on `registerFilter()` by calling its `registerService(service)`

Modifying or replacing the output

The watermark filter example shows how to replace the WMS output with a new image obtained by adding a watermark image on the top of the WMS image generated by the WMS core service:

```

1  from qgis.server import *
2  from qgis.PyQt.QtCore import *
3  from qgis.PyQt.QtGui import *
4
5  class WatermarkFilter(QgsServerFilter):
6
7      def __init__(self, serverIface):
8          super().__init__(serverIface)
9
10     def onResponseComplete(self) -> bool:
11         request = self.serverInterface().requestHandler()
12         params = request.parameterMap( )
13         # Do some checks
14         if (params.get('SERVICE').upper() == 'WMS' \
15             and params.get('REQUEST').upper() == 'GETMAP' \
16             and not request.exceptionRaised() ):
17             QgsMessageLog.logMessage("WatermarkFilter.onResponseComplete: image␣
   ↳ready %s" % request.parameter("FORMAT"))
18             # Get the image

```

(continues on next page)

(continued from previous page)

```

19         img = QImage()
20         img.loadFromData(request.body())
21         # Adds the watermark
22         watermark = QImage(os.path.join(os.path.dirname(__file__), 'media/
↪watermark.png'))
23         p = QPainter(img)
24         p.drawImage(QRect( 20, 20, 40, 40), watermark)
25         p.end()
26         ba = QByteArray()
27         buffer = QBuffer(ba)
28         buffer.open(QIODevice.WriteOnly)
29         img.save(buffer, "PNG" if "png" in request.parameter("FORMAT") else
↪"JPG")
30         # Set the body
31         request.clearBody()
32         request.appendBody(ba)
33         return True

```

In this example the **SERVICE** parameter value is checked and if the incoming request is a **WMS GETMAP** and no exceptions have been set by a previously executed plugin or by the core service (WMS in this case), the WMS generated image is retrieved from the output buffer and the watermark image is added. The final step is to clear the output buffer and replace it with the newly generated image. Please note that in a real-world situation we should also check for the requested image type instead of supporting PNG or JPG only.

Access control filters

Access control filters gives the developer a fine-grained control over which layers, features and attributes can be accessed, the following callbacks can be implemented in an access control filter:

- `layerFilterExpression(layer)`
- `layerFilterSubsetString(layer)`
- `layerPermissions(layer)`
- `authorizedLayerAttributes(layer, attributes)`
- `allowToEdit(layer, feature)`
- `cacheKey()`

Plugin files

Here's the directory structure of our example plugin:

```

1 PYTHON_PLUGINS_PATH/
2   MyAccessControl/
3     __init__.py    --> *required*
4     AccessControl.py --> *required*
5     metadata.txt   --> *required*

```

__init__.py

This file is required by Python's import system. As for all QGIS server plugins, this file contains a `serverClassFactory()` function, which is called when the plugin gets loaded into QGIS Server at startup. It receives a reference to an instance of `QgsServerInterface` and must return an instance of your plugin's class. This is how the example plugin `__init__.py` looks like:

```
def serverClassFactory(serverIface):
    from MyAccessControl.AccessControl import AccessControlServer
    return AccessControlServer(serverIface)
```

AccessControl.py

```
1 class AccessControlFilter(QgsAccessControlFilter):
2
3     def __init__(self, server_iface):
4         super().__init__(server_iface)
5
6     def layerFilterExpression(self, layer):
7         """ Return an additional expression filter """
8         return super().layerFilterExpression(layer)
9
10    def layerFilterSubsetString(self, layer):
11        """ Return an additional subset string (typically SQL) filter """
12        return super().layerFilterSubsetString(layer)
13
14    def layerPermissions(self, layer):
15        """ Return the layer rights """
16        return super().layerPermissions(layer)
17
18    def authorizedLayerAttributes(self, layer, attributes):
19        """ Return the authorised layer attributes """
20        return super().authorizedLayerAttributes(layer, attributes)
21
22    def allowToEdit(self, layer, feature):
23        """ Are we authorised to modify the following geometry """
24        return super().allowToEdit(layer, feature)
25
26    def cacheKey(self):
27        return super().cacheKey()
28
29 class AccessControlServer:
30
31     def __init__(self, serverIface):
32         """ Register AccessControlFilter """
33         serverIface.registerAccessControl(AccessControlFilter(serverIface), 100)
```

This example gives a full access for everybody.

It's the role of the plugin to know who is logged on.

On all those methods we have the layer on argument to be able to customise the restriction per layer.

layerFilterExpression

Used to add an Expression to limit the results.

For example, to limit to features where the attribute `role` is equal to `user`.

```
def layerFilterExpression(self, layer):  
    return "$role = 'user'"
```

layerFilterSubsetString

Same than the previous but use the `SubsetString` (executed in the database)

For example, to limit to features where the attribute `role` is equal to `user`.

```
def layerFilterSubsetString(self, layer):  
    return "role = 'user'"
```

layerPermissions

Limit the access to the layer.

Return an object of type `LayerPermissions()`, which has the properties:

- `canRead` to see it in the `GetCapabilities` and have read access.
- `canInsert` to be able to insert a new feature.
- `canUpdate` to be able to update a feature.
- `canDelete` to be able to delete a feature.

For example, to limit everything on read only access:

```
1 def layerPermissions(self, layer):  
2     rights = QgsAccessControlFilter.LayerPermissions()  
3     rights.canRead = True  
4     rights.canInsert = rights.canUpdate = rights.canDelete = False  
5     return rights
```

authorizedLayerAttributes

Used to limit the visibility of a specific subset of attribute.

The argument `attribute` return the current set of visible attributes.

For example, to hide the `role` attribute:

```
def authorizedLayerAttributes(self, layer, attributes):  
    return [a for a in attributes if a != "role"]
```

allowToEdit

This is used to limit the editing on a subset of features.

It is used in the WFS-Transaction protocol.

For example, to be able to edit only feature that has the attribute `role` with the value `user`:

```
def allowToEdit(self, layer, feature):
    return feature.attribute('role') == 'user'
```

cacheKey

QGIS Server maintains a cache of the capabilities then to have a cache per role you can return the role in this method. Or return `None` to completely disable the cache.

20.4.2 Custom services

In QGIS Server, core services such as WMS, WFS and WCS are implemented as subclasses of `QgsService`.

To implement a new service that will be executed when the query string parameter `SERVICE` matches the service name, you can implement your own `QgsService` and register your service on the `serviceRegistry()` by calling its `registerService(service)`.

Here is an example of a custom service named `CUSTOM`:

```
1 from qgis.server import QgsService
2 from qgis.core import QgsMessageLog
3
4 class CustomServiceService(QgsService):
5
6     def __init__(self):
7         QgsService.__init__(self)
8
9     def name(self):
10        return "CUSTOM"
11
12    def version(self):
13        return "1.0.0"
14
15    def executeRequest(self, request, response, project):
16        response.setStatusCode(200)
17        QgsMessageLog.logMessage('Custom service executeRequest')
18        response.write("Custom service executeRequest")
19
20
21 class CustomService():
22
23     def __init__(self, serverIface):
24         serverIface.serviceRegistry().registerService(CustomServiceService())
```

20.4.3 Custom APIs

In QGIS Server, core OGC APIs such OAPIF (aka WFS3) are implemented as collections of `QgsServerOgcApiHandler` subclasses that are registered to an instance of `QgsServerOgcApi` (or its parent class `QgsServerApi`).

To implement a new API that will be executed when the url path matches a certain URL, you can implement your own `QgsServerOgcApiHandler` instances, add them to an `QgsServerOgcApi` and register the API on the `serviceRegistry()` by calling its `registerApi(api)`.

Here is an example of a custom API that will be executed when the URL contains `/customapi`:

```

1 import json
2 import os
3
4 from qgis.PyQt.QtCore import QBuffer, QIODevice, QTextStream, QRegularExpression
5 from qgis.server import (
6     QgsServiceRegistry,
7     QgsService,
8     QgsServerFilter,
9     QgsServerOgcApi,
10    QgsServerQueryStringParameter,
11    QgsServerOgcApiHandler,
12 )
13
14 from qgis.core import (
15     QgsMessageLog,
16     QgsJsonExporter,
17     QgsCircle,
18     QgsFeature,
19     QgsPoint,
20     QgsGeometry,
21 )
22
23
24 class CustomApiHandler(QgsServerOgcApiHandler):
25
26     def __init__(self):
27         super(CustomApiHandler, self).__init__()
28         self.setContentTypes([QgsServerOgcApi.HTML, QgsServerOgcApi.JSON])
29
30     def path(self):
31         return QRegularExpression("/customapi")
32
33     def operationId(self):
34         return "CustomApiXYCircle"
35
36     def summary(self):
37         return "Creates a circle around a point"
38
39     def description(self):
40         return "Creates a circle around a point"
41
42     def linkTitle(self):
43         return "Custom Api XY Circle"
44
45     def linkType(self):
46         return QgsServerOgcApi.data
47
48     def handleRequest(self, context):
49         """Simple Circle"""
50
51         values = self.values(context)

```

(continues on next page)

(continued from previous page)

```

52     x = values['x']
53     y = values['y']
54     r = values['r']
55     f = QgsFeature()
56     f.setAttributes([x, y, r])
57     f.setGeometry(QgsCircle(QgsPoint(x, y), r).toCircularString())
58     exporter = QgsJsonExporter()
59     self.write(json.loads(exporter.exportFeature(f)), context)
60
61     def templatePath(self, context):
62         # The template path is used to serve HTML content
63         return os.path.join(os.path.dirname(__file__), 'circle.html')
64
65     def parameters(self, context):
66         return [QgsServerQueryStringParameter('x', True,
↪QgsServerQueryStringParameter.Type.Double, 'X coordinate'),
67                 QgsServerQueryStringParameter(
68                     'y', True, QgsServerQueryStringParameter.Type.Double, 'Y
↪coordinate'),
69                 QgsServerQueryStringParameter('r', True,
↪QgsServerQueryStringParameter.Type.Double, 'radius')]
70
71
72 class CustomApi():
73
74     def __init__(self, serverIface):
75         api = QgsServerOgcApi(serverIface, '/customapi',
76                               'custom api', 'a custom api', '1.1')
77         handler = CustomApiHandler()
78         api.registerHandler(handler)
79         serverIface.serviceRegistry().registerApi(api)

```


CHEAT SHEET FOR PYQGIS

Hint

The code snippets on this page need the following imports if you're outside the pyqgis console:

```
1 from qgis.PyQt.QtCore import (
2     QRectF,
3 )
4
5 from qgis.core import (
6     Qgis,
7     QgsProject,
8     QgsLayerTreeModel,
9 )
10
11 from qgis.gui import (
12     QgsLayerTreeView,
13 )
```

21.1 User Interface

Change Look & Feel

```
1 from qgis.PyQt.QtWidgets import QApplication
2
3 app = QApplication.instance()
4 app.setStyleSheet(".QWidget {color: blue; background-color: yellow;}")
5 # You can even read the stylesheet from a file
6 with open("testdata/file.qss") as qss_file_content:
7     app.setStyleSheet(qss_file_content.read())
```

Change icon and title

```
1 from qgis.PyQt.QtGui import QIcon
2
3 icon = QIcon("/path/to/logo/file.png")
4 iface.mainWindow().setWindowIcon(icon)
5 iface.mainWindow().setWindowTitle("My QGIS")
```

21.2 Settings

Get QgsSettings list

```
1 from qgis.core import QgsSettings
2
3 qs = QgsSettings()
4
5 for k in sorted(qs.allKeys()):
6     print (k)
```

21.3 Toolbars

Remove toolbar

```
1 toolbar = iface.helpToolBar()
2 parent = toolbar.parentWidget()
3 parent.removeToolBar(toolbar)
4
5 # and add again
6 parent.addToolBar(toolbar)
```

Remove actions toolbar

```
actions = iface.attributesToolBar().actions()
iface.attributesToolBar().clear()
iface.attributesToolBar().addAction(actions[4])
iface.attributesToolBar().addAction(actions[3])
```

21.4 Menus

Remove menu

```
1 # for example Help Menu
2 menu = iface.helpMenu()
3 menubar = menu.parentWidget()
4 menubar.removeAction(menu.menuAction())
5
6 # and add again
7 menubar.addAction(menu.menuAction())
```

21.5 Canvas

Access canvas

```
canvas = iface.mapCanvas()
```

Change canvas color

```
from qgis.PyQt.QtCore import Qt

iface.mapCanvas().setCanvasColor(Qt.black)
iface.mapCanvas().refresh()
```

Map Update interval

```
from qgis.core import QgsSettings
# Set milliseconds (150 milliseconds)
QgsSettings().setValue("/qgis/map_update_interval", 150)
```

21.6 Layers

Add vector layer

```
layer = iface.addVectorLayer("testdata/data/data.gpkg|layername=airports",
    ↪ "Airports layer", "ogr")
if not layer or not layer.isValid():
    print("Layer failed to load!")
```

Get active layer

```
layer = iface.activeLayer()
```

List all layers

```
from qgis.core import QgsProject

QgsProject.instance().mapLayers().values()
```

Obtain layers name

```
1 from qgis.core import QgsVectorLayer
2 layer = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like", "memory")
3 QgsProject.instance().addMapLayer(layer)
4
5 layers_names = []
6 for layer in QgsProject.instance().mapLayers().values():
7     layers_names.append(layer.name())
8
9 print("layers TOC = {}".format(layers_names))
```

```
layers TOC = ['layer name you like']
```

Otherwise

```
layers_names = [layer.name() for layer in QgsProject.instance().mapLayers().
    ↪ values()]
print("layers TOC = {}".format(layers_names))
```

```
layers TOC = ['layer name you like']
```

Find layer by name

```
from qgis.core import QgsProject

layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
print(layer.name())
```

```
layer name you like
```

Set active layer

```
from qgis.core import QgsProject

layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
iface.setActiveLayer(layer)
```

Refresh layer at interval

```
1 from qgis.core import QgsProject
2
3 layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
4 # Set seconds (5 seconds)
5 layer.setAutoRefreshInterval(5000)
6 # Enable data reloading
7 layer.setAutoRefreshMode(Qgis.AutoRefreshMode.ReloadData)
```

Show methods

```
dir(layer)
```

Adding new feature with feature form

```
1 from qgis.core import QgsFeature, QgsGeometry
2
3 feat = QgsFeature()
4 geom = QgsGeometry()
5 feat.setGeometry(geom)
6 feat.setFields(layer.fields())
7
8 iface.openFeatureForm(layer, feat, False)
```

Adding new feature without feature form

```
1 from qgis.core import QgsGeometry, QgsPointXY, QgsFeature
2
3 pr = layer.dataProvider()
4 feat = QgsFeature()
5 feat.setGeometry(QgsGeometry.fromPointXY(QgsPointXY(10,10)))
6 pr.addFeatures([feat])
```

Get features

```
for f in layer.getFeatures():
    print (f)
```

```
<qgis._core.QgsFeature object at 0x7f45cc64b678>
```

Get selected features

```
for f in layer.selectedFeatures():
    print (f)
```

Get selected features Ids

```
selected_ids = layer.selectedFeatureIds()
print(selected_ids)
```

Create a memory layer from selected features Ids

```
from qgis.core import QgsFeatureRequest

memory_layer = layer.materialize(QgsFeatureRequest().setFilterFids(layer.
```

(continues on next page)

(continued from previous page)

```

↪selectedFeatureIds())
QgsProject.instance().addMapLayer(memory_layer)

```

Get geometry

```

# Point layer
for f in layer.getFeatures():
    geom = f.geometry()
    print('%f, %f' % (geom.asPoint().y(), geom.asPoint().x()))

```

```
10.000000, 10.000000
```

Move geometry

```

1 from qgis.core import QgsFeature, QgsGeometry
2 poly = QgsFeature()
3 geom = QgsGeometry.fromWkt("POINT(7 45)")
4 geom.translate(1, 1)
5 poly.setGeometry(geom)
6 print(poly.geometry())

```

```
<QgsGeometry: Point (8 46)>
```

Set the CRS

```

from qgis.core import QgsProject, QgsCoordinateReferenceSystem

for layer in QgsProject.instance().mapLayers().values():
    layer.setCrs(QgsCoordinateReferenceSystem('EPSG:4326'))

```

See the CRS

```

1 from qgis.core import QgsProject
2
3 for layer in QgsProject.instance().mapLayers().values():
4     crs = layer.crs().authid()
5     layer.setName('{} ({} )'.format(layer.name(), crs))

```

Hide a field column

```

1 from qgis.core import QgsEditorWidgetSetup
2
3 def fieldVisibility (layer, fname):
4     setup = QgsEditorWidgetSetup('Hidden', {})
5     for i, column in enumerate(layer.fields()):
6         if column.name() == fname:
7             layer.setEditorWidgetSetup(idx, setup)
8             break
9     else:
10        continue

```

Layer from WKT

```

1 from qgis.core import QgsVectorLayer, QgsFeature, QgsGeometry, QgsProject
2
3 layer = QgsVectorLayer('Polygon?crs=epsg:4326', 'Mississippi', 'memory')
4 pr = layer.dataProvider()
5 poly = QgsFeature()
6 geom = QgsGeometry.fromWkt("POLYGON ((-88.82 34.99,-88.09 34.89,-88.39 30.34,-89.
↪57 30.18,-89.73 31,-91.63 30.99,-90.87 32.37,-91.23 33.44,-90.93 34.23,-90.30 34.

```

(continues on next page)

(continued from previous page)

```

6 ↪ 99, -88.82 34.99)) ")
7 poly.setGeometry(geom)
8 pr.addFeatures([poly])
9 layer.updateExtents()
10 QgsProject.instance().addMapLayers([layer])

```

Load all vector layers from GeoPackage

```

1 from qgis.core import QgsDataProvider
2
3 fileName = "testdata/sublayers.gpkg"
4 layer = QgsVectorLayer(fileName, "test", "ogr")
5 subLayers = layer.dataProvider().subLayers()
6
7 for subLayer in subLayers:
8     name = subLayer.split(QgsDataProvider.SUBLAYER_SEPARATOR)[1]
9     uri = "%s|layername=%s" % (fileName, name,)
10    # Create layer
11    sub_vlayer = QgsVectorLayer(uri, name, 'ogr')
12    # Add layer to map
13    QgsProject.instance().addMapLayer(sub_vlayer)

```

Load tile layer (XYZ-Layer)

```

1 from qgis.core import QgsRasterLayer, QgsProject
2
3 def loadXYZ(url, name):
4     rasterLyr = QgsRasterLayer("type=xyz&url=" + url, name, "wms")
5     QgsProject.instance().addMapLayer(rasterLyr)
6
7 urlWithParams = 'https://tile.openstreetmap.org/%7Bz%7D/%7Bx%7D/%7By%7D.png&
8 ↪ zmax=19&zmin=0&crs=EPSG3857'
9 loadXYZ(urlWithParams, 'OpenStreetMap')

```

Remove all layers

```
QgsProject.instance().removeAllMapLayers()
```

Remove all

```
QgsProject.instance().clear()
```

21.7 Table of contents

Access checked layers

```
iface.mapCanvas().layers()
```

Remove contextual menu

```

1 ltv = iface.layerTreeView()
2 mp = ltv.menuProvider()
3 ltv.setMenuProvider(None)
4 # Restore
5 ltv.setMenuProvider(mp)

```

21.8 Advanced TOC

Root node

```

1 from qgis.core import QgsVectorLayer, QgsProject, QgsLayerTreeLayer
2
3 root = QgsProject.instance().layerTreeRoot()
4 node_group = root.addGroup("My Group")
5
6 layer = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like", "memory")
7 QgsProject.instance().addMapLayer(layer, False)
8
9 node_group.addLayer(layer)
10
11 print(root)
12 print(root.children())

```

Access the first child node

```

1 from qgis.core import QgsLayerTreeGroup, QgsLayerTreeLayer, QgsLayerTree
2
3 child0 = root.children()[0]
4 print (child0.name())
5 print (type(child0))
6 print (isinstance(child0, QgsLayerTreeLayer))
7 print (isinstance(child0.parent(), QgsLayerTree))

```

```

My Group
<class 'qgis._core.QgsLayerTreeGroup'>
False
True

```

Find groups and nodes

```

1 from qgis.core import QgsLayerTreeGroup, QgsLayerTreeLayer
2
3 def get_group_layers(group):
4     print('- group: ' + group.name())
5     for child in group.children():
6         if isinstance(child, QgsLayerTreeGroup):
7             # Recursive call to get nested groups
8             get_group_layers(child)
9         else:
10            print(' - layer: ' + child.name())
11
12
13 root = QgsProject.instance().layerTreeRoot()
14 for child in root.children():
15     if isinstance(child, QgsLayerTreeGroup):
16         get_group_layers(child)
17     elif isinstance(child, QgsLayerTreeLayer):
18         print ('- layer: ' + child.name())

```

```

- group: My Group
- layer: layer name you like

```

Find group by name

```
print (root.findGroup("My Group"))
```

```
<QgsLayerTreeGroup: My Group>
```

Find layer by id

```
print(root.findLayer(layer.id()))
```

```
<QgsLayerTreeLayer: layer name you like>
```

Add layer

```
1 from qgis.core import QgsVectorLayer, QgsProject
2
3 layer1 = QgsVectorLayer("Point?crs=EPSG:4326", "layer name you like 2", "memory")
4 QgsProject.instance().addMapLayer(layer1, False)
5 node_layer1 = root.addLayer(layer1)
6 # Remove it
7 QgsProject.instance().removeMapLayer(layer1)
```

Add group

```
1 from qgis.core import QgsLayerTreeGroup
2
3 node_group2 = QgsLayerTreeGroup("Group 2")
4 root.addChildNode(node_group2)
5 QgsProject.instance().mapLayersByName("layer name you like")[0]
```

Move loaded layer

```
1 layer = QgsProject.instance().mapLayersByName("layer name you like")[0]
2 root = QgsProject.instance().layerTreeRoot()
3
4 myLayer = root.findLayer(layer.id())
5 myClone = myLayer.clone()
6 parent = myLayer.parent()
7
8 myGroup = root.findGroup("My Group")
9 # Insert in first position
10 myGroup.insertChildNode(0, myClone)
11
12 parent.removeChildNode(myLayer)
```

Move loaded layer to a specific group

```
1 QgsProject.instance().addMapLayer(layer, False)
2
3 root = QgsProject.instance().layerTreeRoot()
4 myGroup = root.findGroup("My Group")
5 myOriginalLayer = root.findLayer(layer.id())
6 myLayer = myOriginalLayer.clone()
7 myGroup.insertChildNode(0, myLayer)
8 parent.removeChildNode(myOriginalLayer)
```

Toggling active layer visibility

```
root = QgsProject.instance().layerTreeRoot()
node = root.findLayer(layer.id())
new_state = Qt.Checked if node.isVisible() == Qt.Unchecked else Qt.Unchecked
node.setItemVisibilityChecked(new_state)
```

Is group selected

```

1 def isMyGroupSelected( groupName ):
2     myGroup = QgsProject.instance().layerTreeRoot().findGroup( groupName )
3     return myGroup in iface.layerTreeView().selectedNodes()
4
5 print(isMyGroupSelected( 'my group name' ))

```

```
False
```

Expand node

```

print(myGroup.isExpanded())
myGroup.setExpanded(False)

```

Hidden node trick

```

1 from qgis.core import QgsProject
2
3 model = iface.layerTreeView().layerTreeModel()
4 ltv = iface.layerTreeView()
5 root = QgsProject.instance().layerTreeRoot()
6
7 layer = QgsProject.instance().mapLayersByName('layer name you like')[0]
8 node = root.findLayer(layer.id())
9
10 index = model.node2index( node )
11 ltv.setRowHidden( index.row(), index.parent(), True )
12 node.setCustomProperty( 'nodeHidden', 'true' )
13 ltv.setCurrentIndex(model.node2index(root))

```

Node signals

```

1 def onWillAddChildren(node, indexFrom, indexTo):
2     print ("WILL ADD", node, indexFrom, indexTo)
3
4 def onAddedChildren(node, indexFrom, indexTo):
5     print ("ADDED", node, indexFrom, indexTo)
6
7 root.willAddChildren.connect(onWillAddChildren)
8 root.addedChildren.connect(onAddedChildren)

```

Remove layer

```
root.removeLayer(layer)
```

Remove group

```
root.removeChildNode(node_group2)
```

Create new table of contents (TOC)

```

1 root = QgsProject.instance().layerTreeRoot()
2 model = QgsLayerTreeModel(root)
3 view = QgsLayerTreeView()
4 view.setModel(model)
5 view.show()

```

Move node

```

cloned_group1 = node_group.clone()
root.insertChildNode(0, cloned_group1)
root.removeChildNode(node_group)

```

Rename node

```
cloned_group1.setName("Group X")
node_layer1.setName("Layer X")
```

21.9 Processing algorithms

Get algorithms list

```
1 from qgis.core import QgsApplication
2
3 for alg in QgsApplication.processingRegistry().algorithms():
4     if 'buffer' == alg.name():
5         print("{}: {} --> {}".format(alg.provider().name(), alg.name(), alg.
↳displayName()))
```

```
QGIS (native c++):buffer --> Buffer
```

Get algorithms help

Random selection

```
from qgis import processing
processing.algorithmHelp("native:buffer")
```

```
...
```

Run the algorithm

For this example, the result is stored in a temporary memory layer which is added to the project.

```
from qgis import processing
result = processing.run("native:buffer", {'INPUT': layer, 'OUTPUT': 'memory:'})
QgsProject.instance().addMapLayer(result['OUTPUT'])
```

```
Processing(0): Results: {'OUTPUT': 'output_d27a2008_970c_4687_b025_f057abbd7319'}
```

How many algorithms are there?

```
len(QgsApplication.processingRegistry().algorithms())
```

How many providers are there?

```
from qgis.core import QgsApplication

len(QgsApplication.processingRegistry().providers())
```

How many expressions are there?

```
from qgis.core import QgsExpression

len(QgsExpression.Functions())
```

21.10 Decorators

CopyRight

```

1  from qgis.PyQt.Qt import QTextDocument
2  from qgis.PyQt.QtGui import QFont
3
4  mQFont = "Sans Serif"
5  mQFontsize = 9
6  mLabelQString = "© QGIS 2019"
7  mMarginHorizontal = 0
8  mMarginVertical = 0
9  mLabelQColor = "#FF0000"
10
11  INCHES_TO_MM = 0.0393700787402 # 1 millimeter = 0.0393700787402 inches
12  case = 2
13
14  def add_copyright(p, text, xOffset, yOffset):
15      p.translate( xOffset , yOffset )
16      text.drawContents(p)
17      p.setWorldTransform( p.worldTransform() )
18
19  def _on_render_complete(p):
20      deviceHeight = p.device().height() # Get paint device height on which this_
↳painter is currently painting
21      deviceWidth  = p.device().width() # Get paint device width on which this_
↳painter is currently painting
22      # Create new container for structured rich text
23      text = QTextDocument()
24      font = QFont()
25      font.setFamily(mQFont)
26      font.setPointSize(int(mQFontsize))
27      text.setDefaultFont(font)
28      style = "<style type='text/css'> p {color: " + mLabelQColor + "}</style>"
29      text.setHtml( style + "<p>" + mLabelQString + "</p>" )
30      # Text Size
31      size = text.size()
32
33      # RenderMillimeters
34      pixelsInchX = p.device().logicalDpiX()
35      pixelsInchY = p.device().logicalDpiY()
36      xOffset = pixelsInchX * INCHES_TO_MM * int(mMarginHorizontal)
37      yOffset = pixelsInchY * INCHES_TO_MM * int(mMarginVertical)
38
39      # Calculate positions
40      if case == 0:
41          # Top Left
42          add_copyright(p, text, xOffset, yOffset)
43
44      elif case == 1:
45          # Bottom Left
46          yOffset = deviceHeight - yOffset - size.height()
47          add_copyright(p, text, xOffset, yOffset)
48
49      elif case == 2:
50          # Top Right
51          xOffset = deviceWidth - xOffset - size.width()
52          add_copyright(p, text, xOffset, yOffset)
53
54      elif case == 3:
55          # Bottom Right
56          yOffset = deviceHeight - yOffset - size.height()

```

(continues on next page)

(continued from previous page)

```
57         xOffset = deviceWidth - xOffset - size.width()
58         add_copyright(p, text, xOffset, yOffset)
59
60     elif case == 4:
61         # Top Center
62         xOffset = deviceWidth / 2
63         add_copyright(p, text, xOffset, yOffset)
64
65     else:
66         # Bottom Center
67         yOffset = deviceHeight - yOffset - size.height()
68         xOffset = deviceWidth / 2
69         add_copyright(p, text, xOffset, yOffset)
70
71     # Emitted when the canvas has rendered
72     iface.mapCanvas().renderComplete.connect(_on_render_complete)
73     # Repaint the canvas map
74     iface.mapCanvas().refresh()
```

21.11 Composer

Get print layout by name

```
1 composerTitle = 'MyComposer' # Name of the composer
2
3 project = QgsProject.instance()
4 projectLayoutManager = project.layoutManager()
5 layout = projectLayoutManager.layoutByName(composerTitle)
```

21.12 Sources

- [QGIS Python \(PyQGIS\) API](#)
- [QGIS C++ API](#)
- [StackOverFlow QGIS questions](#)
- [Script by Klas Karlsson](#)